



Republic of Tunisia
Ministry of Higher Education and Scientific Research
University of Tunis El Manar
National Engineering School of Tunis



Department of Industrial Engineering

Engineering Internship Report

Implementation of a Strategic and Preventive Cutting Tool Lifecycle Tracking System

Prepared by:

Abderrahmen JAWEDI

Othmen BOURBIA

Class: 3rd Year Industrial Engineering

Hosted at:



Company Supervisor:

Mrs. Mouna DHAOUEDI

Duration:

6 Weeks

Academic Year: 2026–2027

Acknowledgements

I would like to express my deepest gratitude to my company supervisor, Mrs. **Mouna Dhaouedi**, for her availability, rigor, invaluable advice, and field expertise, which have significantly enriched the practical dimension of this work throughout this internship.

I would also like to thank the entire staff at **Figeac Aéro Tunisia** for their warm welcome and professionalism.

My thanks extend to the faculty and academic staff of the National Engineering School of Tunis for the high quality of education provided.

Finally, I address my sincere appreciation to my family for their unwavering support and encouragement throughout this project.

Abstract

This project, carried out at **Figeac Aéro Tunisie (FGAT)**, presents a strategic management and predictive system for cutting tools used in aeronautical manufacturing. The developed approach covers the complete cutting tool lifecycle, from procurement to end-of-life, integrating traceability mechanisms and predictive models to anticipate tool wear and optimize replacement strategies. Key performance indicators (KPIs) were defined and monitored through a dedicated dashboard to support data-driven decision making on the shop floor.

Keywords: Cutting Tool Management, Predictive Maintenance, Traceability, Aeronautical Manufacturing, DMAIC, KPI, Lean Manufacturing, FGAT.

Résumé

Ce projet, réalisé au sein de **Figeac Aéro Tunisie (FGAT)**, présente un système de gestion stratégique et prédictive des outils coupants utilisés en fabrication aéronautique. L'approche développée couvre le cycle de vie complet des outils coupants, de l'approvisionnement jusqu'à la mise au rebut, en intégrant des mécanismes de traçabilité et des modèles prédictifs pour anticiper l'usure des outils et optimiser les stratégies de remplacement. Des indicateurs de performance (KPIs) ont été définis et suivis à travers un tableau de bord dédié pour appuyer la prise de décision sur le terrain.

Mots-clés : Gestion des outils coupants, Maintenance prédictive, Traçabilité, Fabrication aéronautique, DMAIC, KPI, Lean Manufacturing, FGAT. [english]babel

List of Figures

1.1	International Footprint of FGA Group	3
1.2	Titre de votre figure	4
1.3	Primary Aerospace OEM Clients and Partners	4
1.4	FGAT Organizational Structure	6
1.5	GEDIX Tool Management System interface	8
1.6	Physical Tool Storage Rack with Generic Bin Classification	9
2.1	Detailed Ishikawa (6M) Diagram for Cutting Tool Degradation and Breakage.	15
2.2	UML Use Case Diagram for the Decision-Support Platform	19
3.1	6-Week Project Implementation Roadmap and Planning Gantt Chart.	22
3.2	Overall Architecture of the Cutting Tool Decision-Support Platform.	22
3.3	Entity-Relationship Model for Item-Level Tool Traceability.	24
3.4	End-to-End Analytical Traceability and Historical Data Flow Diagram.	27
3.5	State transition diagram of the Discrete-Time Markov Chain representing tool health states (S_0 to S_4), including state persistence, progressive wear, severe degradation jumps, and the absorbing state.	39
4.1	Progressive implementation workflow of the tool lifecycle monitoring platform.	48
4.2	Entity relationship chain centered around tool lifecycle sessions.	51
4.3	Flux d'agrégation des données de la session vers le calcul de l'IGSO.	54
4.4	Implemented authentication interface.	65
4.5	Main dashboard of the implemented monitoring platform.	66
4.6	Tool lifecycle monitoring interface.	67
4.7	Presentation of prediction information in the web interface.	67
4.8	Architectural data flow from database to execution pipeline.	68
4.9	End-to-end execution flow of a machining session request.	68
4.10	Sequence diagram of complete execution flow.	69
4.11	Horizontal Analytical Sequence from IGSO to State History.	70
4.12	End-to-end horizontal execution workflow from PostgreSQL session to Web interface.	71
5.1	Future extension toward circular cutting-tool lifecycle management.	78
5.2	Future adaptive architecture based on reinforcement learning.	81
5.3	Evolution of the developed platform toward advanced tool lifecycle management and adaptive intelligence.	82

List of Tables

1.1	Figeac Aéro Tunisia Fact Sheet.	5
1.2	FGAT Departments and Primary Missions	6
1.3	FGAT Product Families and Aircraft Applications	7
1.4	Project Stakeholder RACI Matrix	9
1.5	SMART Objectives and Expected Industrial Targets	11
2.1	Data Attributes Available in GEDIX	14
2.2	Data Gap Analysis for Predictive Tool Health Management	15
2.3	5 Whys Root Cause Analysis	16
2.4	SWOT Analysis of Tool Management Modernization at FGAT	16
2.5	QQOQCCP Synthesis – Current State vs. Target Solution	17
2.6	Implementation Scope and Technical Feasibility Assessment	18
3.1	Engineering rationale behind the five GSO constituent indices	33
3.2	Initial mapping between cumulative degradation and tool health states	36
4.1	Main technologies used during implementation	49
5.1	Proposed industrial validation indicators	75

List of Abbreviations

FGAT	Figeac Aéro Tunisie
ERP	Enterprise Resource Planning
KPI	Key Performance Indicator
GSO	Global Severity Index of Machining
IGSO	Indice Global de Sévérité d’Usinage
TF	Tool Family
WO	Work Order
RUL	Remaining Useful Life
API	Application Programming Interface
REST	Representational State Transfer
ORM	Object-Relational Mapping
CRUD	Create, Read, Update, Delete
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
SQL	Structured Query Language
UI	User Interface

Contents

Acknowledgements	ii
Abstract & Résumé	iii
List of Figures	iv
List of Tables	v
List of Abbreviations	vi
General Introduction	2
1 General Context and Company Presentation	3
1.1 Introduction	3
1.2 Overview of the FGA Group	3
1.2.1 History of FGA	3
1.2.2 Clients and Partners	4
1.2.3 FGA's Position in the Aeronautics Supply Chain	4
1.3 Figeac Aéro Tunisie (FGAT)	5
1.3.1 Departments and Organization Chart	5
1.3.2 Production Units (UAP) and Products	6
1.3.3 Digital Ecosystem: ERP (IFS), GEDIX, and Software Landscape	7
1.4 Internship Framework and Project Positioning	8
1.4.1 Internship Context and Assignment	8
1.4.2 Project Scope and Functional Perimeter	8
1.4.3 Project Stakeholders and RACI Matrix	8
1.5 Problem Statement	9
1.5.1 Cutting Tool Management Challenges	9
1.5.2 Business and Operational Impacts	10
1.6 Project Objectives and Expected Gains	10
1.6.1 General Objective	10
1.6.2 Specific Objectives	10
1.6.3 SMART Objectives and Expected Gains	11
1.7 Conclusion	11
2 Current State Diagnosis and Needs Analysis	12
2.1 Introduction	12
2.2 Mapping of the Current Tool Management Process	12
2.2.1 Current-State Process Flow	12
2.2.2 Tool Procurement and Identification	13
2.2.3 GEDIX Entry, Movements, and Exit	13
2.2.4 Tool Assignment and Usage	14
2.2.5 Sharpening and Reuse	14
2.2.6 Breakage Declaration and Disposal	14
2.3 Analysis of Available Data Sources	14
2.3.1 GEDIX Data Source	14
2.3.2 IFS ERP Data Source	14

2.3.3	Methods / CAD/CAM (FAO) Data Source	15
2.3.4	Data Gap Analysis	15
2.4	Root Cause Diagnosis	15
2.4.1	Ishikawa Diagram 6M	15
2.4.2	5 Whys Analysis	16
2.4.3	SWOT Analysis	16
2.5	QQOQCCP Synthesis of the Problem	17
2.6	From Diagnosis to the Proposed Solution	17
2.6.1	Layer 1 GSO-Based Tool Degradation Assessment	17
2.6.2	Layer 2 Markov-Based Tool State Prediction	17
2.6.3	Layer 3 Circular Economy Decision Support (Future Perspective)	17
2.6.4	Feasibility and Implementation Scope Assessment	17
2.7	Requirements Specification	18
2.7.1	Functional Requirements	18
2.7.2	Non-Functional Requirements	18
2.7.3	Actors and Use Cases	19
2.8	Conclusion	20
3	Solution Design and Development Methodology	21
3.1	Introduction	21
3.2	Development Methodology and Project Planning	21
3.2.1	Iterative Development Approach	21
3.2.2	Functional Decomposition and Development Phases	21
3.2.3	Project Planning and Development Roadmap	21
3.3	Proposed Solution and System Architecture	22
3.3.1	Overall Architecture of the Decision-Support Platform	22
3.3.2	End-to-End Data Flow	23
3.3.3	Core Architecture and Future Extensions	23
3.3.4	Technology Stack and Software Architecture	23
3.4	Database Design and Data Management	23
3.4.1	Entity-Relationship Model	23
3.4.2	Database Structure and Relationships	25
3.4.3	Tool, Machine, and Machining Session Data Management	26
3.4.4	Traceability and Historical Data Management	27
3.5	Layer 1 GSO-Based Tool Degradation Assessment	28
3.5.1	Objective and Operating Principle	28
3.5.2	Engineering Rationale for the Multi-Index Approach	28
3.5.3	Definition of the Global Severity/Degradation Index (GSO)	29
3.5.4	Definition and Engineering Justification of the Five Constituent Indices	30
3.5.5	Complementarity of the Five Indices	33
3.5.6	Weighting Coefficients and Aggregation Mechanism	34
3.5.7	Sensitivity and Criticality Analysis of the GSO Model	34
3.5.8	GSO Calculation and Cumulative Degradation Assessment	35
3.5.9	Tool Health State Determination	35
3.5.10	From Continuous Degradation to Discrete State Prediction	36
3.5.11	Critical Analysis and Anticipated Limitations	36
3.5.12	Anticipated Evolution of the GSO Model	37
3.5.13	Summary of Layer 1	37
3.6	Layer 2 — Stochastic Tool Health Prediction	37

3.6.1	Objective and Operating Principle	37
3.6.2	Markov Chain Modeling Principle	38
3.6.3	Definition of the Tool Health State Space	38
3.6.4	Construction of the Transition Probability Matrix	39
3.6.5	Initial Expert-Calibrated Transition Model	40
3.6.6	Future Tool-State Prediction	40
3.6.7	Failure-Risk Estimation	41
3.6.8	Remaining Useful Life Estimation	41
3.6.9	Conversion of RUL into Estimated Cutting Time	42
3.6.10	Relationship Between Layer 1 and Layer 2	42
3.6.11	Handling Uncertainty and Limited Historical Data	42
3.6.12	Model Assumptions and Critical Analysis	43
3.6.13	Future Evolution of the Predictive Layer	43
3.7	Functional Architecture and Main Use Cases	44
3.7.1	Overview of the Functional Architecture	44
3.7.2	User Roles and Access Management	44
3.7.3	Tool Identification and Traceability	44
3.7.4	Machining Session Recording	45
3.7.5	GSO Calculation and Tool-State Update	45
3.7.6	Markov Prediction and Risk Assessment	45
3.7.7	Monitoring and Decision-Support Dashboard	46
3.7.8	End-to-End Functional Workflow	46
3.8	Conclusion	47
4	Implementation, Testing and System Integration	48
4.1	Introduction	48
4.2	Development Environment	49
4.2.1	Programming Environment	49
4.2.2	Backend Technologies	49
4.2.3	Database Environment	50
4.3	Database Implementation	50
4.3.1	PostgreSQL Database Creation	50
4.3.2	Implementation of ORM Models	50
4.3.3	Implementation of Relationships	51
4.3.4	Database Migration	51
4.3.5	Initialization and Demonstration Data	52
4.4	Implementation of the Data Management Layer	52
4.4.1	User Management	52
4.4.2	Tool Management	52
4.4.3	Machine Management	52
4.4.4	Material, Lubricant and Operation Management	52
4.4.5	Tool Assignment	53
4.4.6	Machining Session Management	53
4.4.7	Cutting Parameter Management	53
4.5	Implementation of Layer 1	53
4.5.1	Implementation Strategy	53
4.5.2	Input Retrieval from the Database	54
4.5.3	Implementation of the Normalization Function	54
4.5.4	Implementation of the Mechanical Index	55

4.5.5	Implementation of the Thermal Index	55
4.5.6	Implementation of the Machine Index	56
4.5.7	Implementation of the Historical Index	57
4.5.8	Implementation of the Process Index	58
4.5.9	Implementation of the Global Calculation	58
4.5.10	Persistence of Calculated Results	59
4.6	Implementation of Damage and State Management	59
4.6.1	Damage Calculation Service	59
4.6.2	Updating the Tool Record	60
4.6.3	State Calculation Service	60
4.6.4	State History Persistence	61
4.7	Implementation of Layer 2	61
4.7.1	Markov Service Organization	61
4.7.2	Transition Matrix Configuration	61
4.7.3	Prediction Calculation	62
4.7.4	RUL Calculation	63
4.7.5	Prediction Persistence	63
4.8	Backend and API Integration	63
4.8.1	Flask Application Initialization	63
4.8.2	Blueprint Organization	64
4.8.3	Calculation Route	64
4.8.4	Request Processing	64
4.8.5	Response and Error Handling	65
4.9	Frontend Implementation	65
4.9.1	Login Interface	65
4.9.2	Dashboard Implementation	65
4.9.3	Dynamic Data Display	66
4.9.4	Tool Monitoring Interface	66
4.9.5	Prediction Display	67
4.9.6	Frontend-Backend Integration	67
4.10	System Integration	68
4.10.1	Database-to-Backend Integration	68
4.10.2	Backend-to-Analytical Pipeline Integration	68
4.10.3	Backend-to-Frontend Integration	68
4.10.4	Complete Implementation Sequence	68
4.11	Testing and Verification	68
4.11.1	Testing Objectives	68
4.11.2	Database Verification	69
4.11.3	Layer 1 Verification	69
4.11.4	Damage and State Verification	70
4.11.5	Layer 2 Verification	70
4.11.6	API Verification	70
4.11.7	Interface Verification	70
4.11.8	End-to-End Verification	71
4.12	Conclusion	71

5	Industrial Discussion, Limitations and Perspectives	72
5.1	Introduction	72
5.2	Industrial Contribution of the Developed Platform	72
5.2.1	Data Centralization and Traceability	72
5.2.2	Tool Lifecycle Monitoring	72
5.2.3	Degradation Assessment and State Monitoring	73
5.2.4	Predictive Decision Support	73
5.3	Industrial Applicability and Deployment Conditions	73
5.3.1	Data Availability and Quality	73
5.3.2	Integration with Existing Information Systems	74
5.3.3	Deployment Across Production Units	74
5.4	Validation Strategy and Industrial Performance Assessment	74
5.4.1	Validation Indicators	74
5.4.2	Prediction Validation	75
5.4.3	Tool-Life and Maintenance Indicators	75
5.4.4	Long-Term Industrial Evaluation	76
5.5	Limitations of the Current Implementation	76
5.5.1	Historical Data Availability	76
5.5.2	Parameter Calibration	76
5.5.3	Markov Transition Calibration	77
5.5.4	Physical Wear Validation	77
5.5.5	Generalization to Other Machining Contexts	77
5.6	Future Development Perspectives	77
5.6.1	Integration with Industrial Data Sources	77
5.6.2	Circular Tool Lifecycle Management	77
5.6.3	Adaptive Calibration through Reinforcement Learning	80
5.6.4	Extension to Multi-Machine and Multi-Tool Monitoring	82
5.7	Conclusion	83
	General Conclusion	84
	Webliography	85
	Bibliography	86

General Introduction

The aerospace manufacturing sector operates under stringent quality standards, high production costs, and non-negotiable safety requirements. Within this high-precision environment, cutting tool management plays a vital role in ensuring dimensional accuracy, surface integrity, and overall cost efficiency. Figeac Aéro Tunisia (FGAT), a major subcontractor specializing in aerospace structural components, constantly seeks to optimize its machining processes and minimize operational risks. In high-speed machining of complex aerospace alloys, cutting tools are subjected to severe thermal and mechanical stresses, leading to continuous wear, unpredictable degradation, and potential tool failure if not properly monitored.

Traditionally, tool replacement strategies on the shop floor rely heavily on conservative empirical estimates or fixed operator schedules. This approach often results in two costly extremes: either tools are discarded prematurely while still usable, increasing tooling costs, or tools are operated beyond their safe limits, risking tool breakage, workpiece scrap, and unplanned machine downtime. To overcome these limitations, transitioning toward a data-driven predictive maintenance framework becomes essential. By systematically capturing machining parameters and modeling tool degradation, manufacturers can estimate the remaining useful life (RUL) of cutting tools and optimize tool changing and regrinding cycles.

The objective of this engineering project is to design, develop, and evaluate an integrated web-based platform for cutting tool lifecycle monitoring and predictive wear assessment tailored to FGAT's operational needs. The platform combines a structured relational database with a two-layer analytical engine: the first layer computes analytical degradation and severity indices for each machining session, while the second layer leverages Markov chain stochastic modeling to predict future wear states and tool life expectancy. Coupled with a user-friendly web interface and a robust Flask backend, the system aims to provide manufacturing engineers and operators with clear, actionable decision support.

This report is structured into five main chapters tracing the complete engineering workflow. Chapter 1 introduces the industrial context of Figeac Aéro Tunisia and formulates the core problem statement. Chapter 2 reviews the scientific literature regarding tool wear mechanisms, machining severity metrics, and stochastic prediction models. Chapter 3 presents the software system requirements, relational database architecture, and global platform design. Chapter 4 details the technical implementation, covering backend development, predictive engine implementation, and user interface integration. Finally, Chapter 5 discusses the testing procedures, computational validation, and industrial results achieved by the deployed platform.

Chapter 1

General Context and Company Presentation

1.1 Introduction

In high-precision aeronautical machining, maintaining stringent quality standards, ensuring process stability, and optimizing production costs represent critical operational imperatives. Cutting tools play a decisive role in this environment, as their physical condition directly impacts surface finish, dimensional tolerances, and overall machining efficiency.

At Figeac Aéro Tunisie (FGAT), cutting tool management relies on the **GEDIX** software suite. While GEDIX ensures basic inventory cataloging and administrative tracking, it operates in a purely descriptive manner. It lacks the functional capability to track individual tool lifecycles, quantify cumulative physical degradation, or forecast catastrophic tool failures.

This chapter outlines the industrial framework of the project. It presents the parent organization, Figeac Aéro Group (FGA), and its Tunisian subsidiary (FGAT). Furthermore, it defines the internship scope, establishes project governance using a RACI matrix, formalizes the industrial problem statement, and details the strategic SMART objectives alongside expected Return on Investment (ROI) targets.

1.2 Overview of the FGA Group

Founded in 1989 by Jean-Claude Maillard, Figeac Aéro Group (FGA) is a leading French industrial group specializing in the design and manufacturing of structural sub-assemblies and precision components for the aerospace sector. Over more than three decades, FGA has transformed from a regional sub-contractor into a Tier-1 partner for major international aircraft manufacturers. Today, the group operates 15 production sites worldwide (including facilities in France, Tunisia, Morocco, Mexico, and the United States), supported by continuous investment in industrial modernization, technology innovation, and supply chain optimization.

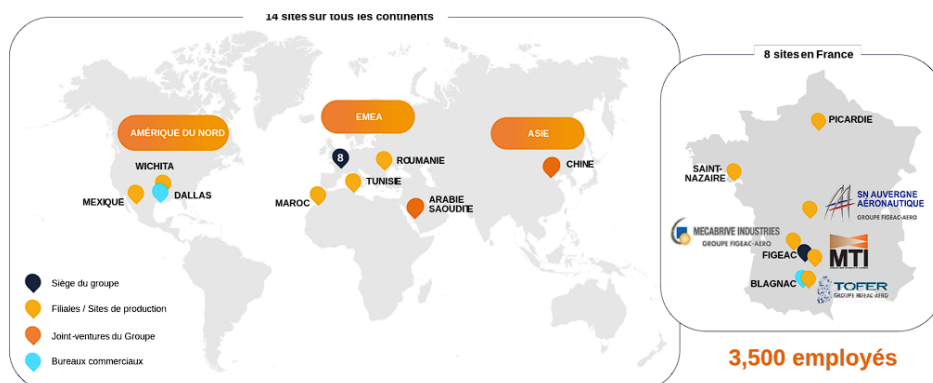


Figure 1.1: International Footprint of FGA Group

1.2.1 History of FGA

The FGA group is constantly evolving, with a trajectory marked by key events demonstrating a clear commitment to progress both nationally and internationally. The figure below made with napkin.ai summarizes

the major historical milestones of the group.

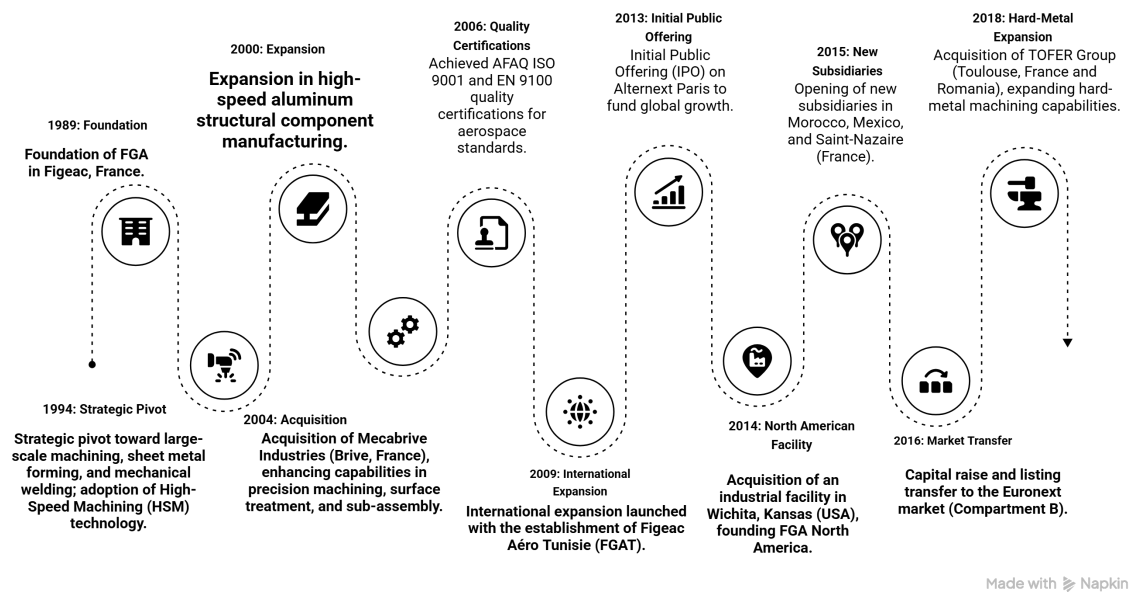


Figure 1.2: Titre de votre figure

1.2.2 Clients and Partners

To sustain its expansion, FGA maintains a client diversification strategy across commercial, military, and business aviation programs. The group acts as a key supplier to world-leading Original Equipment Manufacturers (OEMs), including **Airbus**, **Boeing**, **Safran**, **Stelia Aerospace** (Premium AEROTECH), **Dassault Aviation**, and **Eurocopter** (Airbus Helicopters). Figure 1.3 highlights FGA’s primary industrial partners.

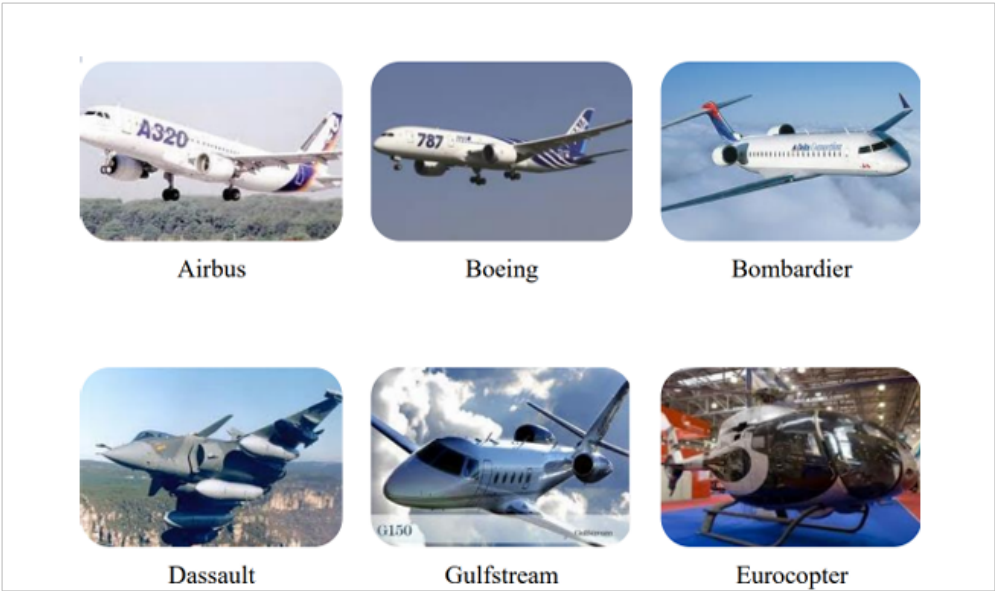


Figure 1.3: Primary Aerospace OEM Clients and Partners

1.2.3 FGA’s Position in the Aeronautics Supply Chain

FGA holds a strategic position as an integrated Tier-1 and Tier-2 aerospace partner. Operating under both build-to-print and build-to-spec mandates, FGA manages the entire manufacturing value chain from raw

material procurement (aluminum extrusions, hard metal forgings), computer-aided design and programming (CAD/CAM), multi-axis CNC machining, and specialized surface treatments, to structural sub-assembly integration and direct logistics delivery to OEM assembly lines.

1.3 Figeac Aéro Tunisie (FGAT)

Figeac Aéro Tunisie (FGAT) was established in 2009 in the El Mghira aerospace industrial hub near Tunis, following strategic alignment with major customers such as Stelia Aerospace Tunisia. Spanning a 2-hectare industrial site with 9,000 m² of covered workshops, FGAT is a regional benchmark for high-precision machining of light alloys (aluminum) and hard metals (titanium, stainless steel).

Company Name	Figeac Aéro Tunisia (FGAT)
Parent Company	Figeac Aéro Group (France)
Year of Establishment	2011
Legal Status	Fully Exporting Company
Sector of Activity	Aeronautical Manufacturing – Machining of structural parts in light alloys and hard metals
Headquarters	M’Ghira III Industrial Zone, Fouchana, Tunis, Tunisia
Workforce (Group)	~3,000 employees worldwide
Main Clients	Airbus, Boeing, Safran, Stelia Aerospace, ...
Certifications	ISO 9001, EN 9100, NADCAP, ISO 14001
Website	www.figeac-aero.com
HR Contact	contactrh.fgat@figeac-aero.com
Logo	

Table 1.1: Figeac Aéro Tunisia Fact Sheet.

1.3.1 Departments and Organization Chart

To ensure operational efficiency and quality compliance, FGAT is structured around seven primary departments whose combined missions cover the full manufacturing process. Table 1.2 details these departments and their core missions.

Table 1.2: FGAT Departments and Primary Missions

Department	Primary Mission
Human Resources (HR)	Recruitment, integration, technical skills matrix management, and safety training.
Production	Order execution, shop-floor dispatching, machine scheduling, and OEE optimization.
Finance & Controlling	Budgeting, cost management, tooling expenditure tracking, and financial control.
Quality Assurance	Part compliance, CMM inspection, EN 9100 certification, and customer audit management.
Maintenance	Total Productive Maintenance (TPM), machine availability, and spindle reliability.
Continuous Improvement	Lean manufacturing implementation, 5S, Kaizen projects, and workflow optimization.
Supply Chain	Material procurement, tool crib inventory control, and customer delivery logistics.

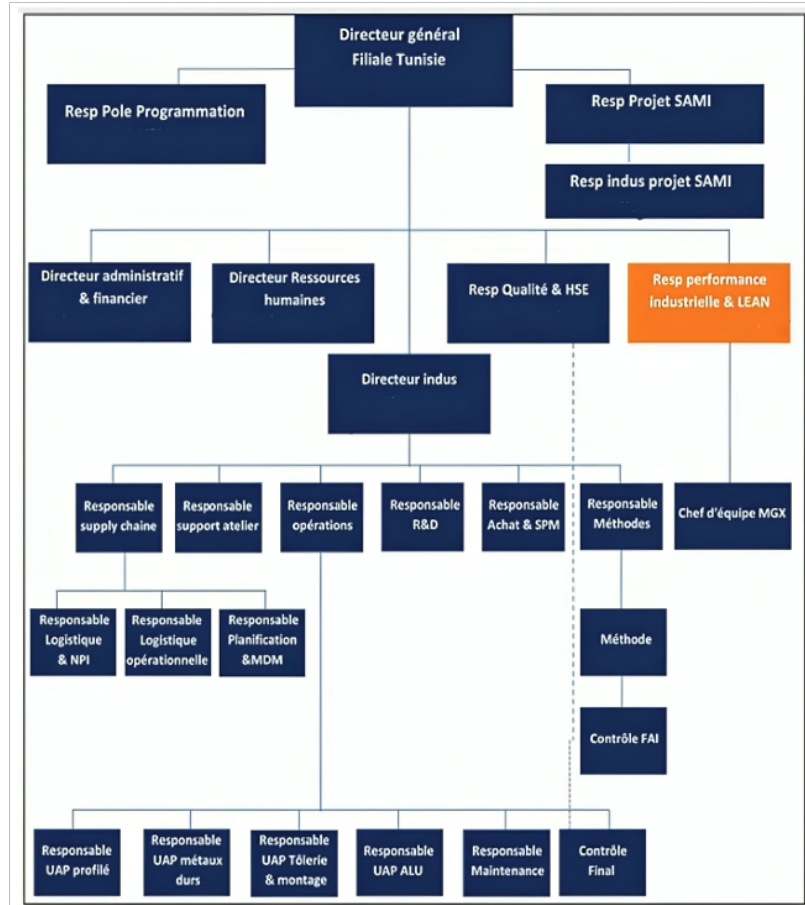


Figure 1.4: FGAT Organizational Structure

1.3.2 Production Units (UAP) and Products

FGATs shop floor is divided into five specialized Autonomous Production Units (*Unités Autonomes de Production* – UAPs), each dedicated to specific component geometries, material constraints, and manufacturing processes.

The operational scopes and strategic focus areas of the five UAPs are detailed below:

Profilé (Extrusions) Equipped with eight 4-axis CNC machining centers designed for long aerospace profiles, this unit serves as a major regional reference for extrusion machining.

Gdim (Large Dimensions) Focuses on large aluminum structural parts using 2, 3, and 5-axis CNC machines, alongside sheet metal forming and sub-assembly workshops.

Métaux Durs (Hard Metals) Exclusively dedicated to hard metal machining (titanium, stainless steel, specialized alloys) requiring optimized cutting speeds and high torque.

Pdim (Small Dimensions) Specializes in high-precision small aluminum and hard metal parts using 4 and 5-axis CNC centers.

Montage (Assembly) Responsible for the final assembly of machined parts, divided into six specialized sub-units covering kitting, quality control, and structural assembly.

FGAT manufactures a wide variety of structural parts categorized into three major product families, detailed in Table 1.3.

Table 1.3: FGAT Product Families and Aircraft Applications

Product Family	Representative Product	Aircraft Location	Product Description
Pdim (Small Parts)	Precision fittings, brackets, hinges	Aircraft empennage & wing structure	Structural joining and support components in aluminum or hard metals.
Gdim (Large Parts)	Formed sheet metal panels, frames	Fuselage structure	High-surface structural panels produced using specialized press forming and CNC contouring.
Profilé (Profiles)	Metallic profiles (U, L, T shapes)	Fuselage internal structure	Structural stiffeners providing mechanical reinforcement across large fuselage panels.

1.3.3 Digital Ecosystem: ERP (IFS), GEDIX, and Software Landscape

FGATs operational workflow relies on an integrated software landscape:

- **IFS Enterprise Resource Planning (ERP):** Manages Work Orders (*Ordres de Fabrication* – OF), master production schedules, routing sheets, bill of materials (BOM), and machine assignments.
- **GEDIX Tool Management System:** Controls central tool crib inventory, geometric tool assembly definitions, presetting offsets, and tool issue/return transactions.
- **Methods / CAD/CAM Environment (CATIA / TOPSOLID):** Defines nominal cutting parameters (cutting speed V_c , feed rate f_z , depth of cut a_p , width of cut a_e) and theoretical machining cycle times.

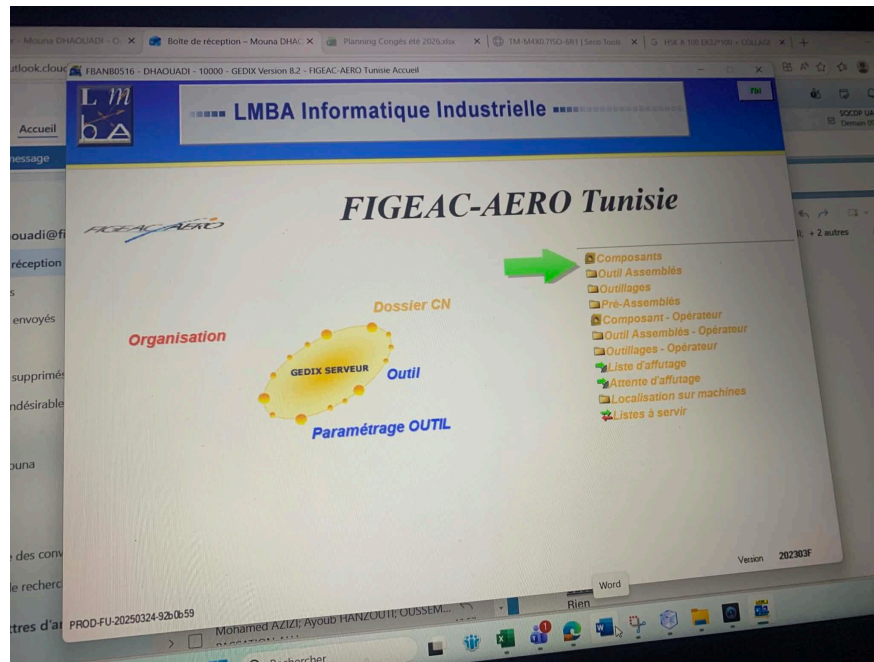


Figure 1.5: GEDIX Tool Management System interface

1.4 Internship Framework and Project Positioning

1.4.1 Internship Context and Assignment

This engineering internship project takes place at FGAT under the presetting of cutting tools department supervision. The core assignment consists of designing, developing, and validating a web-based decision-support prototype platform. This platform establishes item-level tool traceability, computes cumulative physical degradation using a multi-factor engine (**Layer 1 GSO**), and forecasts tool health states via a stochastic model (**Layer 2 Markov Chain**).

1.4.2 Project Scope and Functional Perimeter

The functional scope of the project is defined as follows:

- **Core Developed Prototype Scope:** Full implementation of **Layer 1** (Global Severity/Degradation Index – GSO, combining mechanical, thermal, machine, historical, and parameter indices) and **Layer 2** (Markov chain state transitions, failure risk evaluation, and Remaining Useful Life – RUL estimation).
- **Data Integration Scope:** Aggregation of CAD/CAM cutting parameters, IFS ERP Work Order execution data, and GEDIX tool crib presetting logs.
- **Future Extensions (Out of Immediate Prototype Scope):** Layer 3 (Circular Economy decision support) and adaptive Reinforcement Learning (RL) parameter calibration are treated strictly as strategic perspectives in Chapter 5.

1.4.3 Project Stakeholders and RACI Matrix

To ensure structured governance across cross-functional teams, stakeholder roles and responsibilities are mapped using a RACI matrix in Table 1.4.

Table 1.4: Project Stakeholder RACI Matrix

Project Deliverable / Activity	Student	Methods	Tool Crib	Production	IT Dept
Current State Diagnosis & Requirements	R	A	C	C	I
Data Collection & Data Gap Analysis	R	C	C	I	C
GSO Mathematical Model Design (Layer 1)	R	A	I	I	I
Markov Chain Model Formulation (Layer 2)	R	A	I	I	I
Database Design & Backend Architecture	R	I	I	I	A
Web Interface & Dashboard Development	R	C	C	C	I
End-to-End System Testing & Validation	R	C	C	A	C
Control Strategy & Deployment Planning	R	A	C	C	C
Legend: R = Responsible (Executes), A = Accountable (Approves), C = Consulted, I = Informed.					

1.5 Problem Statement

1.5.1 Cutting Tool Management Challenges

Despite the deployment of GEDIX, cutting tool management at FGAT remains purely administrative. A major vulnerability occurs during tool disassembly at the presetting station: when an operator unloads a tool from its tool holder, it is returned to a generic storage rack labeled only by tool family and diameter.



Figure 1.6: Physical Tool Storage Rack with Generic Bin Classification

As shown in Figure 1.6, individual tool serial identity is completely lost at this stage. Consequently, tool management suffers from critical operational bottlenecks:

- **Lack of Cumulative Degradation Tracking:** No system records the exact accumulated cutting time, thermal stress, or mechanical fatigue sustained by a specific physical tool across multiple Work Orders.
- **Limited Traceability of Tool Usage:** Tools are shared across different part references, machines, and operations without individual session logs.

- **Loss of Sharpening/Regrounding History:** The system cannot distinguish between a brand-new tool and an item reground multiple times.
- **Limited Anticipation of Tool Failure:** Tool replacement decisions rely on theoretical lifespan estimates or operator subjectivity, leading to unexpected tool breakages during critical machining passes.

1.5.2 Business and Operational Impacts

This lack of item-level lifecycle monitoring induces significant business and operational risks:

1. **Tooling Overhead Costs:** Tools are prematurely scrapped due to excessive caution, or expensive cutters are destroyed during catastrophic failure.
2. **Unplanned Machine Downtime:** Sudden tool breakages cause spindle shocks, tool holder damage, and prolonged emergency line stoppages.
3. **Quality and Non-Conformance Risks:** Severely worn tools cause chatter vibrations, dimensional drift, and degraded surface roughness (R_a), leading to part rework or scrap on high-value titanium/aluminum components.
4. **Reduced Overall Equipment Effectiveness (OEE / TRS):** Machine utilization drops due to unplanned setups, tool changes, and troubleshooting pauses.

The core problem statement governing this project is formalized as follows:

How can FGAT transition from a descriptive, administrative tool crib management process to a predictive, item-level lifecycle monitoring platform that maintains individual tool identity (Tool_ID), continuously quantifies multi-factor physical degradation, and accurately forecasts failure risks to eliminate unplanned downtime?

1.6 Project Objectives and Expected Gains

1.6.1 General Objective

The general objective of this project is to design, build, and validate a prototype decision-support web platform that integrates item-level tool identification (Tool_ID), a multi-factor degradation calculation engine (**Layer 1 GSO**), and a stochastic state prediction model (**Layer 2 Markov Chain**) connected to FGATs data landscape.

1.6.2 Specific Objectives

To achieve the general objective, the project focuses on five specific technical goals:

- Implement an individual tool identification protocol (Tool_ID via barcode/QR code) covering the entire tool lifecycle (presetting, loading, machining, regrinding, unloading).
- Formulate the mathematical engine for **Layer 1 GSO**, aggregating five constituent indices: Mechanical (I_M), Thermal (I_T), Machine (I_{MA}), Historical (I_H), and Process-Parameter (I_P).
- Map the continuous GSO score into discrete health states (S_0 : New, S_1 : Normal Wear, S_2 : Monitoring, S_3 : Critical/Regrindable, S_4 : Out of Service/Broken).
- Develop the **Layer 2 Markov Chain** prediction engine to compute state transition probability matrices, forecast future state distributions, evaluate failure risk, and estimate Remaining Useful Life (RUL).

- Build a robust web application (Python/Flask, PostgreSQL, SQLAlchemy) featuring role-based interfaces (Administrator, Operator, Decision Support) and real-time monitoring dashboards.

1.6.3 SMART Objectives and Expected Gains

The targets and business expectations established for industrial validation are structured using the SMART framework in Table 1.5.

Table 1.5: SMART Objectives and Expected Industrial Targets

SMART Criteria	Project Definition & Target Metrics
Specific	Implement an item-level tool tracking platform using a multi-index degradation score (GSO) and a stochastic Markov engine to predict health states.
Measurable	• Target Reduction in Tool Breakages: –60% reduction in catastrophic in-process tool breakages. • Target Reduction in Tooling Costs: –15% decrease in global annual tooling expenditure through optimized tool life consumption [Target Assumption]. • Target Reduction in Unplanned Stops: –20% reduction in unscheduled machine downtime linked to tool failure [Target Assumption].
Achievable	Aggregates existing shop-floor data (GEDIX, IFS ERP, CAD/CAM files) into a lightweight Python/Flask backend and PostgreSQL database.
Relevant	Directly aligns with FGAT’s continuous improvement roadmap, cost-reduction targets, and EN 9100 quality traceability mandates.
Time-bound	Prototype development, API testing, and end-to-end workflow validation scheduled within a 6-week engineering internship framework.

Note on Target Metrics and Financial Assumptions: Target figures (such as –60% breakages, –15% tooling costs, and target ROI $\approx$ 350%) represent **initial strategic targets and working assumptions** established during project definition. Because full empirical verification requires long-term shop-floor execution across multiple production cycles, these metrics serve as baseline performance benchmarks for future industrial deployment.

1.7 Conclusion

This chapter established the industrial context and organizational foundation of the project at Figeac Aéro Tunisie. Analyzing the existing digital ecosystem highlighted the limitations of the GEDIX software, which lacks item-level tool tracking and predictive degradation capabilities. To address these operational challenges, the project scope, RACI framework, formal problem statement, and SMART objectives were established.

Chapter 2

Current State Diagnosis and Needs Analysis

2.1 Introduction

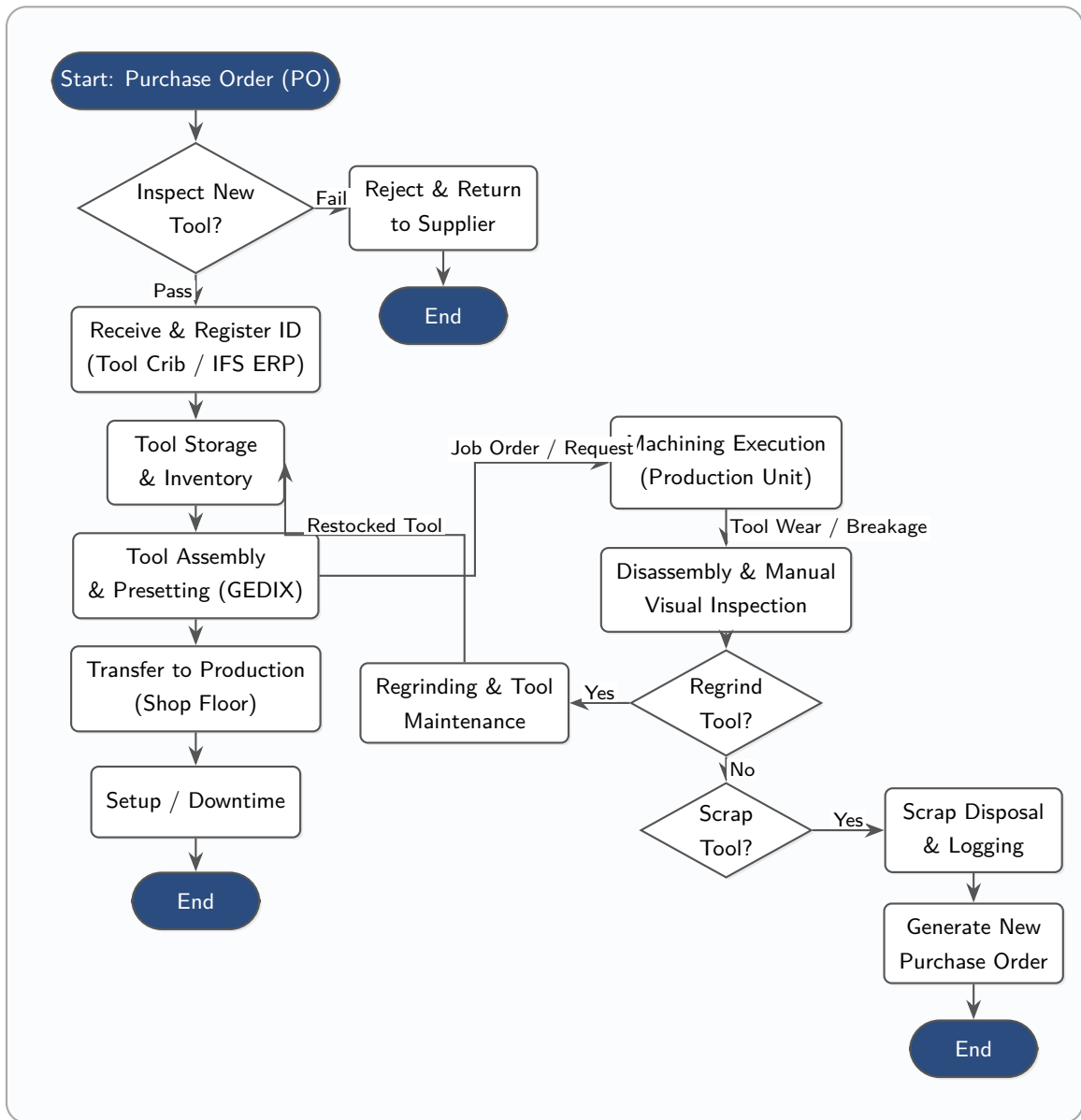
A rigorous engineering solution requires a comprehensive analysis of existing shop-floor practices, data availability, and underlying operational bottlenecks. At Figeac Aéro Tunisie (FGAT), cutting tool management is currently governed by administrative rules and standalone software modules. Although functional for inventory tracking, the current setup exhibits significant gaps regarding physical degradation monitoring and failure anticipation.

This chapter presents a detailed diagnosis of FGAT's tool management ecosystem. It maps the current process flow from procurement to disposal, audits existing data sources across GEDIX, IFS ERP, and CAD/CAM systems, performs a root-cause analysis (Ishikawa 6M, 5 Whys, SWOT, and QQQQCCP frameworks), articulates the transition toward a 3-layer predictive solution, and establishes the complete functional and non-functional requirements specification.

2.2 Mapping of the Current Tool Management Process

2.2.1 Current-State Process Flow

The lifecycle of a cutting tool at FGAT currently follows a traditional linear workflow: procurement, inventory registration, assembly/presetting, shop-floor dispatch, machining execution, disassembly, regrinding (if applicable), and scrap disposal. However, structural discontinuities exist between these phases, particularly during physical transfers between the presetting area and central tool crib storage.



2.2.2 Tool Procurement and Identification

New cutting tools are ordered via the IFS ERP system based on minimum stock thresholds or upcoming Work Order (*Ordres de Fabrication* – OF) requirements. Upon arrival, tools receive a batch-level reference designation in GEDIX. At this stage, tools are identified by generic supplier reference numbers rather than unique individual physical serial numbers (Tool_ID).

2.2.3 GEDIX Entry, Movements, and Exit

GEDIX records tool movements between the central storage crib and the presetting room. When a tool assembly is requested for a specific Work Order:

1. The tool crib manager issues the tool cutter and holder based on the GEDIX tool list.
2. The presetting operator mounts the tool insert/cutter into the tool holder and measures geometric offset parameters (X , Z lengths, diameter, runout).
3. Geometric parameters are loaded into GEDIX and transmitted to the CNC machine controller via a RFID chip or barcode tag on the tool holder assembly.

2.2.4 Tool Assignment and Usage

Once loaded into the CNC machine magazine, the tool assembly executes programmed machining operations across one or multiple Work Orders. Current tracking logs the total operating time of the CNC machine spindle, but fails to isolate the net cutting engagement time or physical stress experienced specifically by the individual cutter.

2.2.5 Sharpening and Reuse

When a tool exhibits visible wear or completes its theoretical planned operation count, it is removed from the spindle and sent to the presetting station for disassembly. At this critical junction:

- The physical cutter is separated from its holder.
- The operator records only the generic tool family and nominal diameter in GEDIX.
- The tool is placed into a generic batch bin or sent to an external vendor for regrinding/sharpening.

Consequently, the accumulated machining history and regrind count of that specific physical tool are permanently lost.

2.2.6 Breakage Declaration and Disposal

If a tool breaks catastrophically during machining, the operator logs a breakage event in GEDIX. However, because historical cutting conditions and cumulative wear were not tracked prior to failure, the root cause cannot be scientifically isolated. Scrapped tools are manually thrown into disposal bins without detailed metallurgical or operational analysis.

2.3 Analysis of Available Data Sources

2.3.1 GEDIX Data Source

GEDIX acts as the primary tool management repository. Table 2.1 categorizes the data attributes maintained within GEDIX and their current operational utility.

Table 2.1: Data Attributes Available in GEDIX

Data Category	Attributes Recorded	Current Limitation
Tool Identity	Generic reference, nominal diameter, tool family, geometry.	No individual physical serial number (Tool_ID).
Movements	Check-out/check-in timestamps, crib location, assigned machine.	Grouped at batch level; lost upon tool holder disassembly.
Sharpening Logs	Total regrind count per reference batch.	Individual regrind history per physical tool is untracked.
Breakage Declarations	Incident timestamp, operator ID, machine ID, basic defect code.	Lack of contextual cutting parameter logs prior to breakage.

2.3.2 IFS ERP Data Source

IFS ERP manages higher-level shop-floor operations and production context. Relevant data fields include Work Order (OF) numbers, operation sequences, assigned CNC machine centers, target part quantities, material grades (e.g., Titanium Ti-6Al-4V, Aluminum 7075), and theoretical operation start/end times.

2.3.3 Methods / CAD/CAM (FAO) Data Source

The Methods department (CAM/FAO using CATIA or TOPSOLID) defines nominal cutting parameters:

- Cutting speed V_c (m/min) and spindle speed N (RPM).
- Feed rate f_z (mm/tooth/rev) and table feed V_f (mm/min).
- Radial depth of cut a_e (mm) and axial depth of cut a_p (mm).
- Theoretical engagement duration per part (t_c).

2.3.4 Data Gap Analysis

To establish an effective predictive degradation engine, a comparative gap analysis between available, required, and missing data was performed, as summarized in Table 2.2.

Table 2.2: Data Gap Analysis for Predictive Tool Health Management

Data Dimension	Available Data	Required Data	Identified Gap & Solution
Identification	Batch reference ID (GEDIX).	Unique physical serial ID (Tool_ID).	Gap: High. Solution: Implement QR code / serialization protocol.
Machining Stress	Nominal CAM parameters (V_c, f_z, a_p).	Real-time / Session actual cutting parameters.	Gap: Medium. Solution: Log actual session parameters via user input/API.
Tool Health State	Binary state (Usable / Scrapped).	Multi-state health index ($S_0 \rightarrow S_4$).	Gap: Critical. Solution: Compute Layer 1 GSO engine score.
Thermal & Machine Context	General coolant type & machine center ID.	Machine rigidity factor & thermal severity index.	Gap: Medium. Solution: Incorporate machine (I_{MA}) & thermal (I_T) indices.

2.4 Root Cause Diagnosis

2.4.1 Ishikawa Diagram 6M

To systematically analyze the root causes behind premature tool breakages and uncontrolled tooling expenditure, an Ishikawa (Fishbone) diagram was constructed across the 6M categories:

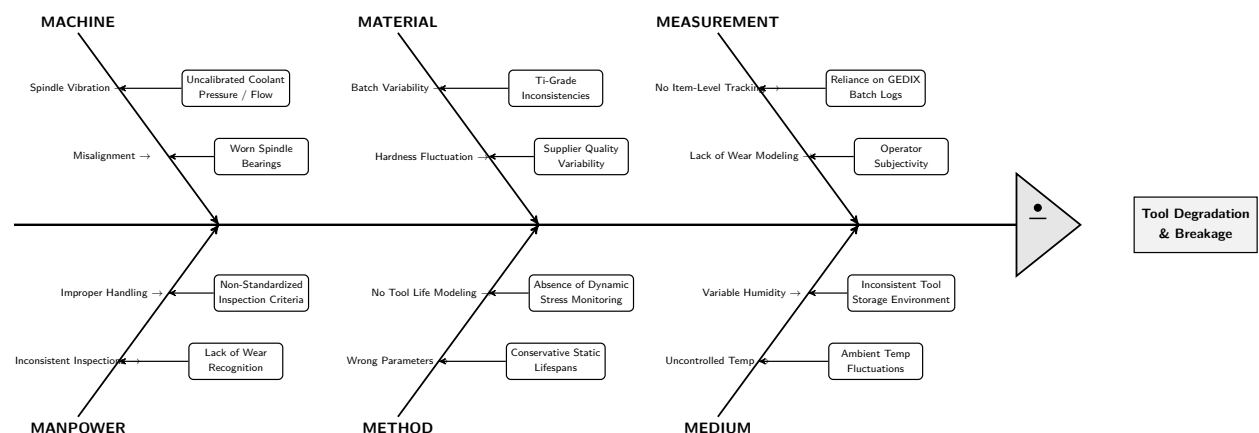


Figure 2.1: Detailed Ishikawa (6M) Diagram for Cutting Tool Degradation and Breakage.

- **Methods:** Lack of individual tool wear modeling; reliance on static, conservative theoretical tool lifespans.

- **Machine:** Spindle vibration variability across aging CNC centers; uncalibrated coolant concentration/pressure.
- **Materials:** Hard metal batch hardness variations (e.g., Titanium grade inconsistencies) increasing tool stress.
- **Manpower:** Operator subjectivity during manual tool inspection; inconsistent wear declaration criteria.
- **Milieu (Environment):** Ambient temperature fluctuations affecting thermal expansion during long dry/wet passes.
- **Measurement:** Absence of item-level tracking (Tool_ID); reliance on GEDIX batch-level logs.

2.4.2 5 Whys Analysis

A triple-stream 5 Whys analysis was executed to drill down into the fundamental root causes of frequent tool breakages, lack of wear tracking, and failure to anticipate end-of-life:

Table 2.3: 5 Whys Root Cause Analysis

Why	Stream 1: In-Process Breakage	Stream 2: No Wear Tracking	Stream 3: No Anticipation
1	Tools break during critical machining passes.	Cumulative tool wear is never quantified.	Tools are replaced reactively after breakage.
2	Tools are operated beyond their real physical fatigue limit.	Tool identity is lost upon holder disassembly.	No predictive model forecasts remaining tool life.
3	Lifespan is based on static theoretical estimates.	GEDIX stores tools in generic reference bins.	Systems lack physical degradation calculation engines.
4	CAM estimates do not reflect real shop-floor stresses.	Serialization protocols were never established.	No mathematical framework merges multi-source data.
5 (Root Cause)	Absence of dynamic, real-time multi-factor stress monitoring.	Absence of unique physical Tool_ID tracking throughout the full lifecycle.	Absence of stochastic modeling engines (GSO + Markov) for state prediction.

2.4.3 SWOT Analysis

Table 2.4 summarizes the internal strengths, internal weaknesses, external opportunities, and external threats regarding tool management at FGAT.

Table 2.4: SWOT Analysis of Tool Management Modernization at FGAT

Strengths (Internal)	Weaknesses (Internal)
▪ Structured digital environment (IFS ERP, GEDIX). ▪ Highly skilled CNC operators and Methods team. ▪ Standardized CAM programming routines.	▪ Loss of tool serial identity at presetting disassembly. ▪ Purely administrative tool crib tracking. ▪ Lack of predictive algorithms in existing software.
Opportunities (External)	Threats (External)
▪ Implementation of Industry 4.0 IoT serialization. ▪ Development of multi-index degradation scoring. ▪ Significant cost reduction through tool life optimization.	▪ High costs of aerospace scrap parts if tools break. ▪ Stringent EN 9100 customer audit requirements. ▪ Rising raw material prices for tungsten carbide cutters.

2.5 QQQQCCP Synthesis of the Problem

To formalize the problem context and define the target solution scope, a QQQQCCP synthesis (*Qui, Quoi, Où, Quand, Comment, Combien, Pourquoi*) was established in Table 2.5.

Table 2.5: QQQQCCP Synthesis – Current State vs. Target Solution

Dimension	Current State at FGAT	Target Solution
Who? (Qui?)	Operators, Tool Crib Managers.	Same stakeholders supported by a central decision-support web platform.
What? (Quoi?)	Generic reference tracking in GEDIX; no physical wear estimation.	Item-level tracking via Tool_ID; GSO wear score and Markov state prediction.
Where? (Où?)	Shop-floor CNC machine centers, presetting station, central tool crib.	Seamless tracking across all physical stations (presetting, CNC, crib).
When? (Quand?)	Tool serial identity lost upon disassembly at presetting station.	Continuous tracking throughout the complete multi-use tool lifecycle.
How? (Comment?)	Subjective visual wear inspection; static CAM lifespan estimates.	QR code scanning at stations; automatic GSO calculation and Markov forecasting.
How Much? (Combien?)	Significant tooling costs, unplanned downtime, scrap risk.	Target Gains: –60% breakages, –15% tooling costs, –20% stops, ROI $\approx$ 350%.
Why? (Pourquoi?)	Reactive, administrative tool management.	Predictive, item-level tool lifecycle control and quality assurance.

2.6 From Diagnosis to the Proposed Solution

Based on the diagnosis, the proposed solution transitions FGAT from administrative tracking to a 3-layer architecture:

2.6.1 Layer 1 GSO-Based Tool Degradation Assessment

Quantifies physical degradation sustained during a machining session by computing a Global Severity Index (GSO):

$$GSO = \alpha I_M + \beta I_T + \gamma I_{MA} + \delta I_H + \varepsilon I_P \quad (2.1)$$

where $I_M, I_T, I_{MA}, I_H, I_P$ represent Mechanical, Thermal, Machine, Historical, and Parameter indices respectively, weighted by coefficients $\alpha, \beta, \gamma, \delta, \varepsilon$.

2.6.2 Layer 2 Markov-Based Tool State Prediction

Uses stochastic discrete-time Markov chains to model tool health state transitions ($S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4$), evaluate short-term failure risks, and estimate Remaining Useful Life (RUL).

2.6.3 Layer 3 Circular Economy Decision Support (Future Perspective)

A future module (detailed in Chapter 5) providing decision support for regrinding eligibility, tool reuse in less demanding operations, and material recycling.

2.6.4 Feasibility and Implementation Scope Assessment

Table 2.6 details the feasibility and scope allocation across the three layers for this graduation project.

Table 2.6: Implementation Scope and Technical Feasibility Assessment

Layer	Technical Description	Project Implementation Scope
Layer 1 (GSO)	Multi-index physical degradation engine.	Core Scope: Fully developed, integrated, and validated in prototype.
Layer 2 (Markov)	Stochastic health prediction & RUL model.	Core Scope: Fully developed with expert-calibrated transition matrix.
Layer 3 (Circular)	Tool reuse & regrinding optimization.	Perspective Scope: Formulated as a future extension (Chapter 5).

2.7 Requirements Specification

2.7.1 Functional Requirements

The proposed system must fulfill the following functional requirements (FRs):

- **FR-01 (Tool Serialization):** Assign and register a unique serial identifier (Tool_ID) with associated QR code labels.
- **FR-02 (Traceability Management):** Record complete movement history across presetting, CNC loading, machining, and crib return.
- **FR-03 (Machining Session Logging):** Capture actual machining session data (Work Order ID, machine ID, actual cutting parameters, duration).
- **FR-04 (GSO Engine Calculation):** Compute constituent indices ($I_M, I_T, I_{MA}, I_H, I_P$) and aggregate weighted GSO values upon session completion.
- **FR-05 (Health State Mapping):** Automatically assign updated discrete tool health states (S_0 through S_4).
- **FR-06 (Markov Prediction):** Execute Markov state transition calculations, evaluate failure probability, and estimate RUL.
- **FR-07 (Dashboard & Alerts):** Display real-time shop-floor tool health, generate high-risk alerts (S_3, S_4), and show analytical charts.

2.7.2 Non-Functional Requirements

The platform must comply with the following non-functional requirements (NFRs):

- **NFR-01 (Usability):** Intuitive web user interface designed for fast shop-floor operation via bar-code/QR readers.
- **NFR-02 (Performance):** GSO calculation and Markov prediction response time under 500 ms per request.
- **NFR-03 (Robustness & Data Consistency):** Relational integrity constraints in PostgreSQL preventing data loss or orphaned records.
- **NFR-04 (Scalability):** Modular micro-service design allowing seamless integration with ERP/GEDIX via REST APIs.

2.7.3 Actors and Use Cases

Three main user roles interact with the platform:

1. **System Administrator:** Manages user accounts, machine profiles, tool master catalogs, and GSO weighting parameters.
2. **CNC / Presetting Operator:** Scans Tool_ID barcodes, registers mounting/unmounting, logs machining session parameters, and views immediate tool health indicators.
3. **Decision-Support Users (Methods / Tool Crib Managers):** Monitors plant-wide tool health dashboards, analyzes failure risks, reviews RUL metrics, and validates regrinding or scrap decisions.

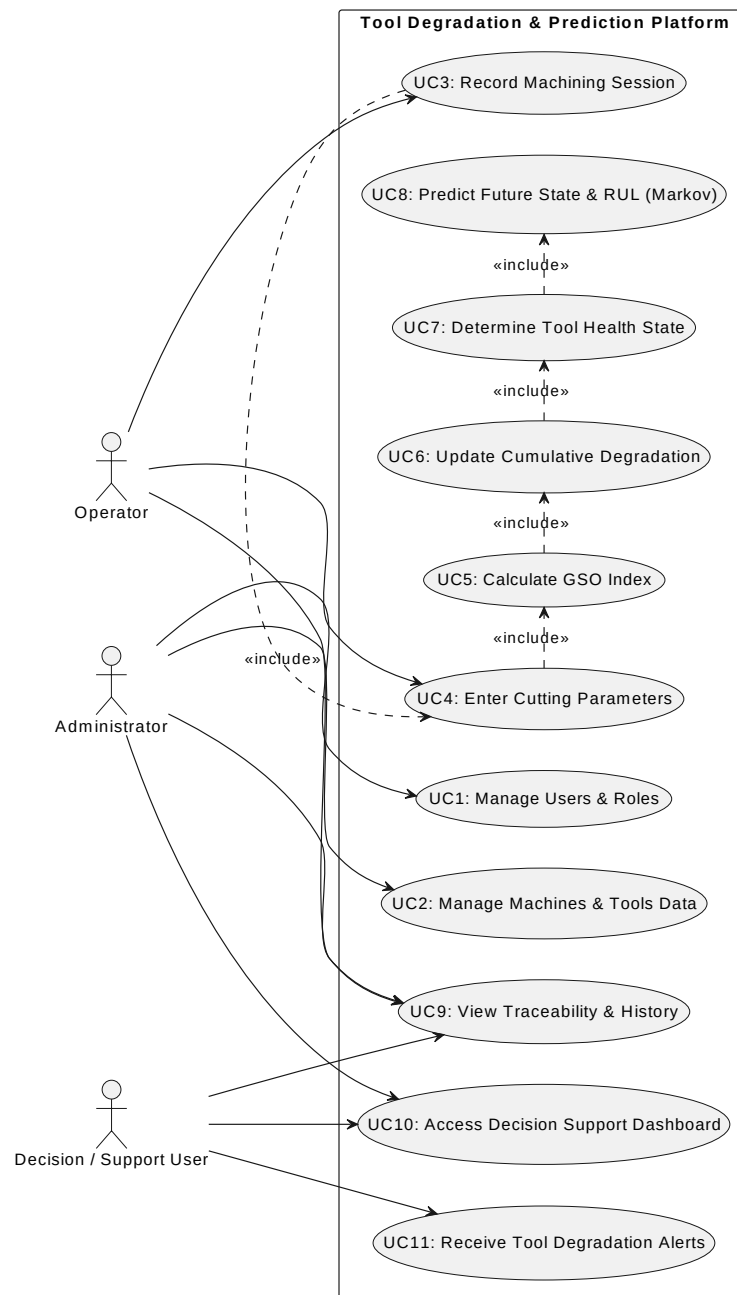


Figure 2.2: UML Use Case Diagram for the Decision-Support Platform

2.8 Conclusion

This chapter provided a rigorous diagnosis of cutting tool management at FGAT. Auditing current workflows and data sources revealed that losing individual physical tool identity at the presetting station represents the primary bottleneck preventing effective tool life optimization. Root-cause analyses (Ishikawa, 5 Whys, SWOT, QQQQCCP) confirmed the necessity of introducing item-level serialization (`Tool_ID`) combined with multi-factor degradation modeling.

These findings established the specifications for a 3-layer architecture (GSO degradation engine, Markov prediction, and Circular Economy perspective). The next chapter (**Chapter 3**) presents the detailed system architecture, database modeling, and mathematical formulation governing Layers 1 and 2.

Chapter 3

Solution Design and Development Methodology

3.1 Introduction

To address the operational gaps identified in Chapter 2 specifically the loss of item-level tool identity and the reliance on static lifespan estimates, this chapter presents the detailed architecture, mathematical models, and database design for the proposed decision-support platform.

The primary goal is to establish an end-to-end digital system that integrates physical machining parameters to compute real-time cumulative degradation, predict short-term tool failure risks, and estimate Remaining Useful Life (RUL). This chapter details the iterative development methodology, the 3-layer system architecture, the relational database structure, the formal mathematical modeling governing Layers 1 and 2, and the functional design of the platform.

3.2 Development Methodology and Project Planning

3.2.1 Iterative Development Approach

An Agile-inspired, iterative software development approach was adopted for this engineering project. This methodology allows progressive refinement of mathematical formulations, database schemas, and REST API endpoints based on incremental feedback from FGAT's Methods engineers, presetting and machining operators.

3.2.2 Functional Decomposition and Development Phases

The engineering lifecycle was decomposed into four structured, sequential phases:

1. **Phase 1 Domain Analysis & Data Schema Design:** Process mapping, root-cause diagnosis, entity-relationship modeling, and PostgreSQL database initialization.
2. **Phase 2 Core Engine Formulation (Layer 1 & Layer 2):** Mathematical formulation of the 5-index Global Severity Index (GSO) and construction of the discrete-time Markov chain transition matrix.
3. **Phase 3 Backend Web Service Development:** Implementation of Flask RESTful microservices, ORM mapping via SQLAlchemy, and business logic endpoints.
4. **Phase 4 User Interface Integration & End-to-End Validation:** Development of interactive web dashboards, integration of QR code scanning, and execution of validation test scenarios.

3.2.3 Project Planning and Development Roadmap

The implementation roadmap spanned a 16-week timeline, structured across the functional phases as illustrated in Figure ??.

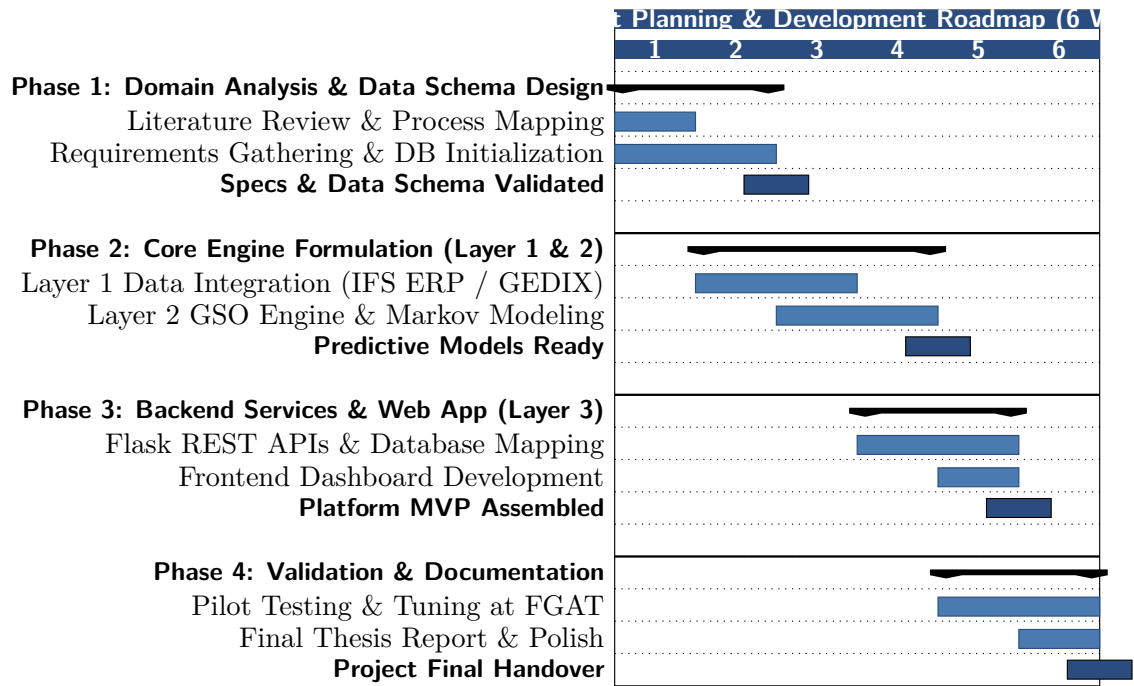


Figure 3.1: 6-Week Project Implementation Roadmap and Planning Gantt Chart.

3.3 Proposed Solution and System Architecture

3.3.1 Overall Architecture of the Decision-Support Platform

The platform is structured around a 3-layer analytical concept operating on a centralized web application backend connected to FGAT's shop-floor entities.

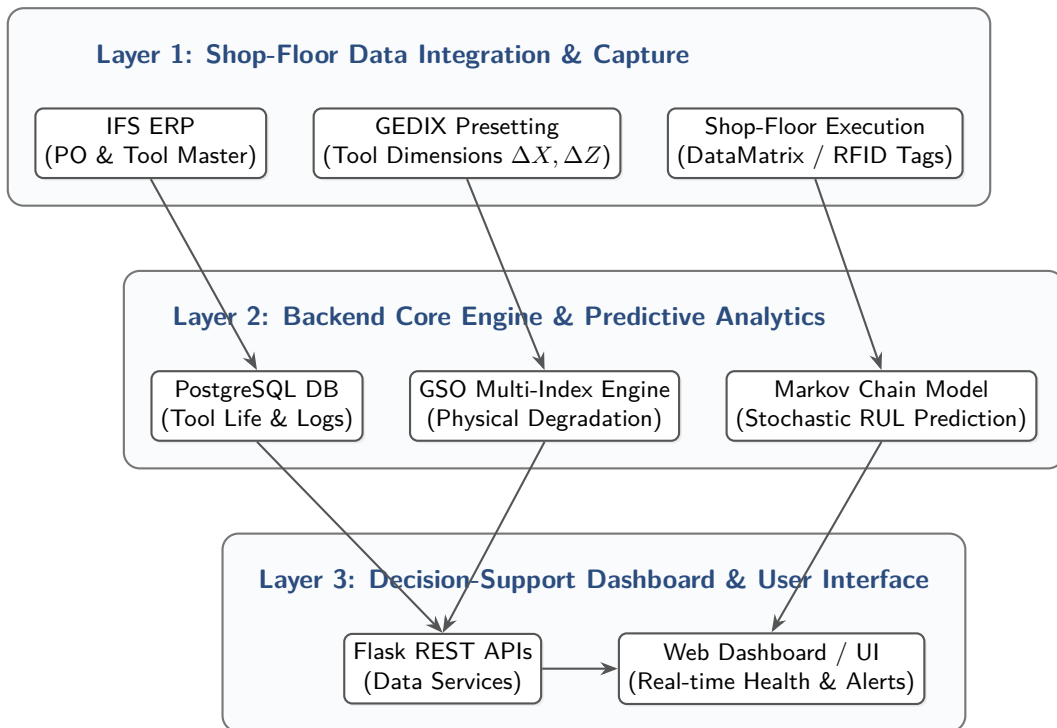
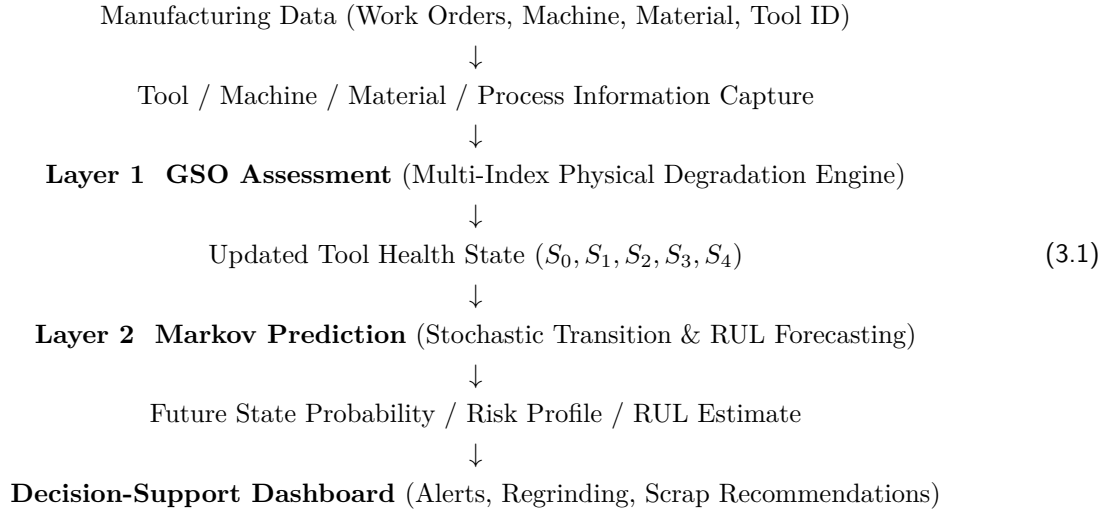


Figure 3.2: Overall Architecture of the Cutting Tool Decision-Support Platform.

3.3.2 End-to-End Data Flow

The functional pipeline processes manufacturing data end-to-end across six structured transformations:



3.3.3 Core Architecture and Future Extensions

The architectural footprint is cleanly divided between core deliverables and long-term perspectives:

- **Core Prototype (Implemented):** Layer 1 (GSO Degradation Assessment) and Layer 2 (Markov-Based Stochastic Prediction), fully supported by the Flask/PostgreSQL web platform.
- **Future Extension 1 (Layer 3 Circular Economy):** Decision-support modules for tool reuse, second-life allocation in less demanding materials, and regrinding lifecycle optimization.
- **Future Extension 2 (Adaptive Intelligence):** Reinforcement Learning (RL) agents dynamically calibrating GSO weighting parameters based on real-time wear feedback.

3.3.4 Technology Stack and Software Architecture

The software technology stack was selected to ensure high performance, modularity, and seamless deployment within FGAT's industrial IT landscape:

- **Programming Language:** Python 3.10+ chosen for its rich ecosystem in scientific computing and web backend framework integration.
- **Backend Framework:** Flask micro-framework providing lightweight REST API routing and modular blueprint organization.
- **ORM & Database Management:** Flask-SQLAlchemy object-relational mapping interface backed by PostgreSQL 15 for robust ACID-compliant relational data management.
- **Frontend UI & Charting:** HTML5, Bootstrap 5, Jinja2 template engine, and Chart.js for real-time interactive shop-floor health visualization.

3.4 Database Design and Data Management

3.4.1 Entity-Relationship Model

To ensure persistent item-level traceability across multiple machining assignments, a comprehensive relational data model was designed.

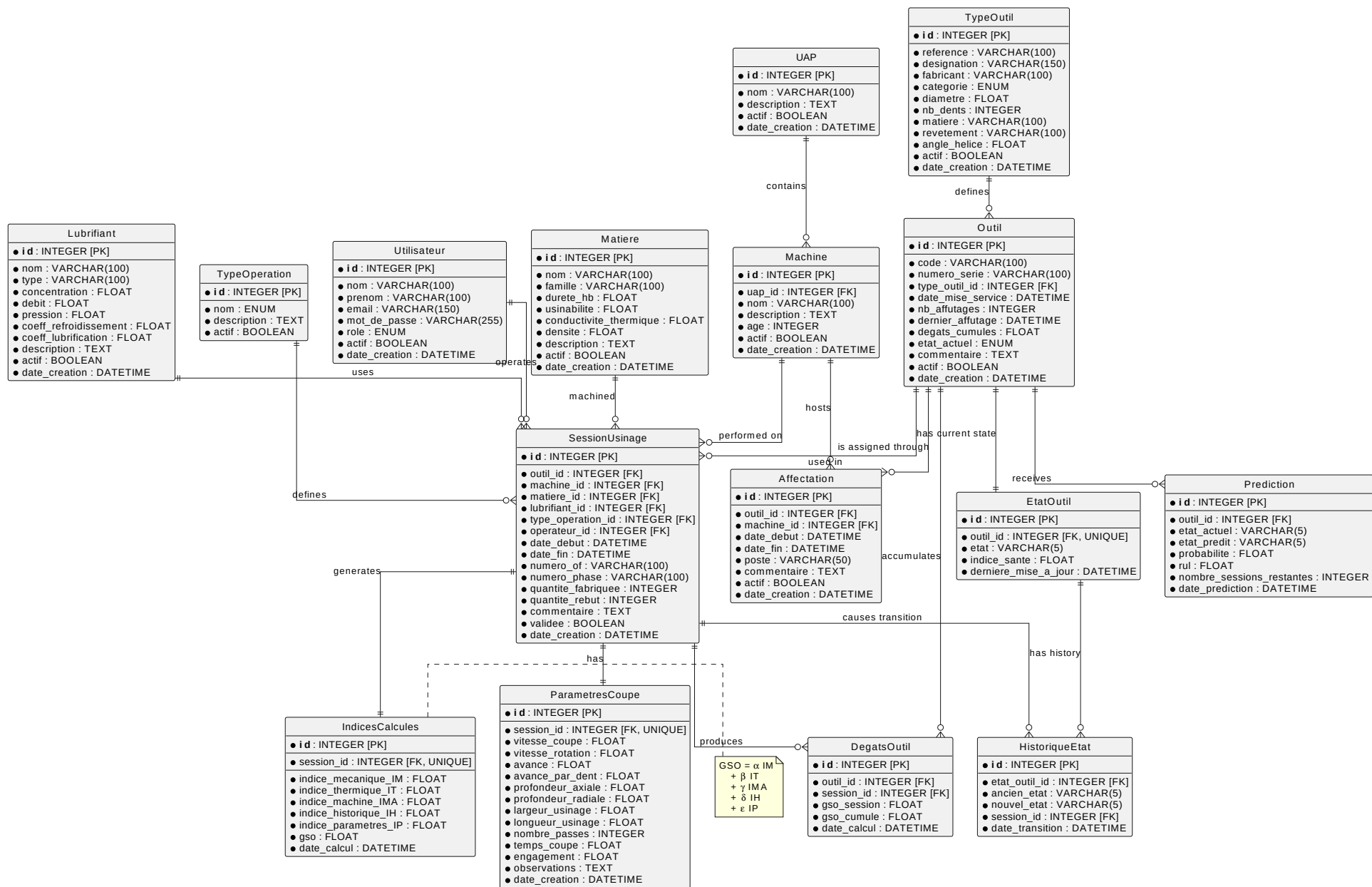


Figure 3.3: Entity-Relationship Model for Item-Level Tool Traceability.

3.4.2 Database Structure and Relationships

The database was designed as a relational data model to ensure the traceability of cutting tools throughout their operational lifecycle and to support the two core analytical layers of the proposed decision-support platform: the GSO-based degradation assessment and the Markov-based health prediction.

Rather than storing all information in a single structure, the database separates master data, operational events, degradation indicators, health states, and prediction results. This organization improves data consistency, avoids unnecessary duplication, and allows the historical evolution of each physical tool to be reconstructed.

The main entities and their relationships are described below.

1. **User (utilisateurs):** Stores the users authorized to interact with the platform. Each user is identified by a unique identifier and is associated with personal information, authentication credentials, and an application role. The main roles are administrator and operator. Operators are associated with machining sessions, while administrators are responsible for system and data management.
2. **UAP (uaps):** Represents the production units within the industrial organization. A UAP can contain several machine centers and provides the organizational context required to locate and classify machines within the production environment.
3. **Machine (machines):** Stores the characteristics of the CNC machine centers on which machining operations are performed. Each machine is associated with a UAP and can be linked to several tool assignments and machining sessions. Machine-related information contributes to the Machine Index (I_{MA}) used by the GSO model.
4. **Tool Type (types_outils):** Defines the generic characteristics of a tool family or tool reference, including its reference, designation, manufacturer, category, diameter, number of teeth, tool material, coating, and helix angle. This entity separates generic tool specifications from the identification of individual physical tools.
5. **Physical Tool (outils):** Represents an individual physical cutting tool identified by a unique code and, when available, a serial number. The entity stores lifecycle information such as commissioning date, number of regrind operations, latest regrinding date, cumulative degradation, and current health state. It therefore constitutes the central entity for tool lifecycle monitoring.
6. **Tool Assignment (affectations):** Records the relationship between a physical tool and a machine over a defined period. Each assignment contains the corresponding tool and machine identifiers, start and end dates, workstation information, and optional comments. This entity preserves the historical context in which a tool was used and contributes to its traceability.
7. **Material (matieres):** Stores the characteristics of the material being machined, such as material family, hardness, machinability, thermal conductivity, and density. These properties provide contextual information for the degradation assessment and contribute to the characterization of machining conditions.
8. **Lubricant (lubrifiants):** Contains information related to the lubrication and cooling conditions used during machining. The stored attributes include lubricant type, concentration, flow rate, pressure, and cooling and lubrication coefficients. These variables provide information for the thermal and process-related assessment of tool degradation.
9. **Operation Type (types_operations):** Defines the type of machining operation associated with a session. This entity allows machining activities to be categorized and provides additional process context for the degradation assessment.

10. **Machining Session (`sessions_usage`):** Represents the central operational event of the database. Each session records a machining activity involving a specific tool, machine, material, lubricant, operation type, and operator. It also stores production information such as the Work Order (OF), phase number, manufactured quantity, rejected quantity, start and end dates, and validation status.
11. **Cutting Parameters (`parametres_coupe`):** Stores the cutting conditions associated with a machining session. The recorded parameters include cutting speed (V_c), spindle speed, feed rate, feed per tooth (f_z), axial and radial depths of cut, machining dimensions, number of passes, cutting time, and engagement. These parameters constitute an important input to the process-parameter and mechanical/thermal degradation indices.
12. **Calculated Indices (`indices_calculés`):** Stores the five constituent indices calculated for each machining session: the Mechanical Index (I_M), Thermal Index (I_T), Machine Index (I_{MA}), Historical Index (I_H), and Process-Parameter Index (I_P). The entity also stores the resulting Global Severity/Degradation Index (GSO).
The GSO is calculated according to the following weighted aggregation model:

$$GSO = \alpha I_M + \beta I_T + \gamma I_{MA} + \delta I_H + \varepsilon I_P \quad (3.2)$$

where α , β , γ , δ , and ε are the weighting coefficients associated with the five degradation dimensions.

13. **Tool Damage (`degats_outils`):** Records the degradation associated with the use of a specific tool during a machining session. It allows the session-level GSO and the cumulative GSO associated with the physical tool to be retained. This entity provides the link between individual machining events and the long-term degradation history of the tool.
14. **Tool State (`etat_outils`):** Stores the current health state of each physical tool and its associated health indicator. The state is updated following the degradation assessment and provides the current condition used as input to the predictive layer.
15. **State History (`historique_etats`):** Maintains the historical transitions of tool health states. Each transition is associated with a tool and, when applicable, with the machining session that caused the state update. This entity makes it possible to reconstruct the evolution of tool health over time.
16. **Prediction (`predictions`):** Stores the outputs generated by the Markov-based prediction layer. For each prediction, the database records the current tool state, predicted future state, associated transition probability, estimated Remaining Useful Life (RUL), and estimated number of remaining machining sessions. Prediction timestamps are also stored to preserve the history of model outputs.

3.4.3 Tool, Machine, and Machining Session Data Management

The database follows a relational structure in which master data, operational data, and analytical results are stored separately while remaining connected through foreign key relationships.

The **Outil** entity represents the physical cutting tool and constitutes the central object of lifecycle monitoring. It is associated with a generic tool type through **TypeOutil** and can be assigned to different machines over time through the **Affectation** entity.

The **SessionUsage** entity represents the main operational event. Each machining session is associated with a physical tool, a machine, an operator, a machined material, a lubricant, and a machining operation type. The session also records production-related information, including the Work Order, phase number, manufactured quantity, rejected quantity, and machining dates.

The cutting conditions associated with a session are stored in the **ParametresCoupe** entity. A one-to-one relationship is used between a machining session and its cutting-parameter record, ensuring that each session has a unique set of recorded process parameters.

The analytical processing is then linked to the machining session. The five degradation indices and the corresponding GSO are stored in **IndicesCalcules**. The resulting degradation is recorded in **DegatsOutil**, while the current tool condition and its historical transitions are managed through **EtatOutil** and **HistoriqueEtat**. Finally, the Markov prediction results are stored independently in **Prediction**.

Foreign key constraints ensure referential integrity between these entities. In particular, analytical records cannot be associated with non-existent tools or machining sessions. This relational organization also facilitates the reconstruction of the complete history of a tool from its identification and assignments to its machining operations, degradation evolution, current state, and predicted future state.

3.4.4 Traceability and Historical Data Management

The proposed database provides end-to-end traceability of each physical cutting tool throughout its operational lifecycle. The traceability model combines tool identification, machine assignments, machining sessions, process conditions, degradation assessment, health-state evolution, and predictive information.

At the operational level, the physical tool is linked to the machines on which it is used through the **Affectation** entity. Each machining session then records the operational context of the tool, including the machine, machined material, lubricant, operation type, operator, Work Order, and machining parameters.

The analytical traceability is established from the machining session toward the degradation and prediction layers:

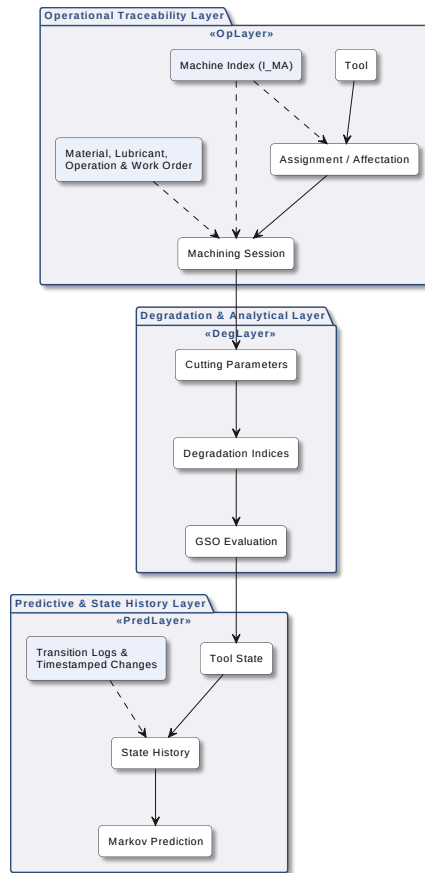


Figure 3.4: End-to-End Analytical Traceability and Historical Data Flow Diagram.

The machine is associated with both the assignment and the machining session, providing the operational context required for the Machine Index (I_{MA}). Similarly, material, lubricant, and operation type are directly

associated with the machining session and provide additional contextual information for the degradation assessment.

This structure makes it possible to answer key traceability questions, such as which machine used a given tool, during which machining sessions, under which process conditions, how the degradation evolved, when the tool changed state, and what future state was predicted by the Markov model.

Historical information is therefore preserved at several levels: operational history through machining sessions, degradation history through calculated GSO values, state history through state transitions, and predictive history through successive Markov predictions.

3.5 Layer 1 GSO-Based Tool Degradation Assessment

3.5.1 Objective and Operating Principle

The first layer of the proposed decision-support architecture is designed to quantify the degradation severity experienced by a cutting tool during machining operations. The main objective is to move beyond a simple time-based estimation of tool life and to account for the multiple physical and operational factors that influence tool degradation.

A purely time-based approach assumes that two tools subjected to the same machining duration experience a comparable level of degradation. This assumption is not sufficiently representative of industrial machining conditions. The same cutting time may correspond to significantly different levels of tool stress depending on the machined material, cutting conditions, machine characteristics, tool history, lubrication conditions, and process parameters.

Therefore, the proposed approach adopts a multi-factor representation of degradation. Five complementary dimensions were selected:

- mechanical loading,
- thermal loading,
- machine-related effects,
- historical tool condition,
- and process-parameter severity.

These dimensions are represented by five constituent indices: I_M , I_T , I_{MA} , I_H , and I_P .

The five indices are subsequently aggregated into a single interpretable indicator, referred to as the Global Severity/Degradation Index (GSO). The GSO provides a unified representation of the severity associated with a machining session and constitutes the principal quantitative output of Layer 1.

The resulting architecture can therefore be summarized as follows:

$$\text{Machining Context} \rightarrow \{I_M, I_T, I_{MA}, I_H, I_P\} \rightarrow GSO \rightarrow \text{Tool Health State} \quad (3.3)$$

The resulting health state is subsequently used by Layer 2, which models the stochastic evolution of the tool state using a Markov Chain.

3.5.2 Engineering Rationale for the Multi-Index Approach

The selection of the five indices was motivated by an engineering analysis of the main sources of variability affecting cutting-tool degradation.

A single indicator based exclusively on machining time would fail to distinguish between different operating conditions. Conversely, directly combining all available production variables into a single black-box

model would reduce interpretability and make it difficult for industrial users to understand the origin of an abnormal degradation score.

The proposed five-index architecture provides an intermediate solution between these two extremes.

First, it preserves the physical and operational meaning of the variables. Second, it allows heterogeneous information originating from different information systems to be transformed into comparable indicators. Third, it provides a modular structure in which individual indices can later be refined without redesigning the entire degradation model.

The five indices were therefore selected according to four main criteria:

1. **Physical relevance:** the index must represent a mechanism or condition that can contribute to tool degradation.
2. **Data availability:** the required variables must be obtainable, directly or indirectly, from the industrial data ecosystem, including machining data, machine information, tool history, and process parameters.
3. **Complementarity:** the indices should capture different degradation dimensions rather than relying on a single repeated measurement.
4. **Interpretability and extensibility:** each contribution to the final GSO must remain understandable to engineers and operators and should be replaceable or refined when additional data become available.

This structure also anticipates future evolution of the system. As additional measurements become available, such as vibration, spindle load, temperature measurements, tool inspection data, or detailed FAO/CNC information, the corresponding index formulation can be refined without changing the overall architecture.

3.5.3 Definition of the Global Severity/Degradation Index (GSO)

The Global Severity/Degradation Index is defined as a weighted aggregation of the five constituent indices:

$$\boxed{GSO = \alpha I_M + \beta I_T + \gamma I_{MA} + \delta I_H + \varepsilon I_P} \quad (3.4)$$

where:

- I_M is the Mechanical Index;
- I_T is the Thermal Index;
- I_{MA} is the Machine Index;
- I_H is the Historical Index;
- I_P is the Process-Parameter Index;
- $\alpha, \beta, \gamma, \delta, \varepsilon$ are non-negative weighting coefficients.

The weighting coefficients satisfy:

$$\alpha + \beta + \gamma + \delta + \varepsilon = 1 \quad (3.5)$$

This normalization ensures that the coefficients represent relative contributions rather than independent scaling factors.

An important design principle is that the five constituent indices should first be transformed into comparable scales before their aggregation. This avoids allowing an index to dominate the GSO simply because it has a larger numerical range.

Thus, the GSO should be interpreted primarily as a **relative severity indicator** rather than as a direct physical measurement of flank wear, cutting-edge temperature, or remaining tool life.

The model is intentionally transparent: each GSO value can be decomposed into five contributions, allowing the system to explain why a particular machining session produced a higher or lower degradation score.

3.5.4 Definition and Engineering Justification of the Five Constituent Indices

Mechanical Index (I_M)

The Mechanical Index represents the mechanical severity imposed on the cutting tool during machining. Mechanical loading is influenced by the cutting conditions, material characteristics, and engagement of the tool with the workpiece.

The initial formulation considers the actual feed per tooth and cutting depths relative to reference values:

$$I_M = \left(\frac{f_z}{f_{z,\text{ref}}} \right) \left(\frac{a_p}{a_{p,\text{ref}}} \right) \left(\frac{a_e}{a_{e,\text{ref}}} \right) K_{\text{mat}} \quad (3.6)$$

where:

- f_z is the actual feed per tooth;
- $f_{z,\text{ref}}$ is the reference feed per tooth;
- a_p is the axial depth of cut;
- $a_{p,\text{ref}}$ is the reference axial depth;
- a_e is the radial width of cut;
- $a_{e,\text{ref}}$ is the reference radial engagement;
- K_{mat} is a material-dependent correction factor.

The multiplication of these normalized terms is intended to represent the combined effect of feed and tool engagement. A machining operation with simultaneously high feed and high engagement is therefore treated differently from an operation in which only one parameter is elevated.

The material correction factor introduces another important dimension. The same geometric cutting conditions do not necessarily impose the same mechanical severity for different workpiece materials. Consequently, material characteristics must be incorporated into the assessment rather than treating all workpieces identically.

The exact values of K_{mat} should be considered calibration parameters rather than universal physical constants. Their values should be established from available industrial knowledge, manufacturer recommendations, machining references, or subsequently validated production data.

Engineering rationale. The Mechanical Index was retained because tool degradation is not governed by machining duration alone. A short operation under highly demanding mechanical engagement conditions may be more severe than a longer operation performed under moderate conditions.

This index therefore introduces the first major improvement over a simple cumulative-time model.

Thermal Index (I_T)

The Thermal Index represents the thermal severity associated with the cutting conditions and cooling/lubrication environment.

An initial formulation is defined as:

$$I_T = \left(\frac{V_c}{V_{c,\text{ref}}} \right)^2 C_{\text{lub}} \quad (3.7)$$

where:

- V_c is the actual cutting speed;
- $V_{c,\text{ref}}$ is the reference cutting speed;
- C_{lub} is a correction coefficient representing the cooling/lubrication condition.

The squared relationship with cutting speed is used as an initial severity assumption to increase the sensitivity of the index to high-speed operating conditions.

The lubrication coefficient allows the same cutting speed to produce different severity values depending on the cooling environment. In the initial conceptual calibration, stronger cooling conditions are associated with a lower thermal penalty, while dry or less effective cooling conditions receive a higher penalty.

However, these coefficients should not be interpreted as universal thermal laws. In the absence of direct temperature measurements, the Thermal Index is a **proxy indicator** for thermal exposure. Its calibration can subsequently be improved using measured spindle load, cutting temperature, infrared measurements, coolant parameters, or other process-monitoring data.

Engineering rationale. The Thermal Index was introduced separately from the Mechanical Index because two operations can have comparable mechanical loading while producing different thermal conditions. This distinction is particularly relevant when cutting speed, material properties, and cooling conditions vary.

The separation also makes the model more interpretable: a high GSO can later be attributed either to mechanical severity, thermal severity, machine effects, historical condition, or process anomalies.

Machine Index (I_{MA})

The Machine Index captures the influence of the machine center on the degradation environment.

The same cutting operation may not produce identical results on two machine centers because machine condition, spindle behavior, rigidity, dynamic stability, and maintenance history can influence machining behavior.

An initial conceptual formulation is:

$$I_{MA} = 1 + \Delta_{\text{machine}} \quad (3.8)$$

where Δ_{machine} represents the aggregated machine-related penalty.

It may be decomposed into several contributions:

$$\Delta_{\text{machine}} = \Delta_{\text{rigidity}} + \Delta_{\text{spindle}} + \Delta_{\text{age}} + \Delta_{\text{condition}} \quad (3.9)$$

depending on the availability and reliability of the corresponding data.

The machine index is therefore designed as a modular component. In the initial prototype, only machine characteristics that can be reliably obtained should be used. Additional information, such as spindle runout measurements, vibration indicators, machine maintenance history, or machine-specific quality indicators, can subsequently be incorporated.

Engineering rationale. This index reflects an important industrial consideration: the tool cannot be analyzed independently from the equipment on which it operates.

The Machine Index also provides a mechanism for identifying systematic differences between machine centers. If similar tools operating under comparable machining conditions systematically exhibit different

degradation patterns on different machines, the machine contribution can become an important diagnostic signal.

Historical Index (I_H)

The Historical Index represents the accumulated condition of the physical tool before the current machining session.

Unlike the other indices, which mainly describe the current operating context, I_H introduces a **memory effect** into Layer 1.

The historical index can incorporate information such as:

- cumulative degradation from previous sessions;
- number of regrinding operations;
- previous high-severity machining events;
- previous state transitions;
- tool usage history.

A conceptual formulation is:

$$I_H = f \left(GSO_{\text{cumul}}^{(k-1)}, N_{\text{regrind}}, H_{\text{usage}} \right) \quad (3.10)$$

where $GSO_{\text{cumul}}^{(k-1)}$ represents the cumulative degradation available **before** machining session k .

For an initial implementation, a simplified formulation may be adopted:

$$I_H = (1 + \lambda_r N_{\text{regrind}}) \left(\frac{GSO_{\text{cumul}}^{(k-1)}}{GSO_{\text{ref}}} \right) \quad (3.11)$$

where λ_r is a calibration coefficient and GSO_{ref} is a reference cumulative degradation level.

The use of $GSO_{\text{cumul}}^{(k-1)}$, rather than the GSO calculated during the current session, is essential. It prevents circularity because the current GSO depends on I_H , while I_H itself must only depend on information available before the current calculation.

Engineering rationale. The historical index is one of the most important elements of the proposed approach because the same machining operation can have different consequences depending on the previous life of the tool.

A new tool and a tool that has already undergone several machining cycles and regrinding operations should not necessarily receive the same degradation assessment under identical current operating conditions.

The historical index therefore introduces a form of **tool memory** into the deterministic Layer 1 model.

Process-Parameter Index (I_P)

The Process-Parameter Index evaluates the severity of the actual process configuration relative to the expected or validated operating window.

Its purpose is deliberately different from the Mechanical Index. While I_M attempts to characterize the mechanical loading imposed on the tool, I_P focuses on whether the actual process parameters deviate from their reference or acceptable operating conditions.

A simplified anomaly-oriented formulation can be written as:

$$I_P = \max \left(1, \frac{V_c f_z a_p}{V_{c,\text{ref}} f_{z,\text{ref}} a_{p,\text{ref}}} \right) \quad (3.12)$$

However, in the final implementation, the formulation should preferably be adapted to the actual process rules and available FAO/CNC information.

The index can incorporate:

- cutting-speed deviation;
- feed deviation;
- depth-of-cut deviation;
- excessive engagement;
- deviation from recommended CAM conditions;
- abnormal combinations of parameters.

Engineering rationale. The distinction between I_M and I_P is intentional.

- I_M answers: *How mechanically demanding is the operation?*
- I_P answers: *How far does the actual process configuration deviate from the expected operating window?*

This distinction allows the model to identify not only severe operations, but also potentially abnormal or non-standard operating conditions.

3.5.5 Complementarity of the Five Indices

The five indices were not selected simply to increase the number of variables in the model. Their purpose is to represent different dimensions of tool degradation and operational risk.

Table 3.1 summarizes their respective roles.

Table 3.1: Engineering rationale behind the five GSO constituent indices

Index	Main question addressed	Engineering contribution
I_M	How mechanically demanding is the operation?	Represents feed, engagement, depth of cut, and material-related mechanical severity.
I_T	How severe is the thermal environment?	Represents cutting-speed and cooling/lubrication-related thermal exposure.
I_{MA}	How does the machine influence the operation?	Accounts for machine-specific characteristics and condition.
I_H	What is the previous condition of the tool?	Introduces tool-life memory through cumulative degradation and regrinding history.
I_P	Is the process operating within its expected window?	Detects deviations and potentially abnormal combinations of process parameters.

This decomposition provides an important interpretability advantage. Instead of producing a single unexplained degradation score, the platform can identify the dominant contributors to a high GSO value.

For example, two machining sessions may produce the same GSO value while having completely different compositions:

$$GSO_A = \text{high } I_M + \text{moderate } I_T + \text{low } I_{MA} + \text{low } I_H + \text{moderate } I_P \quad (3.13)$$

whereas another session may produce:

$$GSO_B = \text{moderate } I_M + \text{high } I_T + \text{low } I_{MA} + \text{high } I_H + \text{moderate } I_P \quad (3.14)$$

The identical aggregate value would therefore correspond to different engineering situations. Retaining the constituent indices allows the platform to preserve this information rather than losing it during aggregation.

3.5.6 Weighting Coefficients and Aggregation Mechanism

The weighting coefficients determine the relative contribution of each degradation dimension to the final GSO:

$$\alpha + \beta + \gamma + \delta + \varepsilon = 1 \quad (3.15)$$

The initial weighting configuration was established using engineering reasoning and domain knowledge rather than being presented as a universally optimal set of parameters.

For the initial prototype, the following baseline configuration can be considered:

$$\alpha = 0.30, \quad \beta = 0.25, \quad \gamma = 0.15, \quad \delta = 0.15, \quad \varepsilon = 0.15 \quad (3.16)$$

This configuration gives greater initial importance to mechanical and thermal severity while maintaining explicit contributions from machine condition, tool history, and process-parameter deviations.

The choice of weights is intentionally kept transparent. It allows the model to be modified when sufficient production feedback becomes available.

The weighting strategy is therefore considered a first calibration stage rather than a final optimization. Future calibration can rely on:

- historical tool failures;
- observed tool-state transitions;
- tool inspection or wear measurements;
- expert feedback;
- sensitivity analysis;
- statistical parameter estimation;
- and, in a later stage, adaptive learning methods.

This design creates a direct path between the interpretable expert-based prototype and a future data-driven calibration strategy.

3.5.7 Sensitivity and Criticality Analysis of the GSO Model

Because the GSO is a weighted aggregation model, its behavior depends on both the constituent indices and the selected coefficients.

A sensitivity analysis can therefore be used to evaluate how variations in each index influence the final score.

For a small variation ΔI_j in one constituent index, the corresponding first-order change in GSO is:

$$\Delta GSO \approx w_j \Delta I_j \quad (3.17)$$

where w_j is the corresponding weighting coefficient.

Thus:

3

- S_0 : New
- S_1 : Normal Wear
- S_2 : Monitoring
- S_3 : Critical / Regrinding
- S_4 : Out of Service / Broken

This provides a simple and interpretable way to identify which dimensions have the greatest influence on the output under the current calibration.

The sensitivity analysis also provides a mechanism for detecting inappropriate parameterization. If a small change in a single index produces an unrealistic change in the GSO, the corresponding normalization, weighting, or index formulation should be reviewed.

3.5.8 GSO Calculation and Cumulative Degradation Assessment

For each machining session k , the five constituent indices are first calculated from the information available for that session and from the tool's previous history.

The session-level GSO is then calculated as:

$$GSO^{(k)} = \alpha I_M^{(k)} + \beta I_T^{(k)} + \gamma I_{MA}^{(k)} + \delta I_H^{(k)} + \varepsilon I_P^{(k)} \quad (3.18)$$

When machining duration is available, it can be used to convert session severity into cumulative exposure. The degradation contribution of session k may therefore be represented as:

$$\Delta D^{(k)} = GSO^{(k)} t_c^{(k)} \quad (3.19)$$

where $t_c^{(k)}$ is the effective cutting duration of the session.

The cumulative degradation is then updated according to:

$$D_{\text{cumul}}^{(k)} = D_{\text{cumul}}^{(k-1)} + \Delta D^{(k)} \quad (3.20)$$

This distinction between **session severity** and **cumulative exposure** is important.

The GSO characterizes the severity of the current machining context, whereas the cumulative degradation indicator represents the accumulated exposure of the physical tool throughout its operational history.

The use of effective cutting time rather than simply elapsed calendar time is intended to avoid assigning degradation to periods during which the tool was not actively cutting.

3.5.9 Tool Health State Determination

The continuous degradation indicator must ultimately be translated into a discrete state representation so that it can be used by the predictive Layer 2.

The proposed state space is:

$$S = \{S_0, S_1, S_2, S_3, S_4\} \quad (3.21)$$

The states represent successive levels of tool condition: **Tool States:** S_0 : New • S_1 : Normal Wear • S_2 : Monitoring • S_3 : Critical / Regrinding-related condition • S_4 : Out of Service / Broken.

The state assignment is based on predefined degradation thresholds.

Table 3.2: Initial mapping between cumulative degradation and tool health states

State	Health Label	Physical Interpretation	Initial Threshold Range
S_0	New	New or recently reconditioned tool with minimal accumulated degradation.	$[0, 20)$
S_1	Normal Wear	Tool operating under an expected wear regime.	$[20, 50)$
S_2	Monitoring	Increased degradation requiring closer monitoring.	$[50, 75)$
S_3	Critical	High degradation requiring maintenance, re-grinding, or a replacement decision.	$[75, 95)$
S_4	Out of Service / Broken	Tool has reached the predefined removal or failure condition.	≥ 95

These thresholds should be considered **initial calibration thresholds**. They are not assumed to represent universal physical boundaries. Their final calibration requires comparison with actual tool observations, inspection data, operator feedback, and historical failure or regrinding events.

This distinction is important because the GSO is an engineering decision-support indicator rather than a direct measurement of physical wear.

3.5.10 From Continuous Degradation to Discrete State Prediction

The transformation from GSO to discrete health states provides the interface between the deterministic Layer 1 and the stochastic Layer 2.

The process can be represented as:

$$\text{Machining Conditions} \rightarrow \text{Five Indices} \rightarrow \text{GSO} \rightarrow \text{Health State} \rightarrow \text{Markov Prediction} \quad (3.22)$$

Layer 1 therefore answers:

“What is the current degradation severity and health state of the tool?”

Layer 2 subsequently addresses:

“Given the current state and historical transitions, how is the tool likely to evolve?”

This separation is fundamental to the architecture. The GSO layer provides an interpretable state estimation mechanism, while the Markov layer introduces temporal and probabilistic reasoning.

3.5.11 Critical Analysis and Anticipated Limitations

The proposed GSO model provides several advantages, but its assumptions must be explicitly recognized.

First, the five indices are not direct physical measurements of tool wear. They are engineering indicators constructed from available operational information. Consequently, their accuracy depends on the quality and representativeness of the underlying data.

Second, some variables may influence more than one index. For example, cutting speed contributes to thermal severity but may also influence mechanical behavior. Similarly, process parameters can affect both the mechanical and thermal dimensions. The decomposition therefore does not imply that the five physical phenomena are completely independent.

Third, the weighted linear aggregation assumes that the contributions can be reasonably combined using a weighted additive model. This provides excellent interpretability but may not capture all nonlinear interactions between machining variables.

Fourth, historical data may be incomplete. Missing information concerning previous tool usage, regrinding, or failure events can affect I_H and consequently the GSO.

Finally, the initial coefficients and thresholds require validation against real industrial observations. The purpose of the prototype is therefore not to claim a universally optimal degradation law, but to establish an interpretable and extensible computational framework that can progressively be calibrated as reliable production data become available.

3.5.12 Anticipated Evolution of the GSO Model

The architecture was deliberately designed to support progressive improvement without changing the fundamental data flow.

The initial implementation relies on expert-defined rules and weighting coefficients. Once sufficient historical data become available, the parameters can be refined through statistical and data-driven methods.

The expected evolution is therefore:

$$\boxed{\text{Expert Knowledge} \rightarrow \text{Calibrated GSO} \rightarrow \text{Data-Assisted GSO} \rightarrow \text{Adaptive Model}} \quad (3.23)$$

Potential future improvements include the integration of additional information such as spindle load, vibration, temperature measurements, detailed CNC/FAO data, inspection measurements, and operator feedback.

In a later stage, adaptive learning methods could be used to update the weighting coefficients or individual index formulations according to observed production outcomes. This constitutes a future extension of the current prototype rather than part of the validated core implementation.

3.5.13 Summary of Layer 1

Layer 1 transforms heterogeneous manufacturing information into an interpretable degradation representation.

The complete computational chain is:

$$\boxed{\{I_M, I_T, I_{MA}, I_H, I_P\} \rightarrow GSO \rightarrow D_{\text{cumul}} \rightarrow S_0, \dots, S_4} \quad (3.24)$$

The resulting tool state constitutes the interface with Layer 2. By combining multi-dimensional degradation assessment with historical state information, the proposed approach provides a structured foundation for probabilistic prediction of future tool health.

3.6 Layer 2 — Stochastic Tool Health Prediction

3.6.1 Objective and Operating Principle

While Layer 1 provides an interpretable estimation of the current degradation severity and maps the tool to a discrete health state, it does not directly answer a forward-looking question: *how is the tool expected to evolve during the next machining sessions?*

Layer 2 addresses this predictive dimension by modeling the evolution of the tool health state as a Discrete-Time Markov Chain (DTMC). The basic principle is to transform the current state estimated by the GSO-based degradation engine into a probabilistic distribution of future states.

The complete logic can therefore be summarized as:

$$\text{Machining data} \rightarrow \text{Layer 1} \rightarrow \text{Current health state} \rightarrow \text{Layer 2} \rightarrow \text{Future state probabilities, failure risk and RUL} \quad (3.25)$$

This separation between assessment and prediction is important from an engineering perspective. Layer 1 evaluates the current condition of the tool, whereas Layer 2 estimates its future evolution. The two layers therefore perform complementary functions rather than duplicating the same calculation.

3.6.2 Markov Chain Modeling Principle

Cutting tool degradation is affected by several sources of variability, including workpiece material variability, changing cutting conditions, machine operating conditions, tool history, and possible dynamic phenomena such as vibration or chatter. Consequently, two tools having the same current health state may not necessarily evolve in exactly the same way.

A deterministic prediction would therefore provide a limited representation of the uncertainty associated with tool degradation. A probabilistic approach is more appropriate for the decision-support objective of the proposed platform.

The tool health evolution is consequently modeled as a Discrete-Time Markov Chain. Each machining session represents one discrete transition step. Let X_k denote the health state of a tool after the k -th machining session. The Markov property is expressed as:

$$P(X_{k+1} = S_j \mid X_k = S_i, X_{k-1}, \dots, X_0) = P(X_{k+1} = S_j \mid X_k = S_i) = p_{ij} \quad (3.26)$$

where p_{ij} represents the probability that the tool moves from state S_i to state S_j during the next machining session.

The Markov assumption does not imply that the physical degradation process has no memory. Rather, it assumes that the information required for the short-term state transition is sufficiently summarized by the current discrete health state. This is a modeling simplification that makes the prediction problem tractable and interpretable.

The adequacy of this assumption should therefore be reassessed when a sufficient amount of production history becomes available. If historical duration, cumulative degradation, or process conditions are shown to have a significant influence beyond the current state, an extended state model or a higher-order stochastic model could subsequently be considered.

3.6.3 Definition of the Tool Health State Space

The Markov model operates on the same health-state space defined by Layer 1:

$$S = \{S_0, S_1, S_2, S_3, S_4\} \quad (3.27)$$

The states represent successive levels of tool degradation:

- S_0 : New tool;
- S_1 : Normal wear;
- S_2 : Tool under monitoring;
- S_3 : Critical degradation / end-of-life approaching;
- S_4 : Out of service / broken.

The use of a common state space between the two layers is a key architectural choice. Layer 1 continuously estimates the current state from machining and tool-related information, while Layer 2 uses this state as the initial condition for stochastic forecasting.

The first four states are considered transient states, while S_4 is modeled as an absorbing state:

$$P(X_{k+1} = S_4 \mid X_k = S_4) = 1 \quad (3.28)$$

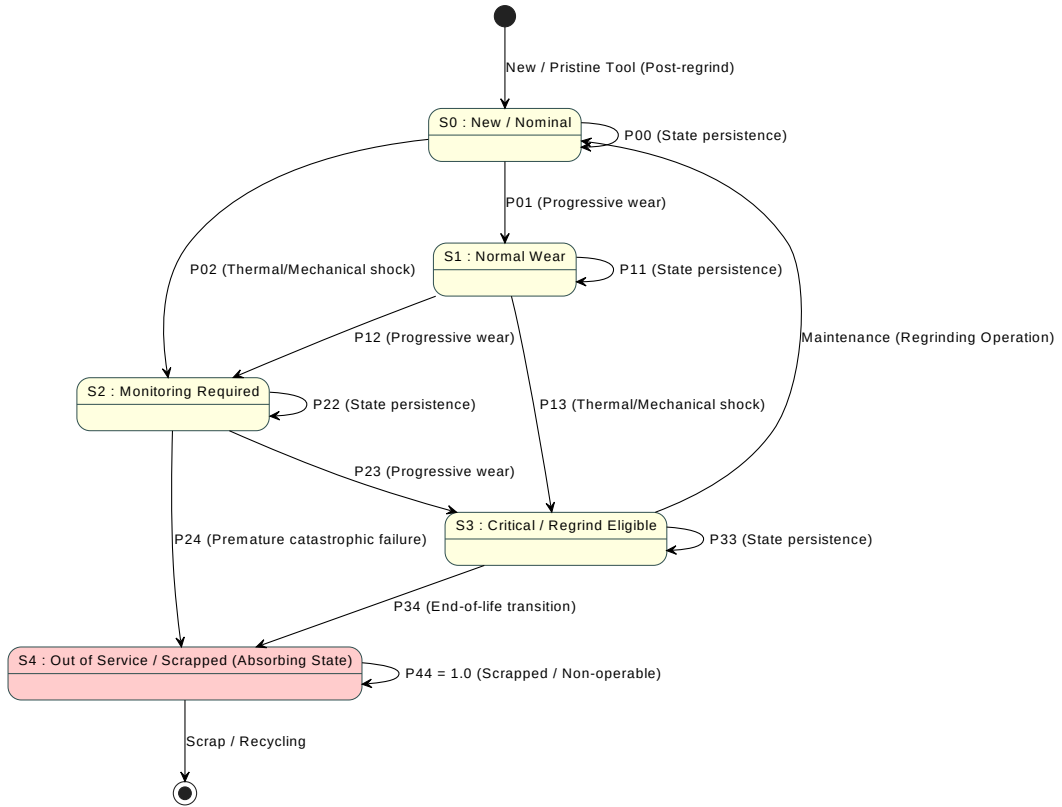


Figure 3.5: State transition diagram of the Discrete-Time Markov Chain representing tool health states (S_0 to S_4), including state persistence, progressive wear, severe degradation jumps, and the absorbing state.

This means that, once a tool reaches the failure state in the predictive model, it remains classified as failed until an external maintenance action such as replacement or reconditioning changes its operational status.

It is important to distinguish this mathematical assumption from the physical possibility of tool reconditioning. Regrinding or tool replacement represents a maintenance event and should therefore be modeled as a new lifecycle or as an explicit state-reset mechanism rather than as a spontaneous transition from S_4 to a healthier state.

3.6.4 Construction of the Transition Probability Matrix

The evolution of the health states is represented by a 5×5 transition probability matrix:

$$P = \begin{bmatrix} p_{00} & p_{01} & p_{02} & p_{03} & p_{04} \\ 0 & p_{11} & p_{12} & p_{13} & p_{14} \\ 0 & 0 & p_{22} & p_{23} & p_{24} \\ 0 & 0 & 0 & p_{33} & p_{34} \\ 0 & 0 & 0 & 0 & 1 \end{bmatrix} \quad (3.29)$$

Each row satisfies the stochastic constraint:

$$\sum_{j=0}^4 p_{ij} = 1 \quad \forall i \in \{0, \dots, 4\} \quad (3.30)$$

and:

$$0 \leq p_{ij} \leq 1 \quad (3.31)$$

The upper-triangular structure reflects the initial assumption that, during a normal machining lifecycle, degradation progresses from healthier states toward more degraded states:

$$S_0 \rightarrow S_1 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \quad (3.32)$$

Consequently:

$$p_{ij} = 0 \quad \text{for } j < i \quad (3.33)$$

This constraint provides a simple and physically interpretable baseline for the prototype. However, it should not be interpreted as a universal law of tool behavior. For example, an explicit regrinding operation may restore part of the tool's usable condition. Such maintenance actions are therefore better represented by lifecycle reset rules or additional transition mechanisms rather than by violating the degradation model.

For the prototype, the transition probabilities are initially obtained through expert calibration because a sufficiently long and reliable run-to-failure dataset was not yet available. The matrix should subsequently be estimated from observed transitions extracted from production history.

For a sufficiently large dataset, the empirical probability of a transition from S_i to S_j can be estimated as:

$$\hat{p}_{ij} = \frac{N_{ij}}{\sum_{k=0}^4 N_{ik}} \quad (3.34)$$

where N_{ij} is the number of observed transitions from state S_i to state S_j .

This provides a direct path toward data-driven calibration of the Markov model once sufficient historical observations become available.

3.6.5 Initial Expert-Calibrated Transition Model

Because the industrial dataset available during the prototype phase does not yet provide a sufficiently large number of complete tool lifecycles, the transition matrix is initially defined using expert knowledge and engineering assumptions.

This approach should be understood as a baseline probabilistic model rather than as a statistically validated representation of FGAT's complete production population.

The role of this initial matrix is therefore threefold:

1. validate the mathematical and software architecture of Layer 2;
2. demonstrate future-state and failure-risk calculations;
3. provide an initial probabilistic prior that can later be replaced or calibrated using production observations.

Once sufficient historical data become available, the transition matrix can be progressively recalibrated using observed state sequences. This evolution would transform the current expert-based prototype into a statistically estimated predictive model.

3.6.6 Future Tool-State Prediction

Let the current probability distribution over the five health states be:

$$\pi(0) = [\pi_0 \quad \pi_1 \quad \pi_2 \quad \pi_3 \quad \pi_4] \quad (3.35)$$

For a tool whose current state is known with certainty, the initial distribution can be represented as a one-hot vector. For example, if the current state is S_2 :

$$\pi(0) = \begin{bmatrix} 0 & 0 & 1 & 0 & 0 \end{bmatrix} \quad (3.36)$$

The probability distribution after n future machining sessions is then obtained by:

$$\pi(n) = \pi(0)P^n \quad (3.37)$$

where P^n represents the n -step transition probability matrix.

The resulting vector:

$$\pi(n) = \begin{bmatrix} P(X_n = S_0) & P(X_n = S_1) & P(X_n = S_2) & P(X_n = S_3) & P(X_n = S_4) \end{bmatrix} \quad (3.38)$$

provides a probabilistic description of the expected health distribution after n future machining sessions.

This representation is more informative than a single deterministic state prediction because it explicitly preserves uncertainty. For example, the model may indicate that after five sessions the tool has a probability of remaining in a monitoring state while simultaneously having a non-negligible probability of reaching the critical or failure state.

3.6.7 Failure-Risk Estimation

The absorbing state S_4 provides a natural representation of tool failure. The probability of failure after n future machining sessions is therefore obtained from the fifth component of the predicted probability vector:

$$R_{\text{failure}}(n) = P(X_n = S_4 \mid X_0 = S_i) = \pi_4(n) \quad (3.39)$$

The evolution of this probability can be monitored over several prediction horizons:

$$R_{\text{failure}}(1), R_{\text{failure}}(2), \dots, R_{\text{failure}}(n) \quad (3.40)$$

This makes it possible to distinguish between immediate and medium-term risk. A low probability at one session may still become significant over a longer prediction horizon.

The failure-risk indicator is therefore intended as a decision-support measure rather than a deterministic statement that a tool will fail at a specific session.

3.6.8 Remaining Useful Life Estimation

In addition to failure probability, the Markov model provides an estimate of the expected Remaining Useful Life (RUL). In this work, RUL is first expressed as the expected number of remaining machining sessions before reaching the absorbing failure state.

The transition matrix can be partitioned into transient and absorbing components:

$$P = \begin{bmatrix} Q & R \\ \mathbf{0} & 1 \end{bmatrix} \quad (3.41)$$

where:

- Q is the 4×4 matrix describing transitions between transient states $S_0, \dots, S_3$;
- R contains transitions from transient states to the absorbing failure state S_4 .

The fundamental matrix of the absorbing Markov chain is:

$$N = (I - Q)^{-1} \quad (3.42)$$

where I is the 4×4 identity matrix.

Each element N_{ij} represents the expected number of visits to transient state S_j when starting from transient state S_i , before absorption.

Consequently, the expected number of remaining machining sessions before failure is obtained by summing the corresponding row:

$$\text{RUL}(S_i) = \sum_{j=0}^3 N_{ij} \quad (3.43)$$

Equivalently:

$$\text{RUL} = N\mathbf{1} \quad (3.44)$$

where $\mathbf{1}$ is a column vector of ones.

This formulation is particularly useful because it provides an interpretable expected lifetime for each current health state. Nevertheless, the result must be interpreted as an expected value rather than an exact lifetime prediction.

3.6.9 Conversion of RUL into Estimated Cutting Time

The session-based RUL can be converted into an approximate cutting-time indicator using an estimated average cutting duration:

$$\text{RUL}_{\text{time}} = \text{RUL}(S_i) \times \bar{t}_c \quad (3.45)$$

where $\bar{t}_c$ represents the estimated average effective cutting time per future machining session.

The use of an average duration is a practical simplification. In real production, machining sessions can have significantly different durations. Consequently, the resulting value should be interpreted as an estimated remaining machining exposure rather than an exact calendar lifetime.

A future refinement could replace the constant $\bar{t}_c$ with a context-dependent estimate derived from the work order, machining operation, part reference, or historical session durations.

3.6.10 Relationship Between Layer 1 and Layer 2

The interface between the two layers is one of the central architectural principles of the proposed system.

Layer 1 calculates the current degradation condition:

$$\{I_M, I_T, I_{MA}, I_H, I_P\} \rightarrow GSO \rightarrow D_{\text{cumul}} \rightarrow S_i \quad (3.46)$$

Layer 2 then uses the resulting state S_i as the initial condition of the stochastic prediction:

$$S_i \rightarrow P \rightarrow \pi(n) \rightarrow R_{\text{failure}}(n), \text{RUL} \quad (3.47)$$

This separation avoids mixing the two objectives. The GSO engine estimates *current degradation severity*, whereas the Markov engine estimates *future state evolution*.

The resulting decision-support chain can therefore be summarized as:

$$\boxed{\text{Process Data} \rightarrow \text{GSO} \rightarrow \text{Current State} \rightarrow \text{Markov Chain} \rightarrow \text{Risk} + \text{RUL}} \quad (3.48)$$

3.6.11 Handling Uncertainty and Limited Historical Data

One of the principal limitations of the initial predictive model is the availability of sufficiently long and reliable tool lifecycle histories.

A complete empirical Markov model requires sequences such as:

$$S_0 \rightarrow S_1 \rightarrow S_1 \rightarrow S_2 \rightarrow S_2 \rightarrow S_3 \rightarrow S_4 \quad (3.49)$$

observed over a sufficiently large number of tools and machining sessions.

If only a small number of transitions are available, direct frequency-based estimation of p_{ij} may lead to unstable or poorly representative probabilities. The expert-calibrated matrix therefore constitutes an initial baseline for the prototype.

As production data accumulate, the transition matrix can be progressively updated using:

- observed state-transition frequencies;
- tool-specific historical degradation;
- machining session duration;
- regrinding history;
- failure and replacement events;
- material and machining-condition information.

This evolution is important because the predictive quality of the Markov model depends directly on the quality and representativeness of the observed state-transition history.

3.6.12 Model Assumptions and Critical Analysis

The proposed Markov formulation provides an interpretable and computationally lightweight prediction mechanism, but several assumptions must be explicitly recognized.

First, the model assumes that the current discrete state contains sufficient information to describe the short-term future evolution. In reality, two tools belonging to the same state may have different degradation histories.

Second, the current model uses a fixed transition matrix. Therefore, it does not explicitly adapt the transition probabilities to every machining condition. A future context-dependent Markov model could instead use:

$$P = P(X_{k+1} \mid X_k, \mathbf{z}_k) \quad (3.50)$$

where $\mathbf{z}_k$ represents contextual variables such as material, cutting speed, feed, tool type, machine, or operation.

Third, the model discretizes a continuous degradation process into five states. This improves interpretability and facilitates decision-making, but it inevitably introduces information loss.

Finally, the absorbing-state assumption is appropriate for representing the end of a tool lifecycle, but maintenance actions such as sharpening, regrinding, or tool replacement must be treated explicitly in the lifecycle logic.

These limitations do not invalidate the prototype. Instead, they define a clear path for progressive improvement once sufficiently rich industrial data become available.

3.6.13 Future Evolution of the Predictive Layer

The current Markov model establishes a transparent stochastic baseline that can later be extended without changing the overall system architecture.

Possible developments include:

- empirical estimation of transition probabilities from production data;
- Bayesian updating of transition probabilities as new observations become available;
- context-dependent transition matrices based on machining conditions;
- semi-Markov models incorporating the duration spent in each state;
- continuous degradation indicators combined with discrete health states;
- integration of sensor-derived information such as vibration, spindle load, or temperature when available.

These extensions remain outside the implemented scope of the present prototype. They constitute possible directions for increasing predictive accuracy after sufficient industrial data have been collected.

3.7 Functional Architecture and Main Use Cases

3.7.1 Overview of the Functional Architecture

The functional architecture translates the mathematical framework into operational functions accessible through the web platform. The system is organized around the complete lifecycle of an individual physical cutting tool, from identification and assignment to machining-session recording, degradation assessment, state prediction, and historical traceability.

The principal functional sequence is:

$$\text{Tool ID} \rightarrow \text{Machining Session} \rightarrow \text{GSO Assessment} \rightarrow \text{State Update} \rightarrow \text{Markov Prediction} \rightarrow \text{Decision Support} \quad (3.51)$$

This architecture ensures that predictive information remains linked to the physical tool and its machining history.

3.7.2 User Roles and Access Management

The platform adopts a role-based access control principle in order to separate operational data entry from configuration and decision-support functions.

The main user profiles are:

- **Administrator:** manages users, reference data, machine configuration, tool catalogs, and system-level parameters such as the initial GSO weighting coefficients;
- **Operator:** records machining-session information and operational data associated with the tool being used;
- **Methods / Tool Management User:** consults tool histories, degradation indicators, prediction results, and risk information to support maintenance and tooling decisions.

The separation of these roles is intended to prevent uncontrolled modification of model parameters while keeping shop-floor data entry simple.

3.7.3 Tool Identification and Traceability

Each physical cutting tool is associated with a unique identifier (`Tool_ID`). A QR-code-based identification mechanism can be used to facilitate rapid access to the corresponding digital record.

Once identified, the platform can retrieve the tool's principal information, including:

- tool type and technical characteristics;
- current machine assignment;
- machining-session history;
- cumulative degradation information;
- current health state;
- historical state evolution;
- latest prediction results.

The purpose of this mechanism is to establish a digital link between the physical cutting tool and its lifecycle information in the database.

3.7.4 Machining Session Recording

Each machining session represents an observation used by the degradation and prediction pipeline.

The recorded information may include:

Parameters: Work Order identifier • machining operation • machine • tool • workpiece material • cutting speed V_c • feed per tooth f_z • axial depth of cut a_p • radial engagement a_e • effective cutting time t_c • manufactured and rejected quantities when available.

The database structure is designed to preserve these observations as historical records rather than replacing previous values. This is essential because the predictive model requires a chronological sequence of tool states and machining events.

3.7.5 GSO Calculation and Tool-State Update

After a machining session has been recorded and validated, Layer 1 processes the relevant parameters to calculate the five degradation indices:

$$I_M, \quad I_T, \quad I_{MA}, \quad I_H, \quad I_P \quad (3.52)$$

The Global Severity of Operation is then obtained from:

$$GSO = \alpha I_M + \beta I_T + \gamma I_{MA} + \delta I_H + \varepsilon I_P \quad (3.53)$$

The resulting session severity contributes to the cumulative degradation indicator defined in Layer 1. The updated cumulative condition is then mapped to the corresponding health state:

$$D_{\text{cumul}} \rightarrow S_i \quad (3.54)$$

This state becomes the input of the Markov prediction engine.

3.7.6 Markov Prediction and Risk Assessment

Once the current state has been updated, Layer 2 evaluates the future evolution of the tool using the transition matrix P .

The system can calculate:

- future state probabilities;
- probability of reaching the failure state;

- expected remaining number of machining sessions;
- estimated remaining cutting time.

The prediction output is therefore not limited to a single label. Instead, the platform provides probabilistic information that can support different maintenance horizons.

For example, a configurable warning rule may be defined as:

$$R_{\text{failure}}(n) > \tau_{\text{risk}} \quad (3.55)$$

where τ_{risk} is a configurable decision threshold.

For the prototype, a value such as 20% within two future sessions can be used as an illustrative demonstration threshold. This value must not, however, be interpreted as an experimentally validated industrial threshold. Its calibration should eventually be based on production consequences, maintenance costs, quality requirements, and observed failure behavior.

3.7.7 Monitoring and Decision-Support Dashboard

The web dashboard provides a visual representation of the information generated by the two analytical layers.

At tool level, the dashboard can display:

- current health state;
- cumulative degradation indicator;
- latest GSO value;
- recent machining history;
- predicted future states;
- failure-risk probability;
- estimated RUL.

At a broader production level, aggregated indicators can provide an overview of the distribution of active tools across the health states:

$$\{S_0, S_1, S_2, S_3, S_4\} \quad (3.56)$$

Such visualization can help identify tools requiring closer monitoring and support prioritization of maintenance or tooling actions.

The dashboard should therefore be considered as a decision-support interface, not as an autonomous maintenance decision-maker. Final actions remain linked to operational constraints, tool availability, machining requirements, and human expertise.

3.7.8 End-to-End Functional Workflow

The complete functional workflow implemented by the prototype can be summarized as follows:

1. The physical tool is identified using its unique identifier.
2. The tool is associated with the relevant machine and machining operation.
3. The machining session and process parameters are recorded.

4. Layer 1 calculates the five degradation indices.
5. The GSO is calculated and the cumulative degradation indicator is updated.
6. The resulting health state is determined.
7. Layer 2 receives the current health state.
8. The Markov model computes future-state probabilities.
9. Failure risk and expected RUL are estimated.
10. The results are stored and displayed through the decision-support interface.

This workflow establishes a continuous analytical chain between shop-floor observations and predictive information:

$$\boxed{\text{Tool} \rightarrow \text{Session} \rightarrow \text{GSO} \rightarrow \text{Health State} \rightarrow \text{Markov} \rightarrow \text{Risk/RUL}} \quad (3.57)$$

3.8 Conclusion

This chapter established the design and mathematical framework of the proposed cutting-tool decision-support platform.

The solution combines two complementary analytical layers. The first layer provides an interpretable assessment of current tool degradation through the five-index GSO model:

$$GSO = \alpha I_M + \beta I_T + \gamma I_{MA} + \delta I_H + \varepsilon I_P \quad (3.58)$$

The second layer extends this assessment into a probabilistic forecasting mechanism based on a Discrete-Time Markov Chain. Starting from the current health state supplied by Layer 1, it estimates future-state probabilities, failure risk, and expected Remaining Useful Life.

The proposed architecture deliberately separates current-condition assessment from future-condition prediction. This separation improves interpretability and makes it possible to progressively enhance each layer as additional industrial data become available.

The current Markov model relies on an expert-calibrated transition matrix due to the limited availability of complete historical tool lifecycles. It therefore constitutes a predictive prototype and a methodological baseline, rather than a fully statistically calibrated industrial model. Future production data can progressively replace the initial assumptions through empirical transition estimation and model calibration.

The next chapter presents the concrete software implementation of this framework, including the Flask backend, PostgreSQL database, REST API, analytical processing pipeline, testing procedures, and validation of the functional prototype.

Chapter 4

Implementation, Testing and System Integration

4.1 Introduction

Following the requirements analysis and the system design presented in Chapter 3, this chapter describes the concrete implementation of the developed tool lifecycle monitoring platform. The objective is not to reformulate the mathematical models or the conceptual architecture introduced previously, but to explain how these elements were translated into executable software components, database structures, services, API routes, and web interfaces.

The implementation was progressively developed around the following sequence:

1. creation and configuration of the PostgreSQL database;
2. implementation of the relational data model using SQLAlchemy ORM;
3. implementation of the data management components;
4. implementation of the Layer 1 degradation calculation engine;
5. implementation of cumulative damage and tool-state management;
6. implementation of the Layer 2 prediction services;
7. integration of the computational services into the Flask backend;
8. development of the web interface;
9. integration between the database, backend, analytical engines, and frontend;
10. functional verification of the complete application.

This progressive implementation strategy made it possible to validate each software component before integrating it with the following component. The resulting application provides a unified environment for recording machining information, calculating tool degradation, updating the tool lifecycle state, storing historical information, and providing predictive information through the implemented prediction layer.

The implementation was carried out using Python and Flask for the backend, PostgreSQL for persistent data storage, SQLAlchemy for object-relational mapping, Flask-Migrate for database schema evolution, and HTML/Jinja with Bootstrap for the web interface.

Figure 4.1 summarizes the practical development progression followed during the implementation phase.

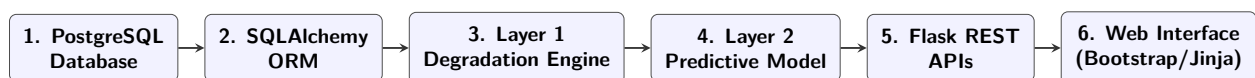


Figure 4.1: Progressive implementation workflow of the tool lifecycle monitoring platform.

4.2 Development Environment

4.2.1 Programming Environment

The application was developed in Python using a modular Flask application structure. Python was selected as the main programming language because it provides the required ecosystem for database interaction, numerical processing, web development, and the implementation of the analytical services.

The development environment was organized around a dedicated Python virtual environment and a project-specific interpreter. The application entry point is provided by the `run.py` file, which initializes the Flask application through the application factory defined in `app/__init__.py`.

The principal development components are summarized in Table 4.1.

Table 4.1: Main technologies used during implementation

Component	Technology / Configuration
Programming language	Python 3.11
Backend framework	Flask
ORM	Flask-SQLAlchemy / SQLAlchemy
Database management system	PostgreSQL
Database migration	Flask-Migrate / Alembic
Frontend	HTML5, Jinja2 and Bootstrap
Development environment	PyCharm
Version control	Git / GitHub
Database administration	pgAdmin

The PostgreSQL database used by the application is named `OC_FGA`. During development, the database server was configured to operate on the local machine and was accessed by the Flask application through SQLAlchemy.

4.2.2 Backend Technologies

The backend is organized around Flask and SQLAlchemy. Flask provides the application server and HTTP routing mechanisms, while SQLAlchemy provides the abstraction layer between Python objects and relational database tables.

The application is divided into several directories:

```
OCstageFGAT/  
  app/  
  models/  
  pipeline/  
  routes/  
  services/  
  templates/  
  __init__.py  
  migrations/  
  run.py  
  seed_demo.py  
  requirements.txt  
  tests/
```

The `models` directory contains the ORM representations of the database entities. The `pipeline` directory contains the computational services associated with degradation, state calculation and prediction. The `routes` directory contains the Flask blueprints used to expose application functionality. HTML templates are stored in the `templates` directory.

This separation was adopted to prevent database structures, analytical calculations, HTTP communication, and presentation logic from being mixed within the same source files.

4.2.3 Database Environment

PostgreSQL was used as the persistent storage layer. The database was administered during development using pgAdmin, which was used to create the database, inspect tables, verify relationships, and check the records generated by the application.

The connection between Flask and PostgreSQL was configured through the SQLAlchemy database URI. The database layer therefore remains independent from the analytical code: the calculation services retrieve their required information through ORM objects rather than directly manipulating SQL queries throughout the application.

4.3 Database Implementation

4.3.1 PostgreSQL Database Creation

The implementation started with the creation of the PostgreSQL database dedicated to the application. The database was created independently from the Flask application and subsequently connected to the backend through SQLAlchemy.

The database was named `OC_FGA` in accordance with the project configuration. Database access was verified from the Flask environment before implementing the analytical services. This verification was important because all subsequent operations depend on the ability to retrieve and persist machining information.

The database implementation was then progressively extended through Flask-Migrate. This allowed changes made to the SQLAlchemy models to be translated into database schema modifications.

4.3.2 Implementation of ORM Models

After configuring the database connection, the relational entities defined in Chapter 3 were implemented as SQLAlchemy models.

The principal entities implemented in the application are:

Database Entities: Utilisateur • UAP • Machine • TypeOutil • Outil • Affectation • Matiere • Lubrifiant • TypeOperation • SessionUsage • ParametresCoupe • IndicesCalcules • DegatsOutils • EtatOutil • HistoriqueEtat • Prediction. Each Python class represents a relational table and contains the corresponding attributes, primary keys, foreign keys, and SQLAlchemy relationships.

For example, the tool entity contains information required to uniquely identify and monitor a physical cutting tool.

```
1 class Outil(db.Model):
2     __tablename__ = "outils"
3
4     id = db.Column(db.Integer, primary_key=True)
5     code = db.Column(db.String(50), unique=True, nullable=False)
6     numero_serie = db.Column(db.String(100))
7     type_outil_id = db.Column(
8         db.Integer,
9         db.ForeignKey("types_outils.id")
10    )
11
```

```

12     date_mise_service = db.Column(db.DateTime)
13     nb_affutages = db.Column(db.Integer, default=0)
14     degats_cumules = db.Column(
15         db.Float,
16         default=0
17     )
18     etat_actuel = db.Column(
19         db.String(5),
20         default="S0 "
21     )

```

Listing 4.1: Simplified SQLAlchemy representation of a cutting tool

The ORM approach allows the analytical pipeline to access a tool using Python objects such as `outil.degats_cumules` or `outil.etat_actuel`, rather than repeatedly constructing SQL statements.

4.3.3 Implementation of Relationships

The relationships between entities were implemented using primary keys, foreign keys and SQLAlchemy relationship declarations.

For example, a machining session references the tool, machine, material, lubricant, operation type, and operator associated with the machining activity. Cutting parameters are then associated with the corresponding machining session.

The relationship between `SessionUsage` and `ParametresCoupe` was implemented as a one-to-one relationship because a machining session contains one corresponding set of cutting parameters.

Similarly, the calculated indices and damage records are associated with the machining session that generated them.

This organization provides the traceability required by the application:

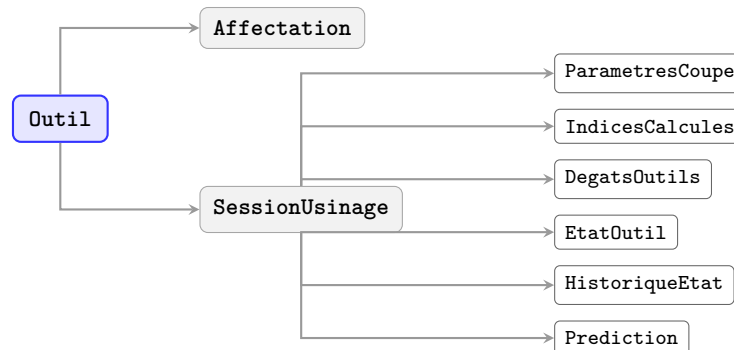


Figure 4.2: Entity relationship chain centered around tool lifecycle sessions.

4.3.4 Database Migration

Flask-Migrate was used to manage database schema evolution during development. After modifying the SQLAlchemy models, migration scripts were generated and applied to the PostgreSQL database.

The migration mechanism allowed the database structure to evolve consistently with the application models while maintaining a traceable history of schema modifications.

The main commands used during development were:

```

1 flask --app run db init
2 flask --app run db migrate -m "create application tables"
3 flask --app run db upgrade
4 flask --app run db current

```

Listing 4.2: Database migration commands used during implementation

The resulting tables were subsequently verified through pgAdmin.

4.3.5 Initialization and Demonstration Data

Once the database structure was operational, demonstration records were inserted to verify the complete data path before connecting the analytical services.

The demonstration dataset contains, among other elements, a CNC machine, a cutting tool, an aluminium material, a lubricant, an operation type, an assignment and a machining session.

The initialization script creates the required entities and links them together so that the analytical pipeline can be executed on a consistent session.

For example, the machining session contains cutting information such as cutting speed, spindle speed, feed per tooth, axial depth, radial depth, number of passes, cutting time and engagement.

This initial dataset was particularly useful during development because it provided a reproducible input for testing the successive stages of the application. =====

4.4 Implementation of the Data Management Layer

4.4.1 User Management

The user management component was implemented through the `Utilisateur` model and the corresponding user routes and services.

Users are stored in the PostgreSQL database with their identification information and application role. The role attribute allows the application to distinguish between administrative and operational access.

The implementation separates user persistence from authentication and business processing. This structure makes it possible to extend the access-control mechanism without modifying the analytical calculation services.

4.4.2 Tool Management

The tool management module provides the database representation of each physical cutting tool.

The `Outil` entity stores the tool code, serial number, tool type, commissioning information, number of regrind operations, cumulative damage, current state and activity status.

The cumulative damage and state attributes are particularly important because they are updated by the analytical services rather than manually entered by the operator.

Consequently, the tool record acts as the persistent reference for the lifecycle calculation.

4.4.3 Machine Management

Machine information is stored through the `Machine` model. The model contains identification information as well as technical characteristics used by the degradation calculation, including machine age, operating hours and rigidity.

The machine record is associated with a machining session through a foreign key. The analytical service can therefore retrieve the machine characteristics directly from the session object.

4.4.4 Material, Lubricant and Operation Management

The material, lubricant and operation-type entities were implemented as independent database tables.

This choice avoids storing repeated textual information inside every machining session. Instead, a session references the appropriate material, lubricant and operation type.

The resulting implementation also allows these characteristics to be reused across multiple sessions and provides a consistent source for the analytical indices.

4.4.5 Tool Assignment

The **Affectation** model records the relationship between a tool and a machine context.

This table provides a historical association between the physical tool and its machining environment without making the machine a sequential stage in the tool lifecycle.

The separation is important because the same tool may be assigned to different machines during its useful life.

4.4.6 Machining Session Management

The machining session represents the operational event from which the analytical calculation is launched.

A session contains references to:

Recorded Parameters: the cutting tool • the machine • the machined material • the lubricant • the operation type • the operator • the work order • the machining phase • the manufactured quantity • the rejected quantity • the validation status. The implementation therefore provides the analytical engine with a structured object containing the context required for the calculation.

4.4.7 Cutting Parameter Management

Cutting parameters are stored in the **ParametresCoupe** entity associated with the machining session.

The implemented fields include cutting speed, spindle speed, feed, feed per tooth, axial depth, radial depth, number of passes, machining time and engagement.

An example of the corresponding ORM structure is:

```
1 class ParametresCoupe(db.Model):
2     __tablename__ = "parametres_coupe"
3
4     id = db.Column(db.Integer, primary_key=True)
5
6     session_id = db.Column(
7         db.Integer,
8         db.ForeignKey("sessions_usinage.id"),
9         unique=True,
10        nullable=False
11    )
12
13    vitesse_coupe = db.Column(db.Float)
14    vitesse_rotation = db.Column(db.Float)
15    avance = db.Column(db.Float)
16    avance_par_dent = db.Column(db.Float)
17    profondeur_axiale = db.Column(db.Float)
18    profondeur_radiale = db.Column(db.Float)
19    nombre_passes = db.Column(db.Integer)
20    temps_coupe = db.Column(db.Float)
21    engagement = db.Column(db.Float)
```

Listing 4.3: Machining cutting parameters model

This implementation provides a clear separation between the machining event itself and the numerical parameters describing the cutting conditions.

4.5 Implementation of Layer 1

4.5.1 Implementation Strategy

Layer 1 was implemented as a sequence of Python functions grouped in the degradation calculation pipeline.

The implementation retrieves the machining session from the database, accesses its associated cutting parameters and contextual entities, computes the five constituent indices, aggregates them according to the configured weights, and stores the resulting values.

The main implementation file is `app/pipeline/igso.py`.

Rather than placing the entire calculation inside a single function, the implementation separates the processing into dedicated methods. This makes each component independently readable and facilitates later calibration.

4.5.2 Input Retrieval from the Database

The calculation is initiated from a machining session identifier.

The calculation service first verifies that the requested session exists and that the corresponding cutting parameters are available.

This verification prevents the analytical engine from operating on an incomplete machining record.

The general implementation sequence is:

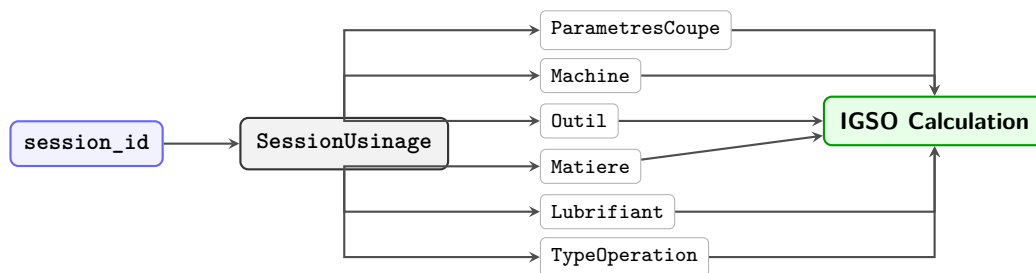


Figure 4.3: Flux d'agrégation des données de la session vers le calcul de l'IGSO.

The implementation therefore relies on the relational model developed previously rather than duplicating operational data inside the calculation service.

4.5.3 Implementation of the Normalization Function

A common normalization function was implemented to convert heterogeneous engineering quantities into bounded values suitable for aggregation.

The function receives a value together with its minimum and maximum reference limits and restricts the resulting normalized value to the interval $[0, 1]$.

The implementation follows the form:

```

1 def _normaliser(valeur, minimum, maximum):
2     if maximum == minimum:
3         return 0.0
4
5     valeur_normalisee = (
6         (valeur - minimum)
7         / (maximum - minimum)
8     )
9
10    return max(
11        0.0,
12        min(1.0, valeur_normalisee)
13    )

```

Listing 4.4: Normalization function used by the Layer 1 implementation

This function is reused by the different index-calculation methods. The use of a single normalization mechanism avoids inconsistent scaling between the individual indices.

4.5.4 Implementation of the Mechanical Index

The mechanical component is implemented through the private method `_calcul_IM`.

The method retrieves the cutting parameters from the machining session and normalizes the relevant machining quantities before combining them through their implementation weights.

The main variables processed by this method are cutting speed, feed per tooth, axial depth, radial depth and engagement.

A representative implementation excerpt is shown below.

```
1 def _calcul_IM(session):
2     params = session.parametres_coupe
3
4     vc = _normaliser(
5         params.vitesse_coupe,
6         VC_MIN,
7         VC_MAX
8     )
9
10    avance = _normaliser(
11        params.avance_par_dent,
12        FZ_MIN,
13        FZ_MAX
14    )
15
16    ap = _normaliser(
17        params.profondeur_axiale,
18        AP_MIN,
19        AP_MAX
20    )
21
22    ae = _normaliser(
23        params.profondeur_radiale,
24        AE_MIN,
25        AE_MAX
26    )
27
28    engagement = _normaliser(
29        params.engagement,
30        ENGAGEMENT_MIN,
31        ENGAGEMENT_MAX
32    )
33
34    return (
35        0.30 * vc
36        + 0.25 * avance
37        + 0.20 * ap
38        + 0.15 * ae
39        + 0.10 * engagement
40    )
```

Listing 4.5: Implementation of the mechanical index

The important implementation choice is that the method does not directly manipulate database records. It only receives the session and returns the calculated index. Persistence is handled later by the recording function.

4.5.5 Implementation of the Thermal Index

The thermal component is implemented in the `_calcul_IT` method.

The method uses machining time and spindle speed together with material and lubricant characteristics retrieved from the session.

The resulting value is treated as an implementation-level thermal exposure indicator derived from available process data rather than a direct temperature measurement.

This distinction is important because the current database does not rely on a physical temperature sensor for the calculation.

```
1 def _calcul_IT(session):
2     params = session.parametres_coupe
3
4     temps = _normaliser(
5         params.temps_coupe,
6         TEMPS_MIN,
7         TEMPS_MAX
8     )
9
10    rotation = _normaliser(
11        params.vitesse_rotation,
12        RPM_MIN,
13        RPM_MAX
14    )
15
16    conductivite = _normaliser(
17        session.matiere.conductivite_thermique,
18        COND_MIN,
19        COND_MAX
20    )
21
22    refroidissement = _normaliser(
23        session.lubrifiant.coefficient_refroidissement,
24        COOL_MIN,
25        COOL_MAX
26    )
27
28    return (
29        0.35 * temps
30        + 0.30 * rotation
31        + 0.20 * conductivite
32        + 0.15 * refroidissement
33    )
```

Listing 4.6: Implementation structure of the thermal index

The method therefore demonstrates how the database entities are directly consumed by the analytical layer.

4.5.6 Implementation of the Machine Index

The machine index is calculated using machine-specific information stored in the **Machine** entity.

The implementation retrieves machine age, operating hours and rigidity from the session and normalizes them before aggregation.

```
1 def _calcul_IMA(session):
2     machine = session.machine
3
4     age = _normaliser(
5         machine.age,
6         AGE_MIN,
7         AGE_MAX
8     )
```

```

9
10     hours = _normaliser(
11         machine.heures_fonctionnement,
12         HOURS_MIN,
13         HOURS_MAX
14     )
15
16     rigidity = _normaliser(
17         machine.rigidite,
18         RIGIDITY_MIN,
19         RIGIDITY_MAX
20     )
21
22     return (
23         0.40 * age
24         + 0.40 * hours
25         + 0.20 * (1.0 - rigidity)
26     )

```

Listing 4.7: Implementation of the machine-related index

This implementation illustrates the benefit of the relational design: machine information does not have to be entered again for every machining session.

4.5.7 Implementation of the Historical Index

The historical index was implemented using information stored at tool level.

The method accesses the cumulative damage, number of regrinding operations and current tool state.

The state is converted into a numerical score before being incorporated into the index.

```

1 def _etat_to_score(etat):
2     scores = {
3         "S0": 0.00,
4         "S1": 0.25,
5         "S2": 0.50,
6         "S3": 0.75,
7         "S4": 1.00
8     }
9
10    return scores.get(etat, 0.0)
11
12
13 def _calcul_IH(session):
14     outil = session.outil
15
16     degats = _normaliser(
17         outil.degats_cumules or 0,
18         0,
19         DEGATS_MAX
20     )
21
22     affutage = _normaliser(
23         outil.nb_affutages or 0,
24         0,
25         AFFUTAGES_MAX
26     )
27
28     etat = _etat_to_score(
29         outil.etat_actuel
30     )
31

```

```

32     return (
33         0.50 * degats
34         + 0.30 * affutage
35         + 0.20 * etat
36     )

```

Listing 4.8: Implementation of the historical index

The implementation ensures that the degradation calculation incorporates information accumulated throughout the tool lifecycle rather than considering each machining session independently.

4.5.8 Implementation of the Process Index

The process index is implemented using machining-specific information, particularly the number of passes, the produced quantity and the operation type.

```

1  def _calcul_IP(session):
2      params = session.parametres_coupe
3
4      passes = _normaliser(
5          params.nombre_passes,
6          1,
7          10
8      )
9
10     quantity = _normaliser(
11         session.quantite_fabriquee,
12         0,
13         1000
14     )
15
16     operation = (
17         session.type_operation.coefficient
18     )
19
20     return (
21         0.40 * passes
22         + 0.30 * quantity
23         + 0.30 * operation
24     )

```

Listing 4.9: Implementation of the process index

The implementation therefore connects the analytical calculation to the operational characteristics of the machining session.

4.5.9 Implementation of the Global Calculation

Once the five indices have been calculated, the global function combines their values using the calibrated implementation weights.

The implementation uses the following weighting configuration:

```

1  def calculer_igso(session):
2      i_m = _calcul_IM(session)
3      i_t = _calcul_IT(session)
4      i_ma = _calcul_IMA(session)
5      i_h = _calcul_IH(session)
6      i_p = _calcul_IP(session)
7
8      igso = (
9          0.30 * i_m

```

```

10         + 0.25 * i_t
11         + 0.15 * i_ma
12         + 0.20 * i_h
13         + 0.10 * i_p
14     )
15
16     return min(1.0, igso)

```

Listing 4.10: Global IGSO aggregation in the implementation

The use of explicit weights makes the implementation transparent and allows their calibration without restructuring the entire computational engine.

4.5.10 Persistence of Calculated Results

After the five indices and the global value have been calculated, the results are stored in the `IndicesCalcules` database table.

The persistence step records the individual values separately rather than storing only the final global indicator. This decision allows the application to preserve the diagnostic information associated with a calculation.

Consequently, a later analysis can determine whether a change in the global value originated primarily from mechanical, thermal, machine, historical or process-related information.

4.6 Implementation of Damage and State Management

4.6.1 Damage Calculation Service

The global degradation indicator produced by Layer 1 is subsequently processed by the damage calculation service implemented in `app/pipeline/degats.py`.

The purpose of this component is to transform the session-level degradation indicator into an incremental damage value and update the cumulative damage associated with the physical tool.

The implementation uses a calibration coefficient defined in the calculation service.

```

1 class CalculDegats:
2
3     COEFFICIENT_DEGATS = 10
4
5     @staticmethod
6     def calculer(session):
7
8         indice = CalculIGSO.calculer(session)
9         igso = indice.igso
10
11         outil = session.outil
12
13         anciens_degats = (
14             outil.degats_cumules or 0
15         )
16
17         degats_session = round(
18             igso * CalculDegats.COEFFICIENT_DEGATS,
19             4
20         )
21
22         nouveaux_degats = (
23             anciens_degats
24             + degats_session
25         )

```

```

26
27     outil.degats_cumules = nouveaux_degats
28
29     # Persistence of the calculation
30     degats = DegatsOutils(
31         session_id=session.id,
32         outil_id=outil.id,
33         igso=igso,
34         degats_session=degats_session,
35         degats_cumules=nouveaux_degats
36     )
37
38     db.session.add(degats)
39
40     CalculEtat.mettre_a_jour(outil)
41
42     db.session.commit()
43
44     return degats

```

Listing 4.11: Damage accumulation implementation

This service creates a persistent link between the machining session, the calculated degradation indicator, the incremental damage and the cumulative damage.

4.6.2 Updating the Tool Record

The cumulative damage is stored directly in the `Outil` record.

This allows the next machining session to retrieve the current lifecycle condition of the same physical tool without recalculating all previous sessions.

The implementation therefore distinguishes between:

- session-level calculated damage;
- cumulative tool damage;
- current tool state.

The three pieces of information are persisted separately so that the evolution of the tool remains traceable.

4.6.3 State Calculation Service

The state management service is implemented in `app/pipeline/etat.py`.

Once the cumulative damage has been updated, the service determines the corresponding state and health indicator according to the state configuration introduced in Chapter 3.

The implementation then updates the `EtatOutil` record and the current state stored in `Outil`.

```

1 class CalculEtat:
2
3     @staticmethod
4     def mettre_a_jour(outil):
5
6         degats = outil.degats_cumules or 0
7
8         if degats < 20:
9             nouvel_etat = "S0"
10            sante = 100
11        elif degats < 40:
12            nouvel_etat = "S1"
13            sante = 80

```

```

14         elif degats < 60:
15             nouvel_etat = "S2"
16             sante = 60
17         elif degats < 80:
18             nouvel_etat = "S3"
19             sante = 40
20         else:
21             nouvel_etat = "S4"
22             sante = 0
23
24         outil.etat_actuel = nouvel_etat
25
26         # Update or create EtatOutil
27         ...

```

Listing 4.12: Tool state update mechanism

The thresholds themselves were defined in Chapter 3 and are therefore not repeated here as a theoretical table. The purpose of this section is to show their software implementation and their integration into the tool record.

4.6.4 State History Persistence

A state transition is recorded when the newly calculated state differs from the previous state.

The `HistoriqueEtat` model stores the previous state, the new state, the reason associated with the transition and the transition date.

This mechanism provides lifecycle traceability at tool level.

For example, a transition from S1 to S2 is not only reflected in the current state of the tool; it is also preserved in the historical record.

4.7 Implementation of Layer 2

4.7.1 Markov Service Organization

The second analytical layer was implemented as a dedicated prediction service in `app/pipeline/markov.py`.

The service is separated from the Layer 1 calculation engine. This separation allows the prediction mechanism to operate on the current state produced by the degradation and state-management components.

The Markov implementation is therefore organized around three main operations:

1. construction or loading of the transition matrix;
2. calculation of future state probabilities;
3. storage of the resulting prediction information.

No recalculation of the degradation indices is performed inside the prediction service.

4.7.2 Transition Matrix Configuration

The transition matrix is represented in the software as a five-state matrix corresponding to the implemented state space.

The values used by the prediction service are configured as implementation parameters. Their calibration is based on expert knowledge and the expected transition behavior of the tool states.

The software structure allows the transition probabilities to be modified without changing the prediction algorithm itself.

For this reason, the exact matrix values should be reported together with their final validated calibration source in the final project version rather than being presented as universal probabilities.

4.7.3 Prediction Calculation

The prediction service receives the current state of the tool and uses the transition matrix to calculate the corresponding future probability distribution.

The implementation follows the matrix-power operation introduced conceptually in Chapter 3, but the theoretical derivation is not repeated here.

A representative implementation structure is:

```
1 import numpy as np
2
3 class MarkovPrediction:
4
5     @staticmethod
6     def predire(outil):
7
8         state_map = {
9             "S0": 0,
10            "S1": 1,
11            "S2": 2,
12            "S3": 3,
13            "S4": 4
14        }
15
16        current_index = state_map[
17            outil.etat_actuel
18        ]
19
20        pi_0 = np.zeros(5)
21        pi_0[current_index] = 1.0
22
23        P = MarkovPrediction.construire_matrice(
24            outil
25        )
26
27        # Future state distribution
28        pi_future = np.dot(
29            pi_0,
30            P
31        )
32
33        predicted_index = int(
34            np.argmax(pi_future)
35        )
36
37        state_list = [
38            "S0", "S1", "S2", "S3", "S4"
39        ]
40
41        return {
42            "etat_predit":
43                state_list[predicted_index],
44            "probabilite":
45                float(pi_future[predicted_index])
46        }
```

Listing 4.13: Structure of the Markov prediction service

The exact implementation can be extended to multiple prediction steps by applying successive matrix powers.

4.7.4 RUL Calculation

The prediction layer also contains the software structure required to express the remaining useful life in terms of future sessions.

The calculation is based on the transition behavior between the non-failure states and the absorbing failure state. The resulting number of expected sessions can subsequently be converted into an estimated time using a representative machining-session duration.

The RUL value is stored in the prediction record together with the predicted state and probability.

Because the reliability of the RUL depends directly on the calibration of the transition matrix and on representative historical observations, the value is interpreted as a model-based prediction rather than as a directly measured quantity.

4.7.5 Prediction Persistence

The `Prediction` model was implemented to preserve the prediction associated with each monitored tool.

The principal stored information includes:

- tool identifier;
- current state;
- predicted state;
- prediction probability;
- RUL;
- estimated remaining number of sessions;
- prediction date.

This database persistence allows the prediction service to remain connected to the rest of the lifecycle monitoring system.

4.8 Backend and API Integration

4.8.1 Flask Application Initialization

The Flask application is initialized through the application factory implemented in `app/__init__.py`.

This mechanism initializes the database extension, registers the application routes, and loads the models required by the application.

The application factory approach allows the application configuration and extension initialization to remain separated from the execution entry point.

The application is launched through:

```
1 python run.py
```

Listing 4.14: Launching the Flask application

The application is then accessible locally through the Flask development server.

4.8.2 Blueprint Organization

The backend routes are separated into Flask Blueprints.

The implemented route organization includes modules for authentication, calculations, main application pages, and user-related operations.

This organization prevents the main Flask application file from becoming a single large routing module.

The calculation routes are grouped in `calcul_routes.py` and are responsible for triggering the analytical pipeline.

4.8.3 Calculation Route

The calculation endpoint receives a machining session identifier and launches the corresponding calculation service.

The route acts as the connection between the HTTP layer and the analytical backend.

The implementation follows the structure:

```
1 @calcul_bp.route(  
2     "/session/<int:session_id>",  
3     methods=["POST"]  
4 )  
5 def calculer_session(session_id):  
6  
7     try:  
8         resultat = CalculGlobal.executer(  
9             session_id  
10        )  
11  
12        return jsonify(  
13            "success": True,  
14            "message":  
15                "Calcul termine avec succes.",  
16            "resultat": resultat  
17        ), 200  
18  
19    except Exception as e:  
20  
21        return jsonify(  
22            "success": False,  
23            "message": str(e)  
24        ), 500
```

Listing 4.15: Flask calculation endpoint

The route therefore has two principal responsibilities:

1. receive the session identifier from the client;
2. return the result of the calculation in JSON format.

4.8.4 Request Processing

The request processing sequence consists of retrieving the machining session, validating the required information, executing the analytical service, persisting the generated records and returning the result.

The HTTP layer does not contain the mathematical calculations themselves. This separation ensures that the same analytical services can later be called by other application components.

4.8.5 Response and Error Handling

The API returns a structured JSON response indicating whether the calculation was successfully completed.

Successful requests contain a success flag, a message and the generated result. Errors are intercepted and returned as a structured error response.

This mechanism simplifies debugging during development and provides a consistent communication format between the backend and frontend.

4.9 Frontend Implementation

4.9.1 Login Interface

The web interface was developed using HTML templates rendered by Flask and Jinja.

The login interface provides the entry point to the application. Its implementation is separated from the analytical services so that authentication and tool calculations remain independent components.

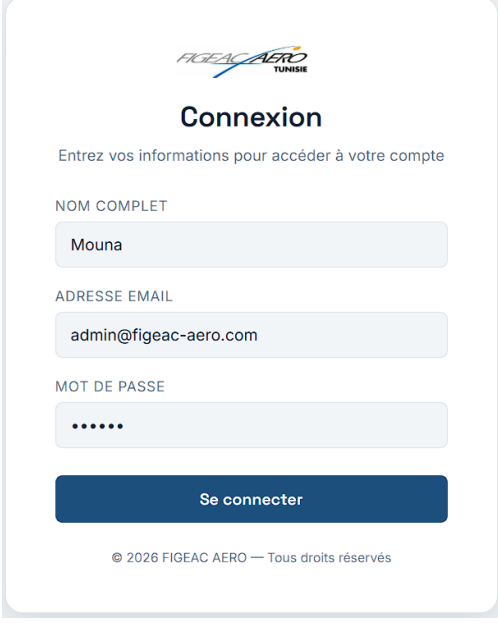
The image shows a web form for login. At the top is the logo for 'FIGEAC AERO TUNISIE'. Below the logo is the title 'Connexion'. Underneath the title is the instruction 'Entrez vos informations pour accéder à votre compte'. There are three input fields: 'NOM COMPLET' with the value 'Mouna', 'ADRESSE EMAIL' with the value 'admin@figeac-aero.com', and 'MOT DE PASSE' with masked characters '.....'. Below these fields is a blue button labeled 'Se connecter'. At the bottom of the form is the copyright notice '© 2026 FIGEAC AERO — Tous droits réservés'.

Figure 4.4: Implemented authentication interface.

4.9.2 Dashboard Implementation

The main dashboard was implemented using Bootstrap-based HTML components.

The dashboard provides a visual entry point to the monitoring platform and displays the principal information associated with monitored tools and machining activity.

The interface is generated from the Flask template `dashboard.html`.

The implementation uses reusable Bootstrap components for cards, navigation elements and information panels.

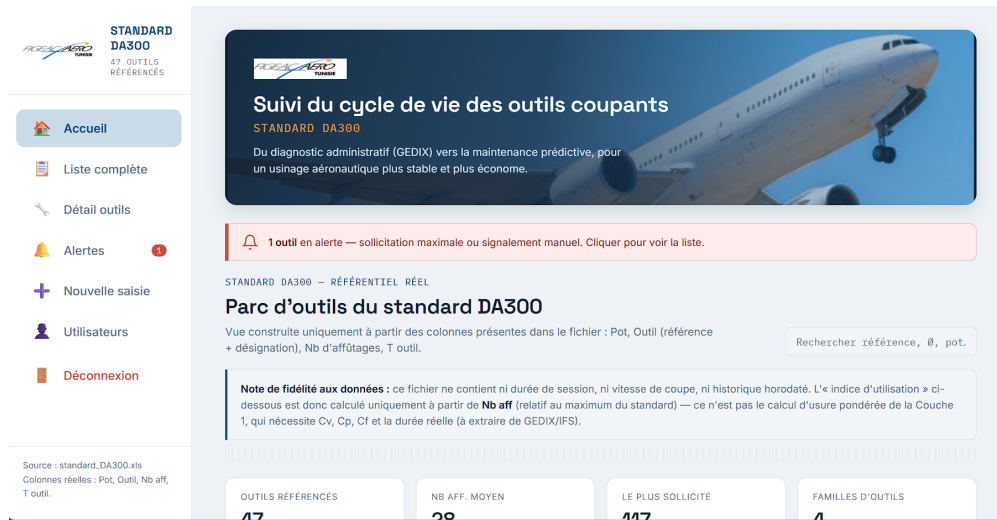


Figure 4.5: Main dashboard of the implemented monitoring platform.

4.9.3 Dynamic Data Display

The frontend is connected to backend data through Flask/Jinja rendering and application routes.

The purpose of this implementation is to avoid duplicating the database logic in the HTML pages. Instead, the backend retrieves the required information and provides it to the corresponding template.

For example, tool information can be passed from the Flask route to the dashboard template and rendered using Jinja syntax.

```

1 <div class="card">
2   <div class="card-body">
3     <h5 class="card-title">
4       Tool Monitoring
5     </h5>
6
7     <p>
8       Current state:
9       {{ outil.etat_actuel }}
10    </p>
11
12    <p>
13      Cumulative damage:
14      {{ outil.degats_cumules }}
15    </p>
16  </div>
17 </div>

```

Listing 4.16: Example of Jinja-based dynamic data rendering

This mechanism provides a direct link between the persistent tool record and its graphical representation.

4.9.4 Tool Monitoring Interface

The tool monitoring interface is intended to provide a direct view of the lifecycle status of a physical tool.

The displayed information can include:

Dashboard Features: tool identification • current state • health indicator • cumulative damage • number of regrounding operations • most recent machining information • prediction information.

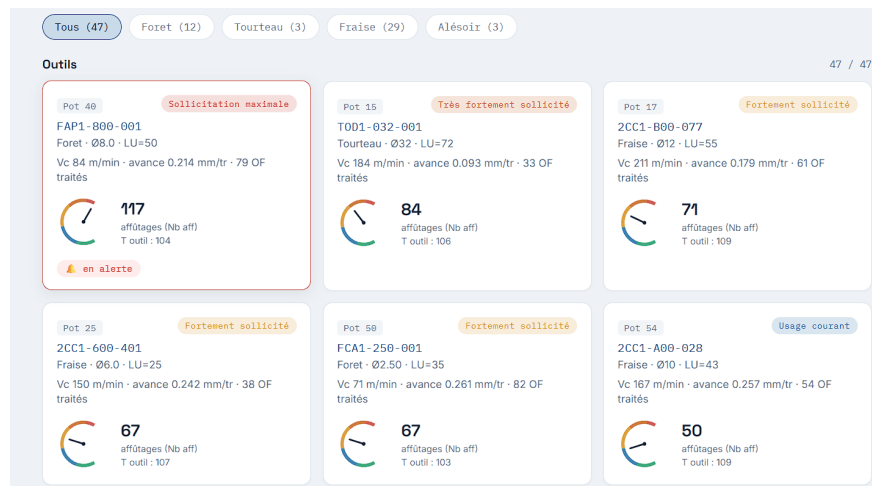


Figure 4.6: Tool lifecycle monitoring interface.

4.9.5 Prediction Display

The prediction information generated by Layer 2 is intended to be presented alongside the current tool state.

The interface can therefore expose the predicted state, associated probability and RUL information stored in the prediction record.

This organization allows the operator or decision-support user to distinguish the current measured/calculated condition from the model-based future condition.

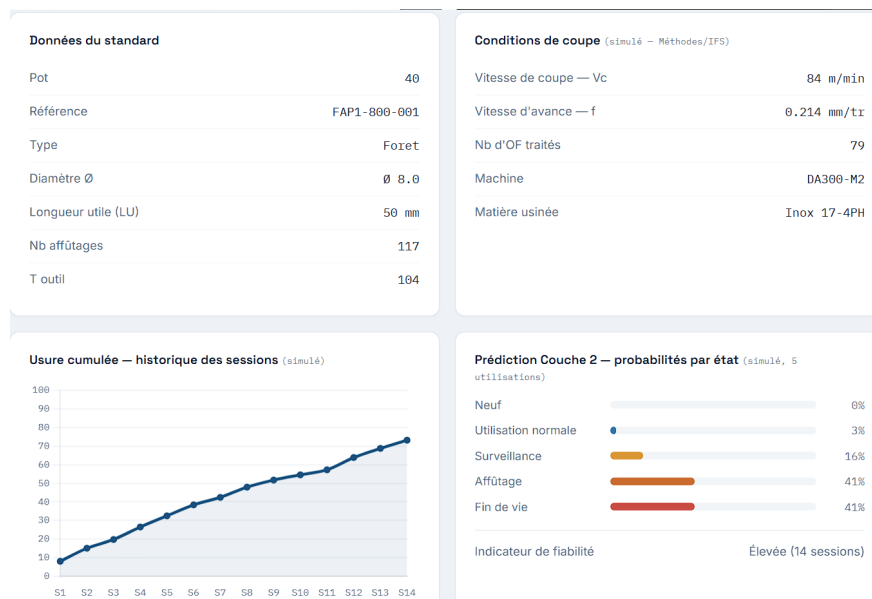


Figure 4.7: Presentation of prediction information in the web interface.

4.9.6 Frontend–Backend Integration

The frontend does not independently reproduce the analytical calculations.

Instead, the interface relies on the backend to retrieve the tool records and calculated information. The frontend is therefore primarily responsible for presentation and user interaction, while the backend remains responsible for data access and analytical processing.

This separation simplifies future modifications because a change to the calculation engine does not require rewriting the visual components.

4.10 System Integration

4.10.1 Database-to-Backend Integration

The first integration level connects PostgreSQL to the Flask application through SQLAlchemy.

At application startup, the database extension is initialized and the ORM models become available to the backend services.

A typical data retrieval operation therefore follows the sequence:

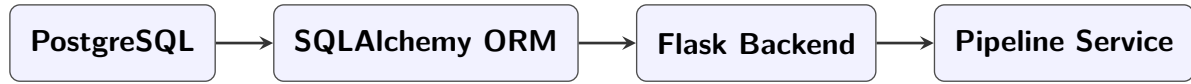


Figure 4.8: Architectural data flow from database to execution pipeline.

The ORM provides the abstraction required by the calculation services to access structured manufacturing information.

4.10.2 Backend-to-Analytical Pipeline Integration

The second integration level connects the Flask backend to the analytical services.

A calculation request received by the backend identifies a machining session. The corresponding service retrieves the session and launches the Layer 1 calculation.

The resulting information is then transferred to the damage and state services. Layer 2 prediction uses the updated tool state.

The integration sequence can therefore be summarized as:

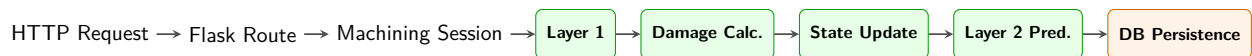


Figure 4.9: End-to-end execution flow of a machining session request.

4.10.3 Backend-to-Frontend Integration

Once the backend has processed the requested operation, the resulting information can be displayed by the web interface.

The integration between backend and frontend was implemented through Flask routes and Jinja templates.

This allows the same database information to be presented in a user-oriented form without exposing internal database operations directly to the user.

4.10.4 Complete Implementation Sequence

The final integrated application follows the technical sequence shown in Figure 4.10.

This sequence represents the software implementation of the architecture designed in Chapter 3. Each component communicates through defined data structures rather than directly manipulating the internal implementation of another component.

4.11 Testing and Verification

4.11.1 Testing Objectives

Testing was conducted progressively throughout development rather than only after completing the entire application.

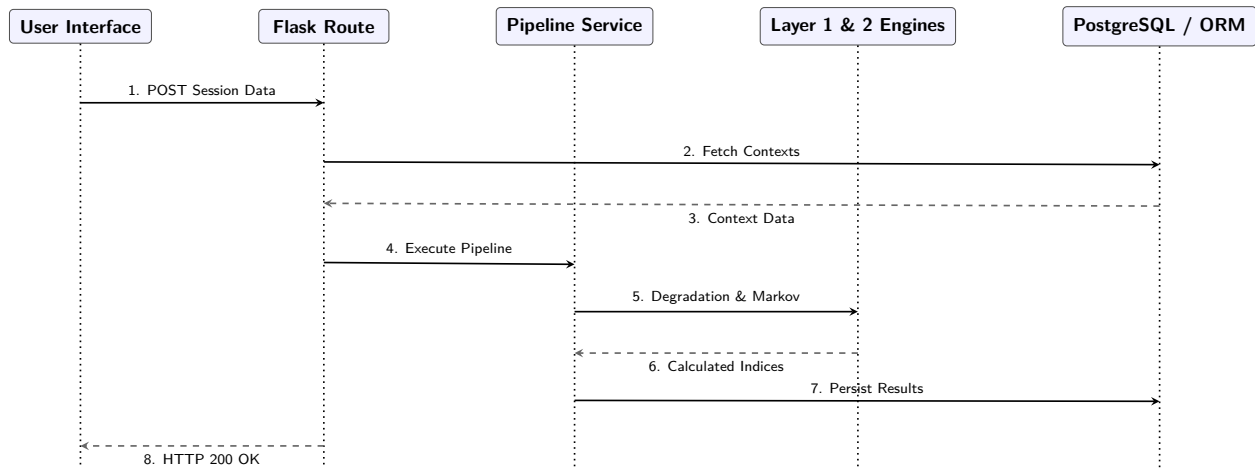


Figure 4.10: Sequence diagram of complete execution flow.

The verification objectives were:

- verify database connectivity and persistence;
- verify the correctness of ORM relationships;
- verify Layer 1 calculations;
- verify cumulative damage updates;
- verify state transitions;
- verify prediction processing;
- verify API request and response behavior;
- verify the rendering of the web interface;
- verify the complete data path from a machining session to the resulting monitoring information.

The testing approach therefore combines component-level verification and end-to-end verification.

4.11.2 Database Verification

The first verification stage was performed directly in PostgreSQL through pgAdmin.

The existence of the implemented tables was checked together with the presence of primary keys, foreign keys and generated records.

The demonstration data were then inspected to ensure that the relationships between tools, machines, sessions and cutting parameters were correctly established.

4.11.3 Layer 1 Verification

Layer 1 was verified using machining sessions containing known input parameters.

The verification process consisted of checking:

1. successful retrieval of the session;
2. successful retrieval of cutting parameters;
3. generation of the five constituent indices;
4. calculation of the global indicator;

5. storage of the calculated values in the database.

The verification was performed at software level by inspecting the returned calculation object and the corresponding database record.

4.11.4 Damage and State Verification

The damage service was verified by executing a calculation on a tool with an initialized cumulative damage value.

The resulting session damage was added to the existing cumulative value. The corresponding state was then recalculated and stored.

The verification therefore focused on consistency between:



Figure 4.11: Horizontal Analytical Sequence from IGSO to State History.

4.11.5 Layer 2 Verification

Layer 2 was verified by providing the prediction service with a valid current tool state and checking that the service produced the corresponding prediction information.

The verification includes:

- correct mapping between state labels and matrix indices;
- correct construction or retrieval of the transition matrix;
- calculation of future state probabilities;
- identification of the predicted state;
- storage of the prediction record.

The exact predictive quality of the model depends on the calibration and subsequent validation of the transition probabilities. The implementation test therefore verifies the computational chain and data persistence independently from long-term industrial statistical validation.

4.11.6 API Verification

The calculation route was tested by sending an HTTP request containing a valid machining-session identifier.

The expected response contains a success status and the generated calculation information.

Invalid or unsuccessful operations are handled through the exception mechanism implemented in the route.

The API verification confirms that the web layer can correctly trigger the backend calculation services.

4.11.7 Interface Verification

The frontend was checked by launching the Flask application and navigating through the implemented pages.

The verification focused on:

- successful loading of the login interface;
- successful loading of the dashboard;
- correct rendering of tool information;

- display of state information;
- display of prediction information where available;
- consistency between the displayed information and database records.

4.11.8 End-to-End Verification

The final verification stage consisted of following one machining session through the complete software chain.

Starting from a session stored in PostgreSQL, the backend retrieves the corresponding entities and cutting parameters. Layer 1 then calculates the degradation indices and global indicator. The damage service updates the cumulative tool damage, after which the state service updates the current state and records any transition. The prediction layer subsequently uses the current state to generate prediction information. Finally, the resulting information can be retrieved by the web interface.

The complete sequence is therefore:

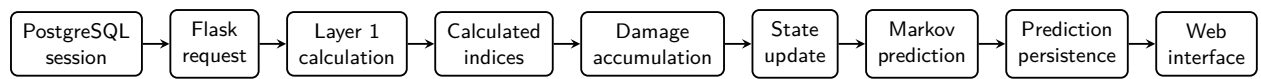


Figure 4.12: End-to-end horizontal execution workflow from PostgreSQL session to Web interface.

This end-to-end verification confirms the technical integration of the principal components implemented during the project.

4.12 Conclusion

This chapter presented the concrete implementation of the tool lifecycle monitoring platform developed for FGAT. The implementation was conducted progressively, beginning with the PostgreSQL database and ORM layer and continuing with the data management components, degradation calculation engine, damage and state management, prediction services, backend API, and web interface.

The implementation of Layer 1 transformed the five-index degradation model defined in Chapter 3 into executable Python services capable of retrieving machining data, calculating normalized indices, aggregating them into the global degradation indicator, and storing the resulting information.

The damage and state-management components then established the connection between session-level calculations and the persistent lifecycle condition of each physical tool. Layer 2 was integrated as a dedicated prediction service operating on the calculated tool state and storing its prediction information in the database.

At application level, Flask routes provided the interface between user requests and the analytical services, while the HTML/Jinja and Bootstrap frontend provided a visual interface for accessing the resulting information.

Finally, progressive component verification and end-to-end testing were used to verify the consistency of the database, analytical pipeline, API and user interface.

The resulting implementation establishes the software foundation required for industrial use of the monitoring approach. The next chapter can therefore focus on the interpretation of the implemented system, its industrial contribution, validation boundaries, and possible extensions of the developed platform.

Chapter 5

Industrial Discussion, Limitations and Perspectives

5.1 Introduction

The previous chapters described the industrial context of the project, analyzed the existing situation and identified the main requirements, then presented the proposed solution and its software implementation. The developed platform combines a degradation assessment mechanism based on the Global Severity/Degradation Index (GSO/IGSO) with a stochastic prediction layer based on Markov chains.

This chapter focuses on the industrial interpretation of the developed solution rather than on its technical implementation. The objective is to discuss how the platform can support cutting-tool lifecycle monitoring, the conditions required for its effective industrial use, and the elements that must be validated before large-scale deployment. The main limitations of the current implementation are also identified. Finally, several perspectives are presented for extending the platform toward a more complete intelligent tool lifecycle management system.

5.2 Industrial Contribution of the Developed Platform

5.2.1 Data Centralization and Traceability

One of the main contributions of the developed platform is the centralization of information related to cutting-tool utilization. The database structure connects the tool with its assignments, machining sessions, machine, material, lubricant, operation type and cutting parameters.

This organization provides a structured representation of the tool's machining history. Instead of considering each machining operation independently, the platform maintains information that can be associated with the evolution of the same physical tool throughout its utilization.

Such traceability is particularly important for industrial environments where a cutting tool can undergo several machining operations, assignments and regrinding operations. The availability of a historical record makes it possible to associate the calculated degradation with the operating conditions under which it was obtained.

The developed system therefore establishes a foundation for transforming operational machining data into information that can be exploited for monitoring and decision support.

5.2.2 Tool Lifecycle Monitoring

The platform provides a continuous representation of the tool condition through a sequence of health states ranging from the initial condition to the failure condition.

The calculated degradation is accumulated over successive machining sessions. The resulting value is then associated with the corresponding tool state and stored in the database. State transitions are also recorded, allowing the evolution of the tool to be reconstructed from its historical records.

From an industrial perspective, this approach provides a simple and interpretable representation of tool condition. Instead of relying exclusively on a final failure event, the operator or maintenance personnel can observe intermediate degradation states.

This principle can support earlier identification of tools requiring attention and can contribute to better organization of tool monitoring activities.

5.2.3 Degradation Assessment and State Monitoring

The first analytical layer combines five complementary aspects of machining conditions: mechanical, thermal, machine-related, historical and process-related information.

The resulting global index provides a normalized indicator of the severity associated with a machining session. Its main industrial interest lies in the aggregation of heterogeneous information into a single interpretable indicator.

The use of several indices also makes it possible to distinguish different sources of degradation. For example, an increase in the mechanical contribution may be associated with demanding cutting conditions, whereas the machine and historical contributions provide information related to the operating environment and previous tool usage.

The degradation indicator is subsequently transformed into a discrete tool state. This provides an interface between continuous degradation assessment and operational decision-making.

Consequently, the system does not only store raw machining data. It transforms these data into indicators that can be interpreted at the tool-management level.

5.2.4 Predictive Decision Support

The second analytical layer introduces a stochastic prediction mechanism based on the evolution of tool states. The Markov-chain approach allows the current state to be used as the basis for estimating possible future states.

The industrial interest of this layer is not to replace operator or maintenance expertise, but to provide additional information about the probable evolution of the tool. In particular, the prediction layer can support anticipation of deterioration and preparation of maintenance or tool-replacement actions.

The combination of current state information, historical degradation and predicted evolution creates a decision-support mechanism rather than a simple monitoring dashboard.

The final objective is therefore to move from:

$$Data\ collection \rightarrow Condition\ monitoring \rightarrow Prediction \rightarrow Decision\ support$$

while keeping the human operator and industrial maintenance personnel involved in the final decision.

5.3 Industrial Applicability and Deployment Conditions

5.3.1 Data Availability and Quality

The effectiveness of the developed approach depends directly on the quality and availability of machining data. The analytical layers require information describing the machining session, cutting parameters, machine characteristics, material, lubricant, tool history and previous degradation.

In an industrial environment, these data may originate from different information systems and may not always have the same format, frequency or level of completeness. Missing parameters, incorrect values or inconsistent records can directly affect the calculated indices.

Data quality should therefore be considered as an essential condition for industrial deployment. Before using the platform for operational decision-making, the following aspects should be controlled:

- completeness of machining-session records;
- consistency of cutting parameters;

- identification and traceability of physical tools;
- reliability of tool regrinding and replacement information;
- consistency between production and tool-management data;
- temporal coherence between successive machining sessions.

The platform consequently requires not only an analytical model but also a reliable data acquisition and preparation process.

5.3.2 Integration with Existing Information Systems

The developed application was designed around a relational database and a web-based backend. For industrial deployment, it can be connected progressively to the information systems already used by the production environment.

In particular, information concerning work orders, machining operations, tools, machines and production quantities can be integrated with the platform when appropriate interfaces and data-access mechanisms are available.

The objective of such integration would not be to replace existing industrial information systems. Instead, the developed platform can act as an analytical layer that exploits available production information and transforms it into tool-condition indicators and predictions.

A progressive integration strategy would therefore be preferable to a complete replacement of existing systems. This approach would also facilitate validation of the platform using real production data before extending its use to additional production units.

5.3.3 Deployment Across Production Units

The current platform was designed with a relational and modular structure. Consequently, the same logical organization can be extended to multiple machines, tools, materials and machining operations.

However, extending the system from a prototype environment to a larger industrial scope requires additional validation. Differences between machines, machining processes, tool families and production conditions may modify the behavior of the degradation indicators.

The parameters of the analytical models should therefore be evaluated according to the characteristics of each production context. A configuration validated for one machining environment should not automatically be considered universally applicable.

5.4 Validation Strategy and Industrial Performance Assessment

5.4.1 Validation Indicators

The technical validation performed during development verifies the consistency of the data flow, calculations, state management, prediction mechanism and web integration. For industrial validation, additional indicators are required to evaluate whether the predictions and degradation assessments correspond to actual tool behavior.

The following indicators can be considered:

Table 5.1: Proposed industrial validation indicators

Indicator	Purpose
State prediction accuracy	Compare predicted tool states with observed states.
Prediction error	Quantify the difference between predicted and observed degradation evolution.
Failure detection rate	Evaluate the ability to identify tools approaching failure.
False alarm rate	Measure unnecessary maintenance or replacement indications.
RUL prediction error	Compare estimated remaining useful life with the observed remaining life.
Tool utilization	Analyze the evolution of tool utilization before replacement or regrinding.
Unplanned tool replacement	Monitor unexpected tool changes after deployment.
Data completeness	Evaluate the availability and quality of the information required by the model.

These indicators should be evaluated on a sufficiently large historical or experimental dataset before conclusions are drawn concerning industrial performance.

5.4.2 Prediction Validation

The validation of the Markov prediction layer requires comparing predicted states with subsequent observed states.

For a sequence of machining sessions, the current state at session k can be used to generate a prediction for subsequent sessions. The predicted state can then be compared with the state calculated from the actual machining data.

For example, if the system predicts a transition toward a higher degradation state, the subsequent machining sessions can be analyzed to determine whether the observed evolution follows the predicted behavior.

This validation should be performed over multiple tools and different operating conditions. A single tool trajectory is insufficient to characterize the general behavior of the prediction model.

The same principle applies to the remaining useful life estimation. The predicted number of remaining sessions should be compared with the actual number of sessions observed before the corresponding end-of-life condition.

5.4.3 Tool-Life and Maintenance Indicators

From an industrial perspective, the final evaluation should extend beyond prediction accuracy. The objective of the platform is ultimately to support tool-management decisions.

Several indicators can therefore be monitored during an industrial evaluation period, including:

- average number of machining sessions per tool;
- number of unexpected tool failures;
- frequency of premature tool replacement;
- number of regrinding operations;
- tool consumption over a defined production period;

- maintenance interventions related to cutting tools;
- correspondence between predicted and observed tool condition.

These indicators should be compared over sufficiently representative production periods. The purpose is to determine whether the information generated by the platform can be translated into measurable operational benefits.

No fixed improvement percentage can be attributed to the developed prototype without such an industrial evaluation.

5.4.4 Long-Term Industrial Evaluation

A reliable evaluation should be conducted over a sufficiently long period to capture different machining conditions, tool batches and production situations.

A possible validation procedure consists of four stages:

1. collect historical machining and tool-lifecycle data;
2. calibrate the analytical parameters using representative observations;
3. operate the platform in parallel with the existing monitoring practices;
4. compare predictions and tool-condition information with actual observations.

This progressive approach makes it possible to evaluate the analytical models without immediately making them responsible for production decisions.

The results of this evaluation can subsequently be used to refine the degradation weights, normalization ranges and transition probabilities.

5.5 Limitations of the Current Implementation

5.5.1 Historical Data Availability

The quality of the historical index and the prediction mechanism depends on the availability of sufficiently representative tool histories.

A limited number of complete tool trajectories reduces the ability to characterize all possible degradation patterns. In particular, rare events such as sudden tool failure may be insufficiently represented.

The collection of additional historical records is therefore an important condition for strengthening the statistical reliability of the system.

5.5.2 Parameter Calibration

The degradation model uses normalization ranges and weighting coefficients to combine the five indices into a global indicator. These parameters constitute engineering calibration choices and are not universal physical constants.

Their relevance depends on the machining environment, tool family, material and machine characteristics.

For industrial deployment, these parameters should consequently be calibrated using representative production data and validated against observed tool behavior. Calibration should be performed periodically when significant changes occur in the production environment.

5.5.3 Markov Transition Calibration

The predictive performance of the second layer depends directly on the transition probabilities between tool states.

The transition matrix must therefore reflect the actual evolution of tools observed in the industrial environment. Expert knowledge can provide an initial calibration, but production observations should progressively be used to refine these probabilities.

The transition matrix should consequently be considered as a model parameter that can evolve with the accumulation of reliable historical data.

5.5.4 Physical Wear Validation

The current platform estimates tool degradation from operational and historical information. This approach provides an indirect assessment of tool condition.

However, a complete industrial validation requires comparison with physical observations of tool wear. Measurements obtained through inspection, dimensional control, microscopic observation or other appropriate industrial methods can provide reference observations for evaluating the calculated states.

Such comparison is important because a mathematical degradation indicator does not constitute a direct measurement of physical tool wear.

5.5.5 Generalization to Other Machining Contexts

The developed platform was designed to be extensible, but the analytical parameters cannot automatically be transferred without validation to every machining configuration.

Changes in tool geometry, cutting material, workpiece material, machining strategy, machine characteristics or lubrication conditions may affect degradation behavior.

The extension to additional machines and tool families should therefore follow a progressive validation process. This makes it possible to identify which parameters can remain common and which require adaptation to a specific production context.

5.6 Future Development Perspectives

5.6.1 Integration with Industrial Data Sources

A first development perspective is to strengthen the connection between the analytical platform and the industrial data sources used during production.

Automated acquisition of machining parameters, tool assignments, work orders, production quantities and tool-history information would reduce manual data entry and improve traceability.

Such integration would also provide a larger and more continuous data flow for the analytical models, thereby creating better conditions for future calibration and validation.

5.6.2 Circular Tool Lifecycle Management

The developed platform currently provides two complementary capabilities: the assessment of tool degradation through the GSO/IGSO-based Layer 1 and the prediction of future tool states through the Markov-based Layer 2. These two layers provide an important basis for extending the system from tool condition monitoring toward circular tool lifecycle management.

The main idea of this perspective is to avoid considering tool failure as the only possible endpoint of the tool lifecycle. In an industrial environment, a tool that reaches a degraded state is not necessarily unusable. Depending on its geometry, accumulated wear, number of previous regrinding operations and expected future utilization, it may still be suitable for regrinding and subsequent reuse.

The proposed circular extension would therefore introduce a decision layer above the existing degradation and prediction mechanisms. The purpose would be to transform the information generated by Layers 1 and 2 into lifecycle-oriented decisions such as:

- continue using the tool;
- monitor the tool more closely;
- schedule a regrinding operation;
- return the tool to production after regrinding;
- restrict the tool to less demanding operations;
- remove the tool from service when further reuse is no longer appropriate.

The important point is that the circular-economy layer would not replace the existing analytical layers. It would use their outputs as decision inputs.

Operating Principle

The proposed circular lifecycle mechanism can be organized into four successive stages.

First, Layer 1 evaluates the degradation associated with machining activity. The resulting GSO/IGSO value and cumulative damage provide an estimation of the current degradation level of the tool.

Second, Layer 2 uses the current tool state and the transition model to estimate its future evolution. This provides information about the probability of entering more critical states and about the estimated remaining useful life.

Third, the circular decision layer combines the current condition, predicted evolution and lifecycle information of the physical tool. Additional information such as the number of previous regrinding operations, tool family, tool geometry and production requirements can be considered.

Finally, the system produces a lifecycle recommendation. The recommendation is not based only on the current state. It considers whether the tool still has recoverable value.

Figure 5.1 illustrates this principle.

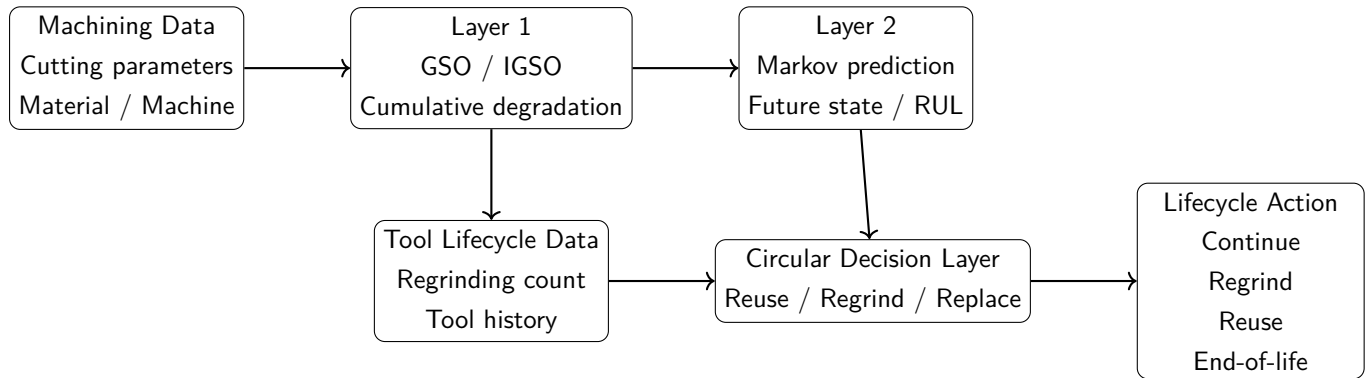


Figure 5.1: Future extension toward circular cutting-tool lifecycle management.

The figure emphasizes the role of the already validated analytical core. Layer 1 provides information about the present degradation condition, while Layer 2 provides information about the expected evolution. The circular layer then uses these outputs to determine how the remaining value of the tool can be exploited.

Industrial Case: Regrinding and Reuse of a Carbide Milling Tool

A representative industrial situation can be considered for a carbide milling cutter used for machining aluminium components.

Assume that a tool has already performed several machining operations. During a new machining session, the platform receives the cutting conditions and production information. Layer 1 calculates the five degradation indices and obtains the corresponding global degradation indicator.

Suppose that the tool is no longer considered to be in its initial state, but it has not yet reached the failure condition. Layer 2 then estimates the probability of transitioning toward more degraded states during subsequent machining sessions.

At this stage, a conventional replacement strategy could simply classify the tool as a candidate for replacement. The circular extension introduces an additional question:

Does the tool still have sufficient usable potential to justify regrinding?

The decision can therefore combine:

- current cumulative degradation;
- predicted future degradation;
- estimated remaining useful life;
- number of previous regrinding operations;
- tool family and geometry;
- requirements of the next machining operation.

For example, a tool presenting moderate degradation and a relatively low number of previous regrinding operations could be directed toward a regrinding process rather than immediate disposal. After regrinding, the tool would be reintroduced into the tool-management database as a new lifecycle stage while preserving its historical traceability.

The sequence can therefore be represented as:

$$\text{Machining} \rightarrow \text{Degradation Assessment} \rightarrow \text{State Prediction} \rightarrow \text{Regrinding Decision} \rightarrow \text{Reuse} \quad (5.1)$$

The important contribution is that the degradation and prediction layers remain the analytical basis of the decision. The circular layer does not independently estimate tool condition; it exploits the information already generated by the developed system.

Added Industrial Value

The potential value of this extension is not limited to reducing the number of discarded tools. It introduces a lifecycle-oriented interpretation of the information generated by the platform.

A circular management strategy could contribute to:

- increasing the exploitation of the usable life of cutting tools;
- supporting systematic regrinding decisions;
- reducing premature replacement;
- improving traceability of reconditioned tools;
- distinguishing tools that remain recoverable from tools approaching definitive end-of-life;

- providing additional information for tooling-cost analysis;
- reducing material waste associated with unnecessary tool disposal.

The extension would therefore transform the output of the current monitoring system from a simple condition indicator into a potential lifecycle-management input.

However, this functionality remains a future extension of the current project. The present implementation establishes the degradation and prediction foundation but does not yet include an automated circular-economy decision policy or experimentally validated regrinding criteria.

5.6.3 Adaptive Calibration through Reinforcement Learning

The second major perspective concerns the adaptive evolution of the developed analytical models. The current implementation relies on calibrated parameters for the GSO/IGSO degradation model and on transition probabilities for the Markov prediction layer. These parameters provide a deterministic and interpretable analytical foundation, but their optimal values may depend on the machining environment and may evolve as new production data become available.

A future reinforcement-learning layer could therefore be introduced to adapt selected model parameters according to feedback obtained from real machining operations.

The objective would not be to replace the GSO/IGSO model or the Markov chain. Instead, reinforcement learning could act as an adaptive layer that progressively adjusts the parameters of the existing system.

This distinction is important:

Current system: Data → Layer 1 → Layer 2 → Prediction

Future adaptive system: Data → Layer 1 → Layer 2 → RL adaptation → Updated parameters

Operating Principle

The future adaptive architecture can be organized as a feedback loop.

The current platform first observes the machining conditions and calculates the degradation indicators through Layer 1. The resulting tool state is then used by Layer 2 to predict its future evolution.

After the actual machining outcome becomes available, the system can compare the prediction with the observed result. This difference provides feedback about the quality of the current model.

A reinforcement-learning agent could then use this feedback to select an adaptation action. Depending on the implementation, the action could consist of modifying one or several model parameters, such as:

- selected GSO weighting coefficients;
- normalization parameters;
- transition probabilities of the Markov model;
- decision thresholds;
- parameters associated with specific tool or machining families.

The updated parameters would subsequently be used for future predictions.

The general feedback mechanism can be expressed as:

Observation → Prediction → Machining Outcome → Feedback → Parameter Adaptation → New Prediction
(5.2)

Figure 5.2 illustrates the proposed extension.

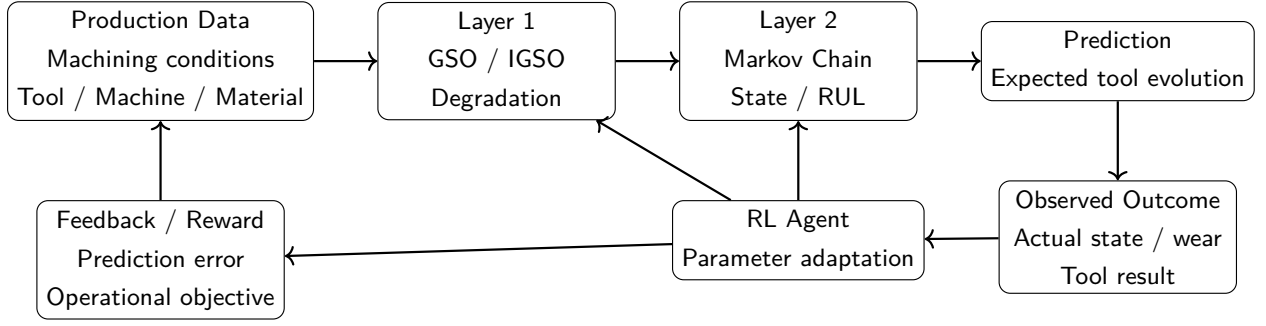


Figure 5.2: Future adaptive architecture based on reinforcement learning.

The architecture shows that reinforcement learning is positioned as an adaptive mechanism around the existing analytical core. Layer 1 remains responsible for degradation assessment and Layer 2 remains responsible for stochastic state prediction. The RL component progressively adjusts the parameters according to observed production feedback.

Industrial Case: Adaptation to Changing Machining Conditions

A representative case can be considered for a milling tool used repeatedly under different machining conditions.

During the initial deployment of the platform, the GSO weighting coefficients are calibrated using available engineering knowledge and representative observations. The Markov transition matrix is similarly configured to represent the expected evolution between the five tool states.

However, after several weeks of production, the system may observe that the actual tool degradation systematically differs from the predictions.

For example, the model may repeatedly predict a relatively slow transition between two health states, while actual production observations indicate that the transition occurs more rapidly under a particular combination of tool, material and machining conditions.

The feedback generated by these observations can be used by a future RL agent.

The agent observes the current model state, the prediction error and the operating context. It then selects an adaptation action, such as modifying selected transition probabilities or adjusting the relative contribution of degradation indices. For instance, if the observed degradation is consistently higher than the predicted degradation for a particular machining context, the adaptive mechanism could progressively modify the relevant parameters so that subsequent predictions better reflect the observed behavior.

The essential principle is therefore not to replace expert-defined knowledge immediately, but to progressively refine the model through production feedback.

Interaction with the Validated Layers

The value of this perspective comes directly from the structure already established by Layers 1 and 2.

Layer 1 transforms heterogeneous machining information into interpretable degradation indicators. This provides a structured state representation that can be exploited by a learning agent.

Layer 2 transforms the current tool state into a probabilistic representation of future evolution. This provides a prediction that can subsequently be compared with the actual observed evolution.

The reinforcement-learning layer would therefore operate on top of these two components:

$$\boxed{\text{Layer 1} \rightarrow \text{Current Condition} \rightarrow \text{Layer 2} \rightarrow \text{Prediction} \rightarrow \text{Observed Feedback} \rightarrow \text{RL Adaptation}} \quad (5.3)$$

This architecture preserves the interpretability of the existing analytical layers while introducing the possibility of adaptive calibration.

In particular, the future RL layer could progressively answer a question that the static model cannot fully address:

How should the model parameters evolve when production conditions and observed tool behavior change?

Potential Added Value

The main expected contribution of reinforcement learning would be adaptability rather than simply increasing model complexity.

A future adaptive mechanism could potentially:

- reduce dependence on permanently fixed calibration parameters;
- adapt the model to different machining contexts;
- exploit feedback from newly observed tool trajectories;
- progressively refine prediction behavior;
- distinguish different degradation patterns associated with different operating conditions;
- maintain model relevance when production conditions evolve.

This perspective could therefore transform the current static analytical framework into a continuously improving decision-support system.

Nevertheless, reinforcement learning should only be introduced after establishing a sufficiently reliable data pipeline and a robust validation protocol. An RL agent requires a clearly defined state representation, action space and reward function. In addition, inappropriate adaptation could degrade the quality of an already validated model. For this reason, the future adaptive mechanism should initially operate in an offline or supervised validation environment before being considered for direct industrial control.

Consequently, reinforcement learning constitutes a long-term research perspective built on top of the validated Layer 1 and Layer 2 architecture rather than a component implemented in the present project.

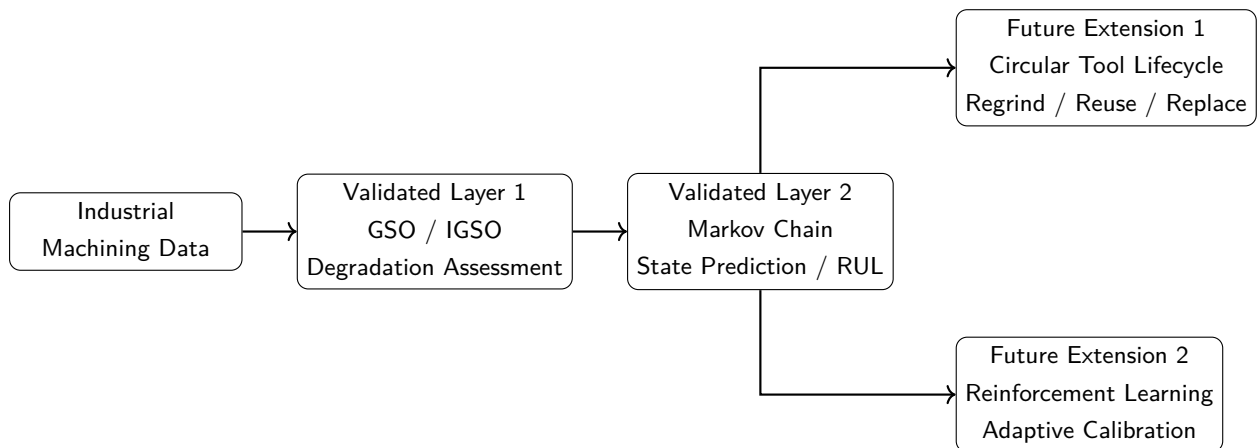


Figure 5.3: Evolution of the developed platform toward advanced tool lifecycle management and adaptive intelligence.

5.6.4 Extension to Multi-Machine and Multi-Tool Monitoring

The database and application structure provide a basis for extending the monitoring system to a larger production environment.

Future development could include simultaneous monitoring of several machines and multiple tool families while preserving the individual history of each physical tool.

Such an extension would allow the platform to evolve from an individual tool-monitoring application toward a more comprehensive production decision-support system.

The analytical parameters could also be specialized according to tool family, machining operation or machine characteristics, while maintaining the same general software architecture.

5.7 Conclusion

This chapter discussed the industrial meaning, validation requirements and development perspectives of the implemented cutting-tool monitoring platform.

The developed solution provides a structured mechanism for centralizing tool-related information, assessing machining-session severity, monitoring tool degradation and predicting its future evolution. Its main contribution lies in connecting production data with interpretable condition indicators and predictive information that can support tool-management decisions.

The analysis also highlighted that industrial deployment requires more than technical implementation. Data quality, historical information, model calibration and comparison with physical tool observations are essential for establishing the reliability of the proposed approach.

The current implementation therefore constitutes a technological and analytical foundation for a broader industrial solution. Future developments can progressively extend this foundation through integration with industrial information systems, circular tool lifecycle management, adaptive calibration and multi-machine monitoring. In particular, Circular Economy and Reinforcement Learning represent complementary extensions that could transform the current monitoring and prediction platform into a more adaptive and lifecycle-oriented decision-support system.

General Conclusion

This engineering internship, conducted within Figeac Aéro Tunisia (FGAT), concludes the design and deployment of a web-based platform dedicated to cutting tool life tracking and predictive degradation assessment. Faced with the strict quality standards of high-precision aerospace manufacturing, replacing conventional, experience-based tool replacement practices with a structured, data-driven methodology represented a crucial operational objective. Through a systematic engineering approach, the project successfully bridges the gap between theoretical tool degradation models and practical shop-floor decision support.

The primary outcome of this work is an integrated digital solution that unifies data persistence, analytical calculations, and stochastic predictive modeling. By structuring machining session data into a relational PostgreSQL database and implementing a two-layer analytical engine, the platform enables real-time tracking of cumulative tool wear alongside Markov-based predictions of remaining tool life. The developed Flask backend and intuitive web interface allow engineers and operators to systematically record machining conditions, evaluate global severity indices, and make informed decisions regarding regrinding or tool replacement before catastrophic failure occurs.

Beyond the software deliverable, this project brings concrete industrial value to FGAT by promoting standardized tool tracking, reducing premature tool disposals, and mitigating machine downtime risks associated with unexpected tool breakage. The deployment of this platform establishes a foundation for predictive maintenance within the facility, transforming raw machining parameters into actionable operational intelligence.

Nevertheless, certain limitations remain to be addressed in future iterations. To further enhance system performance, direct integration with CNC machine controllers via IoT protocols could automate data acquisition, eliminating manual inputs. Additionally, as historical usage data accumulates, the current Markovian prediction models could be augmented with machine learning algorithms to further refine lifetime forecasts under complex machining environments.

On a personal level, completing this project at Figeac Aéro Tunisia has been a highly rewarding experience. It provided a valuable opportunity to apply full-stack software development, database architecture, and mathematical modeling within a demanding industrial setting, while gaining deep practical insight into aerospace manufacturing processes and engineering project management.

Webliography

1. **Figeac Aéro Group** – Official website of the Figeac Aéro Group, a leading global partner for aerospace structural parts and precision machining.
URL: <https://www.figeac-aero.com/en/>
2. **Figeac Aéro Tunisia (FGAT)** – Industrial overview and facility details for the Tunisian manufacturing site.
URL: <https://www.figeac-aero.com/en/group/locations/tunisia>
3. **Flask Documentation** – Official documentation of the Python web framework used for developing the RESTful backend platform.
URL: <https://flask.palletsprojects.com/>
4. **SQLAlchemy ORM Documentation** – Reference guide for Object-Relational Mapping and database abstraction with PostgreSQL.
URL: <https://www.sqlalchemy.org/>
5. **PostgreSQL Global Development Group** – Official technical documentation for the PostgreSQL relational database management system.
URL: <https://www.postgresql.org/docs/>
6. **Sandvik Coromant – Cutting Tool Wear and Maintenance** – Industrial technical guide on cutting tool wear mechanisms and machining parameters.
URL: <https://www.sandvik.coromant.com/en-gb/knowledge/machining-formulas-definitions/pages/tool-wear.aspx>

Bibliography

1. **Trent, E. M., & Wright, P. K.** (2000). *Metal Cutting* (4th ed.). Butterworth-Heinemann.
(Fundamental reference on cutting tool wear mechanisms, cutting forces, and workpiece material behavior).
2. **Altintas, Y.** (2012). *Manufacturing Automation: Metal Cutting Mechanics, Machine Tool Vibrations, and CNC Design* (2nd ed.). Cambridge University Press.
(Core reference for analytical modeling of metal cutting processes and machining severity estimation).
3. **Si, X. S., Wang, W., Hu, C. H., & Zhou, D. H.** (2011). *Remaining useful life estimation A review on statistical data-driven approaches*. European Journal of Operational Research, 213(1), 1–14.
(Key survey paper on statistical approaches for predicting Remaining Useful Life (RUL) in industrial systems).
4. **Grall, A., Bérenguer, C., & Dieulle, L.** (2002). *A condition-based maintenance policy for stochastically deteriorating systems*. Reliability Engineering & System Safety, 76(2), 167–180.
(Research paper addressing stochastic degradation modeling and condition-based maintenance strategies).
5. **Norris, J. R.** (1998). *Markov Chains*. Cambridge University Press.
(Theoretical mathematical reference for Markov chain modeling, stochastic transition probabilities, and state prediction).
6. **Grinberg, M.** (2018). *Flask Web Development: Developing Web Applications with Python* (2nd ed.). O'Reilly Media.
(Comprehensive guide for structuring production-grade Flask web applications, database ORM integration, and RESTful APIs).